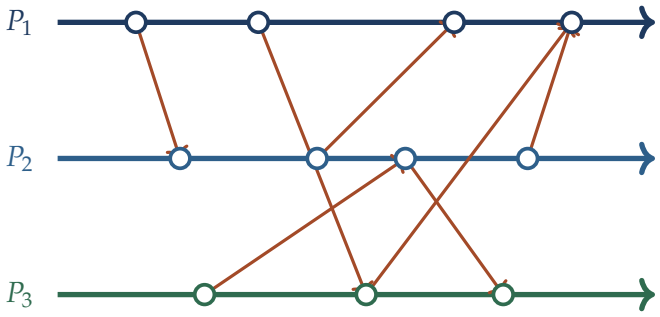
Informatik von Grund auf — Band 9

Die Einigung der Getrennten

Eine vollständige Einführung in Netzwerke und verteilte Systeme



geschieht vor

Vom Bitstrom bis zum Konsens

Jeder Schritt erklärt. Mit vollständig durchgerechneten Übungsaufgaben.

Informatik von Grund auf — Band 9
Die Einigung der Getrennten · Netzwerke und verteilte Systeme

Herausgeber und Urheber: Jürgen Maier
Entstanden 2026 in Zusammenarbeit mit Claude (Anthropic); Konzeption, Vorgaben, Prüfung und Herausgeberschaft: Jürgen Maier.

Lizenz
Dieses Werk ist lizenziert unter einer Creative-Commons-Lizenz **Namensnennung – Weitergabe unter gleichen Bedingungen 4.0 International (CC BY-SA 4.0)**. Sie dürfen das Werk vervielfältigen, verbreiten, bearbeiten und auch kommerziell nutzen, sofern Sie den Urheber nennen und Bearbeitungen unter derselben Lizenz weitergeben.
Lizenztext: `https://creativecommons.org/licenses/by-sa/4.0/legalcode.de`

Quellen und Fassungen
Quelltexte, Fehlermeldungen und aktuelle Fassungen aller Bände:
`https://github.com/juergenmaier2026/informatik-von-grund-auf`
Dauerhaft archiviert bei Zenodo: `https://doi.org/10.5281/zenodo.23773283`
Schwesterreihe: *Mathematik von Grund auf*, `https://doi.org/10.5281/zenodo.23698547`

Zitiervorschlag
Maier, J. (2026). *Informatik von Grund auf, Band 9: Die Einigung der Getrennten*. Fassung 1.0.1. `https://doi.org/10.5281/zenodo.23773283`

Satz: LaTeX mit Palatino. Fassung 1.0.1, Oktober 2026. 166 Seiten.

Stand der Forschung: August 2026

Inhaltsverzeichnis

Vorwort 1

I Das Netz 9

1 Warum Schichten 11

2 Die unteren Schichten 21

3 Adressen und Wegewahl 31

4 Transport 41

5 Namen und Anwendungen 51

II Verteiltes Rechnen 61

6 Was es schwer macht 63

7 Zeit und Ordnung 71

8 Kopien und Konsistenz 79

9 Konsens 87

10 Wenn Knoten lügen 95

III Bauen im Großen 103

11 Daten verteilen 105

12 Nachrichten und Ströme 113

13 Ausfälle und Betrieb 123

14 Leistung im Verteilten 131

15 Der Stand im Jahr 2026 141

IV Anhang **147**

A Zahlen und Formeln **149**

B Landkarte des Bandes **153**

C Glossar **155**

D Zeittafel **157**

E Kommentierte Quellen **159**

Vorwort: Das Netz ist nicht zuverlässig

Dieser Band handelt von Rechnern, die zusammenarbeiten sollen und einander nicht sehen können. *Sie laufen nicht gleichzeitig, sie erfahren nichts voneinander, wenn einer ausfällt, und sie können sich über nichts einigen, ohne miteinander zu reden.*

Ein Aufruf, der nicht zurückkehrt

Der einfachste Unterschied zwischen einem lokalen und einem entfernten Aufruf lässt sich in einer Zeile ausdrücken.

	lokal	**entfernt**
Mögliche Ausgänge	Erfolg, Ausnahme	Erfolg, Fehler, **keine Antwort**
Dauer	Nanosekunden	Millisekunden bis nie
Teilausfall	unmöglich	der Normalfall

Die dritte Zeile ist der Kern des Fachs. Ein lokales Programm läuft ganz oder gar nicht. Ein verteiltes System kann zur Hälfte laufen, und der laufende Teil weiß nicht, welcher andere Teil nicht mehr läuft.

Der Fall, an dem sich alles zeigt.
Ein Dienst schickt eine Anfrage und bekommt keine Antwort. Was ist geschehen?

1. Die Anfrage ging unterwegs verloren.
2. Der Empfänger war ausgefallen, bevor er sie las.
3. Er hat sie ausgeführt und ist danach ausgefallen.
4. Er hat sie ausgeführt, und die Antwort ging verloren.
5. Alles ist in Ordnung, es dauert nur länger als erwartet.

Diese fünf Fälle lassen sich vom Absender aus nicht *unterscheiden. In den Fällen 1, 2 und 5 wäre eine Wiederholung richtig; in den Fällen 3 und 4 würde sie den Vorgang ein zweites Mal ausführen.*

Entscheidung	Risiko	Beispiel, wo es zählt
Nicht wiederholen	Vorgang verloren	Bestellung geht unter
Wiederholen	doppelt ausgeführt	**zweimal abgebucht**

Es gibt keinen dritten Weg. Man kann die Ungewissheit nicht beseitigen, sondern nur verschieben: Man macht den Vorgang so, dass eine zweite Ausführung nichts ändert. Kapitel 12 behandelt das unter dem Namen Idempotenz.

Die acht Trugschlüsse

PETER DEUTSCH und andere sammelten in den 1990er Jahren bei Sun Microsystems eine Liste von Annahmen, die jeder Neuling im verteilten Rechnen macht und die alle falsch sind.

	Die Annahme	Die Wirklichkeit
1.	Das Netz ist zuverlässig	Pakete gehen verloren
2.	Die Verzögerung ist null	Lichtgeschwindigkeit
3.	Die Bandbreite ist unendlich	Sie kostet Geld
4.	Das Netz ist sicher	Band 8
5.	Die Aufteilung ändert sich nicht	Ständig
6.	Es gibt einen Verwalter	Es sind viele
7.	Der Transport ist kostenlos	Serialisieren kostet
8.	Das Netz ist einheitlich	Nie

Die Liste ist über dreißig Jahre alt und wird jedes Jahr aufs Neue bestätigt. Sie ist kein Spott über Anfänger, sondern eine Beschreibung dessen, was passiert, wenn man ein lokales Programmiermodell auf ein Netz überträgt.

Der zweite Punkt in Zahlen.

Strecke	Umlaufzeit	Bemerkung
Innerhalb eines Rechners	≈ 0,1 µs	Band 5
Im selben Rechenzentrum	≈ 0,5 ms	
Frankfurt–Amsterdam	≈ 10 ms	
Frankfurt–New York	≈ 80 ms	
Frankfurt–Sydney	≈ 250 ms	
Über Satellit (geostationär)	≈ 600 ms	Physik

Die letzten Zeilen lassen sich durch keine Technik verbessern. Licht braucht im Glas etwa fünf Mikrosekunden je Kilometer, und Frankfurt liegt von Sydney gut 16 000 Kilometer entfernt. Ein Protokoll, das drei Umläufe braucht, kostet dorthin eine Dreiviertelsekunde, gleichgültig wie schnell die Rechner sind.

Verbindungsaufbau	Umläufe	nach Sydney
TCP allein	1	250 ms
TCP + TLS 1.2	3	750 ms
TCP + TLS 1.3	2	500 ms
QUIC, erste Verbindung	1	250 ms
QUIC, Wiederaufnahme	**0**	≈ 0

Der ganze Aufwand um QUIC und TLS 1.3 hat einen einzigen Zweck: Umläufe einzusparen. Bandbreite lässt sich kaufen, Lichtgeschwindigkeit nicht.

Was dieses Buch will

Erstens: Die beiden Hälften zusammenbringen.

Netzwerktechnik und verteilte Algorithmen werden meist getrennt gelehrt. Das ist verständlich und führt dazu, dass man die eine Seite kennt und die andere nicht. Beide Teile stehen hier in einem Band, weil sie dieselben Fragen von zwei Seiten behandeln.

Zweitens: Die Unmöglichkeitssätze ernst nehmen.

> *Es gibt in diesem Fach eine Handvoll Sätze, die sagen, was nicht geht. Sie sind wichtiger als die Verfahren, denn sie erklären, warum die Verfahren so aussehen, wie sie aussehen. Kapitel 9 beweist den bekanntesten davon.*

Drittens: Die Zahlen mitliefern.

Verteilte Systeme werden von Größenordnungen bestimmt: Umlaufzeiten, Ausfallraten, Warteschlangenlängen. Ich rechne sie durch, weil die meisten Entwurfsfehler daher rühren, dass jemand sie nicht überschlagen hat.

Drei Warnungen

Erstens: Verteilen Sie nur, wenn Sie müssen.
Ein verteiltes System ist um Größenordnungen schwerer zu bauen, zu betreiben und zu verstehen als ein einzelner Rechner. Ein heutiger Server hat hunderte Kerne und Terabytes Speicher.

> *Die Gründe zu verteilen sind: Es passt nicht auf eine Maschine, es soll einen Maschinenausfall überstehen, oder es muss geographisch nah bei den Benutzern liegen. Alles andere ist meist eine Modeerscheinung.*

Zweitens: Fast alles ist ein Kompromiss zwischen Konsistenz und Verfügbarkeit.
Kapitel 8 zeigt, dass man bei einer Netztrennung nicht beides haben kann. Welche Seite man aufgibt, ist eine Entscheidung über den Anwendungsfall und keine technische Frage.

Drittens: Die Zahlen altern, die Sätze nicht.

> *Bandbreiten und Latenzen ändern sich. Der Satz von Fischer, Lynch und Paterson von 1985 gilt unverändert, und er wird gelten, solange es Netze gibt, in denen Nachrichten sich verzögern können.*

Zum Aufbau

- **Teil I** (Kap. 1–5): Das Netz. Schichten, Bitübertragung, Wegewahl, Transport, Namen und Anwendungen.
- **Teil II** (Kap. 6–10): Verteiltes Rechnen. Was es schwer macht, Zeit und Ordnung, Konsistenzmodelle, Konsens, byzantinische Fehler.
- **Teil III** (Kap. 11–15): Bauen im Großen. Datenverteilung, Nachrichtenströme, Ausfälle und Betrieb, Leistung, und der Stand im Jahr 2026.

Ein Netz, das jeden Verlust überlebt

Anfang der 1960er Jahre stellte sich in den Vereinigten Staaten eine militärische Frage: Wie baut man ein Nachrichtennetz, das den Ausfall großer Teile übersteht?

Aufbau	Eigenschaft	bei Ausfall
Sternförmig	einfach	Mitte weg, alles weg
Baumförmig	sparsam	ein Ast weg, Teil weg
Vermascht	aufwendig	**es gibt Umwege**

Paul Baran schlug 1964 ein vermaschtes Netz vor, in dem Nachrichten in kleine Stücke zerlegt werden, die einzeln ihren Weg suchen. Donald Davies kam in England unabhängig auf dasselbe und prägte den Namen *packet switching*.

> *Die Telefongesellschaften hielten die Idee für unsinnig. Ihr Netz beruhte auf durchgeschalteten Leitungen: Wer telefoniert, bekommt eine Verbindung für sich allein, von Anfang bis Ende. Nachrichten in Stücke zu zerlegen und diese unabhängig laufen zu lassen, erschien als Rückschritt.*

	Leitungs-vermittlung	Paketvermittlung
Ressource	für die Dauer reserviert	**geteilt**
Ungenutzte Zeit	verfällt	andere nutzen sie
Verzögerung	gleichmäßig	schwankend
Bei Ausfall	Verbindung weg	Umweg
Zustand im Netz	je Verbindung	keiner

Die letzte Zeile ist der eigentliche Einfall. Ein Vermittlungsknoten muss sich nichts über laufende Verbindungen merken. Er sieht ein Paket, schaut nach, wohin es soll, und gibt es weiter.

> *Daraus folgt, dass ein ausgefallener Knoten nichts mitreißt. Die nächsten Pakete nehmen einen anderen Weg, und niemand muss davon in Kenntnis gesetzt werden. Der Preis ist, dass die Reihenfolge und die pünktliche Ankunft nicht mehr garantiert sind.*

Das ARPANET. Am 29. Oktober 1969 wurde die erste Nachricht von der UCLA zum Stanford Research Institute geschickt. Sie sollte `LOGIN` lauten. Nach den ersten beiden Buchstaben stürzte das System ab.

Die erste Nachricht des Internets war also `LO`. Eine Stunde später klappte es.

1974, Cerf und Kahn. VINTON CERF und ROBERT KAHN veröffentlichten das Protokoll, aus dem TCP/IP hervorging. Ihr entscheidender Beitrag war nicht das Netz selbst, sondern die Verbindung verschiedener Netze mit unterschiedlicher Technik zu einem Netz der Netze.

Der Gedanke	Jedes Teilnetz bleibt, wie es ist; die Verbindung geschieht über eine gemeinsame Schicht darüber
Die Folge	Man muss sich nicht auf eine Technik einigen, sondern nur auf ein Format

Genau das ist der Grund, warum das Internet gewachsen ist und die konkurrierenden, technisch durchdachteren Entwürfe der Telekommunikationsbranche nicht. Es verlangte von niemandem, etwas aufzugeben.

Sechzig Jahre Netze und verteilte Systeme

1964	Baran, Davies	Paketvermittlung
1969	ARPANET	erste vier Knoten
1973	Metcalfe	**Ethernet**
1974	**Cerf, Kahn**	**TCP: ein Netz der Netze**
1978	**Lamport**	**Zeit, Uhren, Ordnung von Ereignissen**
1981	IPv4	
1982	Lamport u. a.	Byzantinische Generäle
1983	ARPANET auf TCP/IP	Stichtag 1. Januar
1984	**Saltzer, Reed, Clark**	**Das Ende-zu-Ende-Argument**
1984	Mockapetris	DNS
1985	**Fischer, Lynch, Paterson**	**Konsens ist unmöglich**
1986	Netzzusammenbruch	Durchsatz fällt um Faktor 1000
1988	**Jacobson**	**Staukontrolle in TCP**
1989	Berners-Lee	Web
1990er	Deutsch u. a.	Die acht Trugschlüsse
1996	Chandra, Toueg	Ausfalldetektoren
1997	Karger u. a.	konsistentes Hashing
1998	**Lamport**	**Paxos**

2000	**Brewer**	**CAP-Vermutung**
2002	Gilbert, Lynch	CAP bewiesen
2004	Dean, Ghemawat	MapReduce
2006	Chang u. a.	Bigtable; Chubby
2007	**DeCandia u. a.**	**Dynamo: Verfügbarkeit vor Konsistenz**
2008	Nakamoto	Bitcoin
2011	Shapiro u. a.	konfliktfreie Datentypen
2012	**Corbett u. a.**	**Spanner: Uhren mit Fehlerschranke**
2013	Dean, Barroso	Die Schwanzlatenz
2014	Ongaro, Ousterhout	Raft
2015	Kubernetes	
2018	TLS 1.3	
2021	**QUIC**	**Transport neu gedacht**
2026	—	Latenz bleibt das Maß aller Dinge

Zwei Zeilen verdienen eine Anmerkung.
1985, Fischer, Lynch, Paterson. Sie bewiesen, dass in einem asynchronen System bereits ein einziger möglicher Ausfall genügt, um Konsens unmöglich zu machen. Der Satz klingt vernichtend und ist es nicht; Kapitel 9 zeigt, warum Paxos und Raft trotzdem funktionieren.
1986, der Netzzusammenbruch. Zwischen Berkeley und dem Lawrence Berkeley Laboratory, gut vierhundert Meter entfernt, fiel der Durchsatz von 32 kbit/s auf 40 bit/s. VAN JACOBSON untersuchte das und entwickelte daraus die Staukontrolle, die TCP bis heute prägt (Kapitel 4).

Ein verteiltes System ist eines, bei dem der Ausfall eines Rechners, von dem Sie nie gehört haben, Ihre eigene Arbeit unmöglich machen kann.

— LESLIE LAMPORT, in einer Notiz von 1987. *Der Satz war als Scherz gemeint und ist die beste Definition des Fachs geblieben.*

Part I

Das Netz

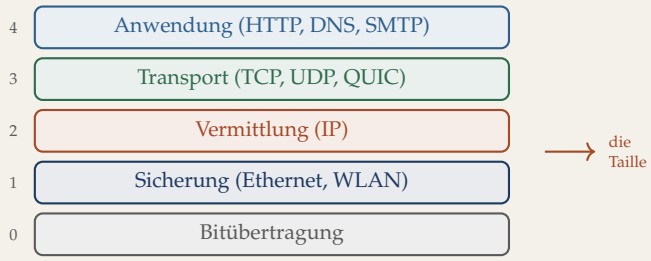
1 Warum Schichten

Das Problem der Vielfalt

Ein Netz muss sehr verschiedene Techniken verbinden und sehr verschiedene Anwendungen tragen.

Unten	Oben	
Kupfer, Glas, Funk	Web, Post, Video	
m Techniken	n Anwendungen	$m \cdot n$ Kombinationen

Es ist dieselbe Rechnung wie beim Übersetzerbau in Band 6. Eine Zwischenschicht in der Mitte macht aus $m \cdot n$ Aufgaben $m + n$. Dort hieß sie Zwischenform, hier heißt sie IP.



Die Sanduhr.

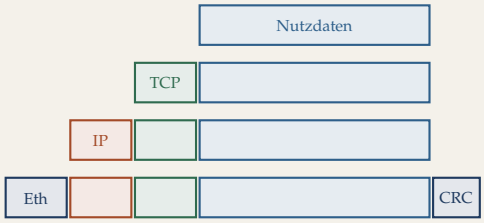
Unten viele	Ethernet, WLAN, Mobilfunk, Glasfaser, Satellit
In der Mitte eines	**IP**
Oben viele	HTTP, DNS, SMTP, SSH, alles Weitere

Die schmale Stelle in der Mitte ist der Grund für den Erfolg des Internets. Wer eine neue Übertragungstechnik erfindet, muss nur IP darauf abbilden. Wer eine neue Anwendung erfindet, muss nur IP benutzen. Niemand muss mit den anderen reden.

Die Kehrseite: IP selbst zu ändern ist außerordentlich schwer. Die Umstellung auf IPv6 begann 1998 und ist 2026 noch nicht abgeschlossen. Was in der Taille steckt, ist praktisch unbeweglich.

Kapselung

Jede Schicht behandelt das, was von oben kommt, als undurchsichtige Nutzlast und hängt einen eigenen Kopf an.



Schicht	Kopf	Aufgabe des Kopfes
Ethernet	14 B + 4 B	Nachbar im gleichen Netz
IPv4	20 B	Ziel im ganzen Netz
IPv6	40 B	dasselbe, größere Adressen
TCP	20–60 B	Anschluss, Reihenfolge, Stau
UDP	8 B	nur Anschluss

Bei einem typischen Ethernet-Rahmen mit 1500 Byte Nutzlast machen die Köpfe zusammen etwa 58 Byte aus, also unter vier Prozent. Bei einem einzelnen Tastendruck über SSH sind es 58 Byte Kopf für ein Byte Nutzlast.

Nutzlast	Gesamt	Wirkungsgrad
1460 B	1518 B	96,2 %
512 B	570 B	89,8 %
64 B	122 B	52,5 %
1 B	59 B	1,7 %

Deshalb sammeln Protokolle kleine Sendungen, wo sie können. Der Nagle-Algorithmus in TCP tut genau das, und er ist zugleich eine häufige Ursache für unerwartete Verzögerungen (Kapitel 4).

Das Ende-zu-Ende-Argument

1984 formulierten JEROME SALTZER, DAVID REED und DAVID CLARK einen Grundsatz, der bestimmt, welche Aufgabe in welche Schicht gehört.

> *Eine Funktion, deren Richtigkeit nur an den Endpunkten festgestellt werden kann, gehört auch an die Endpunkte. Eine Umsetzung in der Mitte kann sie nicht vollständig leisten und ist deshalb bestenfalls eine Beschleunigung.*

Das Beispiel aus dem Aufsatz.
Eine Datei soll über ein Netz übertragen werden. Wo prüft man auf Fehler?

1. Beim Lesen von der Platte kann ein Fehler entstehen.
2. Im Speicher des Absenders ebenfalls.
3. Auf jeder einzelnen Teilstrecke.
4. In jedem Vermittlungsknoten dazwischen.
5. Im Speicher des Empfängers.
6. Beim Schreiben auf dessen Platte.

Eine Prüfung je Teilstrecke deckt nur Punkt 3 ab. Nur eine Prüfung, die die Datei beim Absender liest und beim Empfänger vergleicht, deckt alles ab. Also braucht man sie ohnehin, und dann sind die Prüfungen dazwischen keine Notwendigkeit mehr.

Warum es sie trotzdem gibt.

Nicht für die Korrektheit	Die garantieren die Enden
Für die Leistung	Ein Fehler auf einer schlechten Funkstrecke wird dort behoben, statt eine Wiederholung über die ganze Strecke auszulösen

Fehlerrate je Strecke	ohne Prüfung dort	mit
10^{-9}	irrelevant	unnötig
10^{-4}	merkbar	sinnvoll
10^{-2} (Funk)	unbrauchbar	notwendig

Deshalb wiederholt WLAN verlorene Rahmen selbst, obwohl TCP das auch täte. Ohne diese lokale Wiederholung würde TCP jeden Verlust als Stau deuten und die Senderate drosseln, was genau falsch wäre.

Wo der Grundsatz gebrochen wurde

Das Internet hat sich seit den 1990er Jahren von diesem Ideal entfernt.

Adressumsetzung	Ein Gerät in der Mitte schreibt Adressen und Anschlussnummern um und muss sich Zustand merken
Firewalls	Entscheiden anhand des Inhalts, was durchkommt
Zwischenspeicher	Liefern Antworten, die nie vom Ziel kamen
Optimierer	Verändern Protokolle unterwegs

Alle vier verletzen die Annahme, dass das Netz Pakete nur weiterreicht. Die Folge ist, dass sich neue Transportprotokolle kaum noch einführen lassen: Geräte in der Mitte kennen sie nicht und verwerfen sie.

Protokoll	Jahr	Verbreitung
SCTP	2000	gering; oft blockiert
DCCP	2006	sehr gering
QUIC	**2021**	**hoch**

QUIC hat die Lehre gezogen und setzt auf UDP auf, das überall durchkommt, und verschlüsselt fast den gesamten Kopf. Damit können Geräte in der Mitte nichts mehr verändern, weil sie nichts mehr sehen. Es ist die Rückeroberung des Ende-zu-Ende-Grundsatzes mit den Mitteln aus Band 8.

Aufgabe 1.1.

(a) Verfolgen Sie ein Paket durch alle Schichten.
(b) Rechnen Sie den Aufwand der Kapselung für verschiedene Lasten aus.
(c) Untersuchen Sie, wann eine Aufgabe in die Mitte gehört.

Lösung Schritt für Schritt.

(a) Ein Paket durch alle Schichten.
Der Fall. Ein Browser lädt eine Seite von einem Server in einem anderen Netz.

Beim Absender, von oben nach unten.

1. Die Anwendung erzeugt eine HTTP-Anfrage von etwa 400 Byte.
2. TLS verschlüsselt sie und hängt einen eigenen Kopf an.
3. TCP hängt Anschlussnummern, Folgenummer und Prüfsumme an.
4. IP hängt Absender- und Zieladresse an.
5. Ethernet hängt die Adressen des **nächsten** Geräts an.

Punkt 5 ist die wichtigste Unterscheidung des Kapitels. Die IP-Adresse benennt das Ziel; die Ethernet-Adresse benennt nur den nächsten Schritt. Sie wird bei jedem Zwischenknoten neu gesetzt.

Unterwegs.

Gerät	schaut auf	ändert
Switch	Ethernet-Adresse	nichts
Router	**IP-Adresse**	Ethernet-Köpfe, TTL
NAT-Gerät	IP und Anschluss	Adresse, Anschluss, Prüfsumme
Firewall	alles bis TCP	meist nichts

Der Zähler, der Schleifen verhindert.
Jeder Router verringert das Feld TTL um eins. Bei null wird das Paket verworfen und eine Meldung zurückgeschickt.

Anfangswert	typisch
Linux	64
Windows	128
Router	255

Genau darauf beruht `traceroute`: Man schickt Pakete mit TTL 1, 2, 3 und so fort und sammelt die Fehlermeldungen ein. Jede stammt von einem anderen Router auf dem Weg. Ein Diagnosewerkzeug, das eine Sicherheitsvorkehrung zweckentfremdet.

(b) Der Aufwand der Kapselung.
Fall 1 – Dateiübertragung.

Anteil	**Byte**	
Nutzdaten	1460	
TCP	20	
IP	20	
Ethernet	18	mit Prüfsumme
Präambel und Lücke	20	auf dem Draht
Gesamt	1538	**Wirkungsgrad** 94,9 %

Fall 2 – Ein Tastendruck über SSH.

Nutzdaten	1	
SSH und Verschlüsselung	≈ 36	mit MAC
TCP, IP, Ethernet	58	
Auffüllung auf Mindestlänge	—	Rahmen ≥ 64 B
Gesamt	≈ 115	**Wirkungsgrad** 0,9 %

Und dazu kommt die Antwort: Der Server schickt das Zeichen zurück, damit es auf dem Bildschirm erscheint. Ein Tastendruck erzeugt also etwa 230 Byte Verkehr und einen vollen Netzumlauf.

Fall 3 – Sprachübertragung.

Nutzdaten je Paket	20 B	20 ms Sprache
RTP, UDP, IP, Ethernet	54 B	
Gesamt	74 B	Wirkungsgrad 27 %
Rate mit Kopf	29,6 kbit/s	bei 8 kbit/s Sprache

Der Kopf kostet das Dreifache der Nutzlast. Deshalb komprimiert man bei Funkstrecken die Köpfe: Sie ändern sich von Paket zu Paket kaum, und man überträgt nur die Unterschiede. Aus 54 Byte werden so etwa drei.

> *Das ist eine Anwendung des Gedankens aus Band 7: Redundanz entfernen, indem man vorhersagt. Hier ist der Vorhersager trivial, weil der nächste Kopf fast immer der vorige mit einem hochgezählten Zähler ist.*

(c) Wann gehört eine Aufgabe in die Mitte?
Die Prüffrage. Kann die Mitte die Aufgabe *vollständig* lösen?

Aufgabe	**Mitte reicht?**	**Wo sie hingehört**
Wegewahl	ja	**Mitte**
Stauvermeidung	teilweise	Mitte meldet, Enden reagieren
Fehlerkorrektur	nein	Enden, örtlich als Beschleunigung
Verschlüsselung	nein	**Enden**
Reihenfolge	nein	Enden
Zugangskontrolle	teilweise	beides

Die zweite Zeile im Detail.
Stau entsteht in der Mitte, und nur die Mitte sieht ihn. Handeln können aber nur die Absender, denn nur sie können langsamer senden.

Klassisch	Der Router verwirft Pakete; der Absender schließt aus dem Verlust auf Stau
Mit Markierung	Der Router setzt ein Bit im Kopf, statt zu verwerfen; der Empfänger meldet es zurück

Die zweite Zeile ist offensichtlich besser: Man erfährt vom Stau, ohne Daten zu verlieren. Sie ist seit 2001 genormt und wurde jahrelang kaum eingesetzt, weil einzelne Geräte in der Mitte Pakete mit gesetztem Bit verwarfen. Erst seit etwa 2015 verbreitet sie sich.

Der Fall ist lehrreich für das ganze Kapitel. Eine bessere Lösung setzt sich nicht durch, weil sie besser ist, sondern nur, wenn sie sich schrittweise einführen lässt, ohne dass jemand etwas ändern muss.

Saltzer, Reed und Clark[25] formulierten das Ende-zu-Ende-Argument; Cerf und Kahn[5] entwarfen die Sanduhr; Kurose und Ross[17] führen die Schichten mit vielen Beispielen ein.

2 Die unteren Schichten

Vom Signal zum Bit

Ganz unten steht ein physikalischer Vorgang: eine Spannung, ein Lichtimpuls, eine Funkwelle. Daraus muss eine Folge von Bits werden.

Taktrückgewinnung	Der Empfänger muss wissen, wann ein Bit endet und das nächste beginnt
Gleichanteil	Lange Folgen gleicher Werte stören die Übertragung
Rahmen finden	Wo beginnt eine Nachricht, wo hört sie auf

Die erste Zeile ist die grundlegendste. Sender und Empfänger haben getrennte Uhren, die niemals exakt gleich laufen. Bei tausend Bit ohne Flanke hat sich der Empfänger unweigerlich verzählt.

Die Lösung: Flanken erzwingen.

Verfahren	Vorgehen	Wirkungsgrad
Manchester	jedes Bit als Flanke	50 %
4B/5B	je 4 Bit werden 5	80 %
8B/10B	je 8 Bit werden 10	80 %
64B/66B	je 64 Bit werden 66	97 %
Verwürfelung	mit Pseudozufall verknüpfen	$\approx$ 100 %

Die Reihe zeigt eine Entwicklung: Je schneller die Übertragung wurde, desto teurer wurde es, die Hälfte der Bandbreite für die Taktrückgewinnung zu opfern. Bei zehn Gigabit je Sekunde begnügt man sich mit zwei Zusatzbit auf vierundsechzig.

Die letzte Zeile ist ein Anwendungsfall der Verdichtungslogik aus Band 7 mit umgekehrtem Vorzeichen: Man verknüpft die Daten mit einer Pseudozufallsfolge, um sie zufälliger *zu machen, damit lange gleichförmige Abschnitte verschwinden.*

Wer darf senden?

Teilen sich mehrere Geräte ein Medium, so muss geregelt werden, wer wann sendet.

Verfahren	Regel	Eigenschaft
Feste Zuteilung	jeder bekommt einen Zeitschlitz	verschwendet bei Leerlauf
Reihum-Marke	wer die Marke hat, sendet	gerecht, empfindlich
Wettbewerb	einfach senden und hoffen	**einfach, gut bei geringer Last**
Zentrale Vergabe	ein Verwalter teilt zu	Verzögerung

Ethernet: hinhören, senden, mithören.

1. Ist das Medium belegt, warten.
2. Ist es frei, senden.
3. Während des Sendens mithören.
4. Weicht das Gehörte vom Gesendeten ab, liegt eine Kollision vor.
5. Sofort abbrechen, ein Störsignal senden, zufällige Zeit warten.
6. Bei wiederholter Kollision die Wartespanne verdoppeln.

Punkt 6 ist das exponentiell wachsende zufällige Zurückweichen, das in dieser Reihe schon mehrfach aufgetaucht ist: bei Sperren in Band 5, bei Datenbanktransaktionen ebendort. Es ist eine der brauchbarsten Faustregeln der Informatik.

Warum die Mindestlänge eines Rahmens 64 Byte beträgt.
Ein Sender muss noch senden, wenn die Kollision zu ihm zurückkommt. Sonst merkt er sie nicht.

Größe	Wert	
Höchste Ausdehnung	2500 m	mit Verstärkern
Ausbreitungs-geschwindigkeit	$\approx 2 \cdot 10^8$ m/s	
Einfache Laufzeit	12,5 µs	
Hin und zurück	$\approx$ 25 µs	mit Verstärkern $\approx$ 50 µs
Bei 10 Mbit/s	500 bit	
Mindestrahmen	**512 bit** = 64 B	

Die 64 Byte sind also keine willkürliche Zahl, sondern folgen aus Kabellänge und Datenrate. Als Ethernet auf 100 Mbit/s und dann auf 1 Gbit/s beschleunigt wurde, musste man entweder die Kabel kürzen oder die Rahmen verlängern.

Rate	Ausdehnung	Mindestrahmen	
10 Mbit/s	2500 m	64 B	
100 Mbit/s	205 m	64 B	Kabel gekürzt
1 Gbit/s	200 m	512 B	**aufgefüllt**
10 Gbit/s	—	—	keine Kollisionen mehr

Die letzte Zeile beendet das Thema. Moderne Netze arbeiten geschaltet und im Vollduplexbetrieb: Jedes Gerät hat seine eigene Leitung zum Switch, und es gibt niemanden, mit dem es kollidieren könnte. Der ganze Mechanismus ist Geschichte und steckt aus Verträglichkeitsgründen noch in den Normen.

Ein Netz, benannt nach einem widerlegten Stoff

ROBERT METCALFE entwarf 1973 bei Xerox PARC ein Netz, das mehrere Rechner über ein einziges Koaxialkabel verband. Die erste Beschreibung schrieb er auf ein Memo mit einer Handzeichnung.

> *Den Namen wählte er nach dem Äther, jenem hypothetischen Medium, durch das sich Licht nach der Vorstellung des 19. Jahrhunderts ausbreiten sollte. Das Michelson-Morley-Experiment hatte es 1887 widerlegt. Metcalfe fand den Namen trotzdem passend: ein allgegenwärtiges Medium, das alles trägt.*

Metcalfe und DAVID BOGGS veröffentlichten 1976 die Beschreibung. Bemerkenswert daran ist, wie wenig davon überlebt hat und wie viel.

	1976	2026
Medium	ein Koaxialkabel	Punkt-zu-Punkt
Kollisionen	der Normalfall	keine mehr
Rate	2,94 Mbit/s	bis 800 Gbit/s
Rahmenformat	**Adressen, Typ, Daten, CRC**	**im Kern unverändert**

> *Der Zugriffsmechanismus, um den es damals ging, ist verschwunden. Das Rahmenformat ist geblieben, und über ihm läuft heute praktisch der gesamte kabelgebundene Verkehr der Welt. Ein Entwurf überlebt selten durch das, was ihn ausmachte, sondern durch das, was er nebenbei festlegte.*

Zur Konkurrenz gehörte das Token Ring von IBM, technisch in mehrfacher Hinsicht überlegen: gerechter, mit vorhersagbarer Verzögerung, ohne Kollisionen.

	Token Ring	Ethernet
Verzögerung	vorhersagbar	schwankend
Bei hoher Last	stabil	bricht ein
Aufwand	hoch	sehr gering
Preis je Anschluss	hoch	niedrig

Ethernet gewann über den Preis, und danach über die Menge. Es ist derselbe Ausgang wie beim Streit um Mikrokerne in Band 5: Der sauberere Entwurf verliert gegen den, der billig genug ist, um überall eingesetzt zu werden, und dann verbessert wird.

Aufgabe 2.1.

(a) Berechnen Sie den Durchsatz von Ethernet unter Last.
(b) Untersuchen Sie die Fehlererkennung durch zyklische Prüfung.
(c) Vergleichen Sie geteilte und geschaltete Netze.

Lösung Schritt für Schritt.

(a) Durchsatz unter Last.
Das Modell. n Geräte senden mit Wahrscheinlichkeit p je Zeitschlitz.
Die Wahrscheinlichkeit, dass genau eines sendet, also ein Erfolg vorliegt:

$$P_{\text{Erfolg}} = n\,p\,(1-p)^{n-1}.$$

Das Maximum. Ableiten nach p liefert $p = \frac{1}{n}$, und damit

$$P_{\max} = \left(1-\tfrac{1}{n}\right)^{n-1} \xrightarrow{n\to\infty} \frac{1}{e} \approx 0{,}368.$$

n	p_{opt}	$P_{\max}$	
2	0,500	0,500	
5	0,200	0,410	
10	0,100	0,387	
100	0,010	0,370	
∞	—	**0,368**	$1/e$

> *Im günstigsten Fall bleiben also nur siebenunddreißig Prozent des Mediums nutzbar. Die übrigen dreiundsechzig gehen an leere Schlitze und Kollisionen. Und das setzt voraus, dass jeder mit der optimalen Wahrscheinlichkeit sendet, was niemand weiß.*

Warum das reale Ethernet besser war.
Das Verfahren wartet nicht in Zeitschlitzen, sondern hört das Medium ab und bricht bei Kollision sofort ab.

Verfahren	Nutzbarer Anteil	
Reines Aloha	18 %	
Getaktetes Aloha	37 %	
Ethernet mit Abhören	≈ 90 %	bei langen Rahmen
Geschaltetes Ethernet	≈ 100 %	**keine Kollisionen**

Die dritte Zeile gilt nur, solange die Rahmen lang gegenüber der Laufzeit sind. Genau deshalb hat die Ausdehnung des Netzes so großen Einfluss: Bei einem langen Kabel wird eine Kollision spät bemerkt, und die verschwendete Zeit wächst.

(b) Zyklische Prüfung.
Der Gedanke. Die Nachricht wird als Polynom über $\mathbb{F}_2$ aufgefasst und durch ein festes Generatorpolynom geteilt. Der Rest wird angehängt.
Ein Beispiel von Hand. Nachricht 1101011011, Generator $G = 10011$ (Grad 4).
Schritt 1 – Anhängen von vier Nullen.

$$1101011011\,0000.$$

Schritt 2 – Division ohne Übertrag.

```
11010110110000 :  10011
10011
---
 10011
 10011
 ---
 00001 10110000
 1 0011
 ----
 1010 000
 1001 1
 -----
 011 100
 10 011
 ----
 1 1110
 1 0011
 ----
 1101     Rest
```

Schritt 3 – Das gesendete Wort.

1101011011 **1110**.

(Der Rest wird angehängt; die Probe ergibt beim Empfänger Rest null.)
Was CRC-32 leistet.

Alle Einzelfehler	sicher erkannt
Alle Doppelfehler	bei geeignetem Generator sicher
Ungerade Fehlerzahl	sicher, wenn G den Faktor $x + 1$ enthält
Bündel bis 32 bit	sicher
Längere Bündel	mit Wahrscheinlichkeit 2^{-32} übersehen

Die vierte Zeile ist der eigentliche Grund für die Wahl: Übertragungsfehler treten nicht vereinzelt auf, sondern in Bündeln, weil eine Störung mehrere Bit hintereinander trifft. Ein Verfahren, das genau dagegen wirkt, ist hier richtig gewählt.

Eine Prüfsumme ist keine Signatur

CRC schützt gegen Zufall, nicht gegen Absicht. Wer den Inhalt ändern will, kann die Prüfsumme leicht nachziehen.

	CRC	**MAC (Band 8)**
Schützt gegen	Übertragungsfehler	**Angreifer**
Braucht Schlüssel	nein	**ja**
Nachziehbar	trivial	nein

Die Verwechslung hat reale Systeme gekostet. Bei WEP, der ersten WLAN-Verschlüsselung, diente CRC-32 als Integritätsschutz. Weil CRC linear ist, ließen sich Pakete gezielt verändern und die Prüfsumme mitverändern, ohne den Schlüssel zu kennen.

(c) Geteilt oder geschaltet.
Der Unterschied.

	Hub (geteilt)	Switch (geschaltet)
Weiterleitung	an alle Anschlüsse	**nur an den richtigen**
Bandbreite	einmal für alle	je Anschluss
Kollisionen	ja	nein
Mithören	trivial	schwieriger

Die Rechnung bei 24 Geräten und 1 Gbit/s.

	Hub	Switch
Gesamtdurchsatz	$\approx$ 0,9 Gbit/s	**12 Gbit/s**
Je Gerät im Mittel	37 Mbit/s	1000 Mbit/s

Der Switch erreicht zwölf Gigabit, weil zwölf Paare gleichzeitig sprechen können. Der Gewinn liegt nicht darin, dass er schneller wäre, sondern darin, dass er Gespräche voneinander trennt.

Wie ein Switch lernt.

1. Bei jedem eingehenden Rahmen wird die Absenderadresse mit dem Anschluss vermerkt.
2. Ist das Ziel bekannt, geht der Rahmen nur dorthin.
3. Ist es unbekannt, geht er an alle anderen Anschlüsse.
4. Einträge verfallen nach einigen Minuten.

Der Switch braucht dafür keine Konfiguration und kein Protokoll. Er lernt allein durch Zuhören, und das ist der Grund, warum man ihn einfach einstecken kann. Punkt 4 sorgt dafür, dass ein umgestecktes Gerät nach kurzer Zeit wiedergefunden wird.

Punkt 3 hat allerdings eine unangenehme Kehrseite. Wer die Tabelle mit erfundenen Adressen überflutet, verdrängt die echten Einträge, und der Switch verhält sich wieder wie ein Hub. Dann kann jeder alles mitlesen.

Metcalfe und Boggs[21] beschrieben Ethernet 1976; Peterson und Davie[24] behandeln die unteren Schichten gründlich; Kurose und Ross[17] rechnen die Zugriffsverfahren vor.

3 Adressen und Wegewahl

Zwei Aufgaben, die man trennen muss

	Weiterleiten	**Wegewahl**
Frage	Wohin mit *diesem* Paket?	Welche Wege gibt es überhaupt?
Wann	bei jedem Paket	alle paar Sekunden
Wo	in der Hardware	in der Software
Zeitrahmen	Nanosekunden	Sekunden bis Minuten

Die Trennung ist der Grund, warum ein Router bei hunderten Millionen Paketen je Sekunde bleiben kann. Das Nachschlagen in der Tabelle geschieht in eigens dafür gebauten Schaltungen; das Erstellen der Tabelle geschieht nebenher auf einem gewöhnlichen Prozessor.

Adressen mit Struktur.
Eine IP-Adresse ist keine Kennung, sondern eine Ortsangabe. Sie zerfällt in einen Netzanteil und einen Geräteanteil.

Schreibweise	**Netzanteil**	**Geräte**
`192.0.2.0/24`	24 bit	254
`10.0.0.0/8`	8 bit	≈ 16,7 Mio.
`203.0.113.64/26`	26 bit	62

Die Struktur ist der eigentliche Trick. Ein Router muss sich nicht Milliarden Adressen merken, sondern nur einige hunderttausend Netze. Ohne diese Bündelung wäre das Internet nicht baubar.

Was	Anzahl	
IPv4-Adressen insgesamt	$4{,}3 \cdot 10^9$	
Einträge in der globalen Tabelle	$\approx 10^6$	Stand 2026
Autonome Systeme	$\approx 78\,000$	

Die Regel des längsten Treffers.
Mehrere Einträge können passen. Es gewinnt der mit dem längsten Netzanteil.

Eintrag	Ausgang	passt auf 10.1.2.3?
`0.0.0.0/0`	A	ja (Standardweg)
`10.0.0.0/8`	B	ja
`10.1.0.0/16`	C	ja
`10.1.2.0/24`	**D**	**ja, gewinnt**

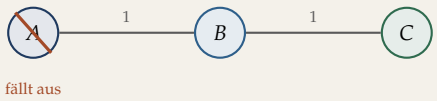
Damit lässt sich für ein Teilnetz ein Sonderweg festlegen, ohne die allgemeine Regel anzutasten. Es ist dieselbe Logik wie bei Ausnahmen in einem Regelwerk: Das Speziellere schlägt das Allgemeinere.

Zwei Familien von Wegewahlverfahren

	Distanzvektor	Zustandsverbreitung
Jeder kennt	**nur die Nachbarn**	die **ganze** Topologie
Austausch	eigene Tabelle an Nachbarn	eigene Nachbarschaft an alle
Rechnung	Bellman-Ford, verteilt	Dijkstra, lokal
Konvergenz	langsam	schnell
Speicher	gering	höher
Beispiel	RIP, EIGRP	OSPF, IS-IS

Der Unterschied ist grundsätzlich. Beim Distanzvektor sagt jeder seinen Nachbarn, wie weit er wohin kommt, und niemand hat ein Bild vom Ganzen. Bei der Zustandsverbreitung erzählt jeder allen, wer seine Nachbarn sind, und dann rechnet jeder für sich den kürzesten Weg aus.

Das Zählen bis unendlich.
Der Distanzvektor hat eine bekannte Schwäche. Fällt eine Verbindung aus, so kann sich eine falsche Auskunft im Kreis fortpflanzen.



Runde	*B* **zu** *A*	*C* **zu** *A*	
0	1	2	
1	3	2	*B glaubt C*
2	3	4	
3	5	4	
⋮	⋮	⋮	sehr langsam

B verliert die Verbindung zu A, sieht aber, dass C angeblich in zwei Schritten hinkommt, und schließt daraus auf drei. C weiß nicht, dass sein eigener Weg über B läuft. Beide zählen sich langsam hoch.

Unendlich klein setzen	Bei RIP gilt 16 als unendlich; das begrenzt die Netzgröße
Nicht zurücksagen	Man meldet einen Weg nicht demjenigen, über den er läuft
Vergiften	Man meldet ihm ausdrücklich „unendlich“

Die zweite und dritte Regel helfen bei Kreisen der Länge zwei und nicht bei längeren. Deshalb benutzen große Netze die Zustandsverbreitung, bei der das Problem gar nicht erst entsteht.

Wegewahl zwischen Betreibern

Innerhalb eines Netzes sucht man den kürzesten Weg. Zwischen Betreibern geht es um etwas anderes.

	intern	**zwischen Betreibern**
Ziel	kürzester Weg	**Geschäftsinteresse**
Maßstab	Verzögerung, Bandbreite	Verträge, Kosten
Protokoll	OSPF, IS-IS	**BGP**
Vertrauen	vollständig	keines

Die zweite Zeile überrascht viele. Ein Betreiber leitet ein Paket nicht über den schnellsten Weg, sondern über den, der ihn am wenigsten kostet. Ein Weg über einen Partner, mit dem ein kostenloser Austausch besteht, wird einem kürzeren über einen bezahlten Anbieter vorgezogen.

Beziehung	**Wer zahlt**	**Weiterleitung**
Kunde	zahlt an den Anbieter	alles
Anbieter	wird bezahlt	alles vom Kunden
Gleichrangig	niemand	**nur eigene Kunden**

Die letzte Zeile erklärt eine Beobachtung, die viele machen: Der Weg von A nach B ist manchmal absurd lang, etwa von Berlin nach Hamburg über Frankfurt und London. Er folgt den Verträgen, nicht der Landkarte.

BGP beruht auf Vertrauen, das es nicht gibt

Ein Betreiber gibt bekannt, welche Netze er erreichen kann. Wer prüft das?

Im Grundsatz	niemand
Die Folge	Wer behauptet, ein fremdes Netz zu bedienen, bekommt dessen Verkehr

2008	Ein Betreiber in Pakistan lenkt YouTube weltweit auf sich um; die Sperre im eigenen Land wird versehentlich global
2018	Verkehr zu einem Zahlungsdienst wird umgeleitet und dabei Guthaben entwendet
2021	Eine falsche Ankündigung nimmt einen großen Anbieter für Stunden vom Netz

Die Gegenmaßnahmen benutzen die Mittel aus Band 8: Die Inhaber von Adressbereichen signieren Erklärungen darüber, wer sie ankündigen darf. Die Einführung läuft seit 2012 und deckt 2026 einen wachsenden, aber nicht vollständigen Teil des Netzes ab.

Auch hier gilt die Lehre aus Kapitel 1: Eine Verbesserung setzt sich nur durch, wenn sie schrittweise einführbar ist. Wer signiert, hat zunächst keinen Vorteil, solange die anderen nicht prüfen.

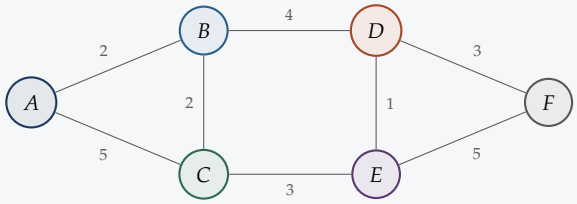
Aufgabe 3.1.

(a) Führen Sie Dijkstras Verfahren an einem Netz vollständig durch.
(b) Rechnen Sie die Erschöpfung der Adressen und ihre Umgehung nach.
(c) Untersuchen Sie, wie Verkehr gleichmäßig verteilt wird.

Lösung Schritt für Schritt.

(a) Kürzeste Wege.

Das Netz.



Der Ablauf von A aus.

Schritt	gewählt	Entfernungen danach
0	—	A:0, alles Übrige ∞
1	A (0)	B:2, C:5
2	B (2)	C:**4**, D:6
3	C (4)	E:7
4	D (6)	E:**7**, F:9
5	E (7)	F:**9** (über E wäre 12)
6	F (9)	fertig

Das Ergebnis.

Ziel	Kosten	Weg	erster Schritt
B	2	$A \to B$	B
C	4	$A \to B \to C$	B
D	6	$A \to B \to D$	B
E	7	$A \to B \to D \to E$	B
F	9	$A \to B \to D \to F$	B

In der Weiterleitungstabelle steht nur die letzte Spalte. Der Router merkt sich nicht den ganzen Weg, sondern nur den nächsten Schritt. Dass daraus zusammen ein widerspruchsfreier Weg entsteht, liegt daran, dass alle dasselbe Bild und dasselbe Verfahren benutzen.

Genau deshalb ist es gefährlich, wenn zwei Router unterschiedliche Bilder haben, etwa während einer Umstellung. Dann können Pakete im Kreis laufen, und nur der TTL-Zähler aus Kapitel 1 beendet das.

(b) Adressen und ihre Erschöpfung.
Der Vorrat.

$$2^{32} = 4\,294\,967\,296.$$

Davon sind erhebliche Teile für Sonderzwecke reserviert.

Zweck	Anteil	
Private Netze	≈ 18 Mio.	`10/8`, `172.16/12`, `192.168/16`
Rundsendung	≈ 268 Mio.	`224/4`
Reserviert	≈ 268 Mio.	`240/4`
Eigener Rechner	≈ 16,7 Mio.	`127/8`
Nutzbar	≈ 3,7 Mrd.	

Bei über acht Milliarden Menschen und mehreren Geräten je Person reicht das offensichtlich nicht. Die letzten freien Blöcke wurden 2011 vergeben.

Die Umgehung: Adressumsetzung.
Ein Gerät am Netzrand ersetzt private Adressen durch seine eigene öffentliche und merkt sich die Zuordnung.

Innen	Außen	Eintrag
`10.0.0.5:4711`	`203.0.113.9:52001`	gemerkt
`10.0.0.6:4711`	`203.0.113.9:52002`	gemerkt

Damit teilen sich beliebig viele Geräte eine Adresse, unterschieden durch die Anschlussnummer. Der Vorrat wächst rechnerisch um den Faktor 65 536, und das hat IPv4 zwei Jahrzehnte länger am Leben gehalten, als vorhergesagt war.

Der Preis.

Zustand in der Mitte	Das Gerät muss sich alle Verbindungen merken; fällt es aus, sind alle weg
Keine Verbindung von außen	Wer innen sitzt, ist von außen nicht erreichbar
Ende-zu-Ende gebrochen	Kapitel 1
Zeitüber-schreitungen	Ein Eintrag verfällt; eine ruhende Verbindung stirbt unbemerkt

Die letzte Zeile erklärt eine verbreitete Beobachtung: Eine offene, aber ungenutzte Verbindung bricht nach einigen Minuten ab. Deshalb schicken Protokolle regelmäßig Lebenszeichen, deren einziger Zweck darin besteht, einen Tabelleneintrag frisch zu halten.

Die eigentliche Lösung.

	IPv4	**IPv6**
Adresslänge	32 bit	128 bit
Anzahl	$4{,}3 \cdot 10^{9}$	$3{,}4 \cdot 10^{38}$
Je Quadrat-meter Erde	≈ 8	$\approx 6 \cdot 10^{23}$
Kopfgröße	20 B	40 B
Verbreitung 2026	—	$\approx 45\,\%$

Die letzte Zeile ist bemerkenswert. IPv6 ist seit 1998 genormt und nach achtundzwanzig Jahren bei knapp der Hälfte. Der Grund ist wieder derselbe: Wer umstellt, hat zunächst keinen Vorteil, solange die Gegenseite es nicht tut, und die Adressumsetzung nimmt den Druck.

(c) Verkehr gleichmäßig verteilen.
Die Aufgabe. Zwischen zwei Punkten gibt es vier gleichwertige Wege. Wie verteilt man?

Verfahren	Vorgehen	Problem
Reihum je Paket	bestes Gleichgewicht	Pakete überholen einander
Zufällig je Paket	dasselbe	dasselbe
Streuwert je Fluss	alle Pakete eines Flusses auf denselben Weg	**ungleich bei wenigen Flüssen**

Die dritte Zeile gewinnt, weil TCP das Überholen von Paketen als Verlust deutet und die Senderate drosselt (Kapitel 4). Ein Verfahren, das perfekt ausgleicht und dabei die Reihenfolge zerstört, verschlechtert die Lage.

Wie der Streuwert gebildet wird.

$$\text{Weg} = h(\text{Quelladresse}, \text{Zieladresse}, \text{Quellport}, \text{Zielport}, \text{Protokoll}) \bmod 4.$$

Alle Pakete einer Verbindung haben dieselben fünf Werte und nehmen deshalb denselben Weg. Die Reihenfolge bleibt erhalten.

Die Ungleichverteilung bei wenigen großen Flüssen.

Flüsse	Erwartete Abweichung	
4	sehr groß	oft alle auf einem Weg
40	$\approx 16\,\%$	
400	$\approx 5\,\%$	
4000	$\approx 1{,}6\,\%$	gut

Bei vielen kleinen Flüssen gleicht sich das statistisch aus. Bei wenigen großen, etwa bei der Spiegelung einer Datenbank, kann es geschehen, dass alle auf demselben Weg landen und drei Leitungen leer bleiben.

Die Abhilfe besteht darin, große Übertragungen künstlich in mehrere Verbindungen aufzuteilen. Das ist unschön und wirksam, und man findet es in praktisch jedem System, das große Datenmengen zwischen Rechenzentren bewegt.

Peterson und Davie[24] behandeln die Wegewahlverfahren gründlich; Kurose und Ross[17] führen sie mit Beispielen ein; Stevens[27] beschreibt die Protokolle bis auf die Bitebene.

4 Transport

Was der Transport hinzufügt

IP liefert Pakete aus, vielleicht. Es garantiert nichts.

	UDP	TCP
Zustellung	nicht garantiert	garantiert
Reihenfolge	beliebig	erhalten
Doppelte	möglich	ausgefiltert
Flusskontrolle	keine	ja
Staukontrolle	keine	**ja**
Kopf	8 B	20–60 B

Die vorletzte Zeile ist die wichtigste und die am wenigsten bekannte. Wer UDP benutzt, muss selbst dafür sorgen, dass er das Netz nicht überlastet. Ein Programm, das das versäumt, schadet nicht nur sich, sondern allen anderen auf derselben Strecke.

Flusskontrolle und Staukontrolle sind zweierlei.

	Flusskontrolle	Staukontrolle
Schützt	den **Empfänger**	das **Netz**
Vor	Überlaufen des Puffers	Überlastung der Leitungen
Gesteuert durch	angekündigtes Fenster	**geschätztes Fenster**
Erfährt man durch	Ansage	Rückschluss aus Verlust

Die letzte Zeile ist der Kern der Schwierigkeit. Der Empfänger sagt, wie viel er aufnehmen kann. Das Netz sagt gar nichts; der Absender muss aus dem Verhalten schließen, wie viel es verträgt.

Gesendet werden darf das Minimum aus beiden:

$$\text{Sendefenster} = \min\{\text{Empfängerfenster}, \text{Staufenster}\}.$$

Wie viel darf unterwegs sein?

Um eine Leitung auszulasten, muss so viel unterwegs sein, wie in sie hineinpasst.

Definition 4.1. Das **Bandbreiten-Verzögerungs-Produkt** ist

$$\text{BDP} = \text{Bandbreite} \times \text{Umlaufzeit}.$$

Strecke	Rate	Umlauf	BDP
Lokales Netz	1 Gbit/s	0,5 ms	62 KB
Nach New York	100 Mbit/s	80 ms	1 MB
Nach Sydney	1 Gbit/s	250 ms	**31 MB**
Satellit	100 Mbit/s	600 ms	7,5 MB

Die dritte Zeile bedeutet: Um eine Gigabit-Leitung nach Sydney auszulasten, müssen einunddreißig Megabyte gleichzeitig unterwegs sein, für die noch keine Bestätigung vorliegt. Beide Seiten brauchen entsprechend große Puffer.

Warum das ein Problem war.
Das Fensterfeld im TCP-Kopf ist 16 bit breit, also höchstens 64 KB.

Fenster	Nach Sydney	
64 KB	2 Mbit/s	unabhängig von der Leitung
1 MB	33 Mbit/s	
31 MB	1 Gbit/s	voll

Mit dem ursprünglichen Feld wären über weite Strecken höchstens zwei Megabit je Sekunde möglich, gleichgültig wie dick die Leitung ist. Seit 1992 gibt es deshalb eine Erweiterung, die das Fenster um bis zu vierzehn Stellen verschiebt.

Der Zusammenbruch von 1986

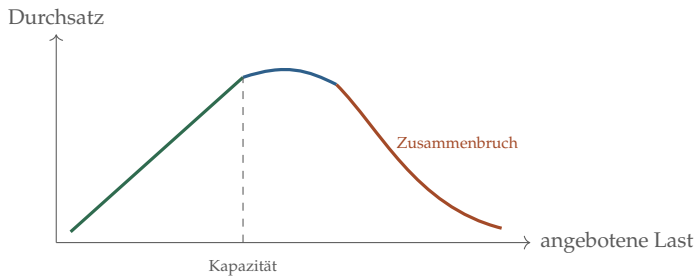
Im Oktober 1986 fiel der Durchsatz zwischen der Universität Berkeley und dem benachbarten Lawrence Berkeley Laboratory von 32 Kilobit je Sekunde auf 40 Bit je Sekunde. Die beiden Standorte liegen gut vierhundert Meter auseinander.

> *Faktor achthundert, ohne dass sich an der Hardware etwas geändert hatte.* VAN JACOBSON *und* MICHAEL KARELS *gingen der Sache nach und fanden ein Verhalten, das man heute Stauzusammenbruch nennt.*

Der Mechanismus.

1. Die Absender senden mehr, als die Leitung verträgt.
2. Die Warteschlange im Router läuft über, Pakete gehen verloren.
3. Die Absender bemerken den Verlust und **wiederholen**.
4. Die Wiederholungen erhöhen die Last weiter.
5. Noch mehr Verluste, noch mehr Wiederholungen.

> *Die Rückkopplung hat das falsche Vorzeichen. Ein System, das bei Überlast mehr sendet, ist unweigerlich instabil. Am Ende besteht der ganze Verkehr aus Wiederholungen, und fast nichts kommt an.*



Jacobsons Antwort.

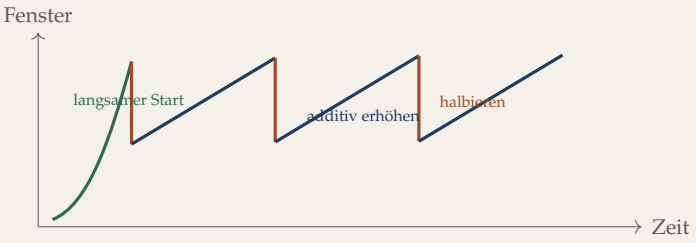
Erhaltung der Pakete	Ein neues Paket erst senden, wenn ein altes das Netz verlassen hat
Langsamer Start	Nicht mit voller Rate beginnen, sondern sich herantasten
Verlust heißt Stau	Ein verlorenes Paket wird als Überlastungszeichen gedeutet
Multiplikativ senken	Bei Stau das Fenster halbieren, sonst langsam erhöhen

Die letzte Zeile ist der entscheidende Einfall und stammt aus einer Arbeit von CHIU *und* JAIN*: Additiv erhöhen und multiplikativ senken führt dazu, dass sich mehrere Absender von selbst auf einen gerechten Anteil einpendeln. Additiv senken täte das nicht.*

Bemerkenswert an der Geschichte ist, dass keine Norm geändert wurde. Jacobson veränderte nur, wie der Absender sich verhält. Das Protokoll blieb dasselbe, und die Änderung verbreitete sich über die Betriebssysteme.

Der Sägezahn

Das Verhalten von TCP ergibt ein charakteristisches Bild.



Langsamer Start	Das Fenster **verdoppelt** sich je Umlauf; trotz des Namens sehr schnell
Stauvermeidung	Danach wächst es um ein Paket je Umlauf
Bei Verlust	Halbieren und wieder additiv wachsen

Der Name „langsamer Start“ ist irreführend. Er ist exponentiell und damit die schnellste Phase. Langsam ist er nur im Vergleich zum vorherigen Verhalten, sofort mit voller Rate loszulegen.

Verlust ist heute ein schlechtes Stausignal

Jacobsons Annahme von 1988 lautete: Pakete gehen fast nur durch Überlauf verloren. Das war für Kabelnetze richtig.

Funknetze	Verluste entstehen durch Störungen, nicht durch Stau; das Drosseln ist falsch
Große Puffer	Router mit sehr großen Warteschlangen verwerfen erst spät; bis dahin steigt nur die Verzögerung

*Die zweite Zeile beschreibt ein Phänomen, das unter dem Namen **Bufferbloat** bekannt wurde. Ein Router mit einem Puffer von einer Sekunde meldet keinen Stau, sondern verzögert alles um eine Sekunde. TCP merkt nichts und sendet weiter.*

Puffer	Verzögerung	Wirkung
1 BDP	gering	richtig
10 BDP	10× Umlauf	spürbar
100 BDP	Sekunden	**Videoanruf unbrauchbar**

Die Abhilfe besteht darin, Puffer klein zu halten oder früh und gezielt zu verwerfen. Die Verfahren dafür heißen CoDel und FQ-CoDel und sind seit etwa 2012 in Linux enthalten.

Der neuere Weg.
Das Verfahren BBR schätzt Bandbreite und kleinste Umlaufzeit unmittelbar, statt aus Verlusten zu schließen.

	verlustbasiert	**BBR**
Signal	Paketverlust	**Messung**
Bei Funkverlusten	drosselt falsch	unbeeindruckt
Bei großen Puffern	füllt sie	hält sie leer
Gerechtigkeit	bewährt	umstritten

Die letzte Zeile ist der offene Punkt. Ein Verfahren, das sich nicht an Verlusten orientiert, kann gegenüber einem, das es tut, im Vorteil sein und ihm Bandbreite abnehmen. Die Frage, wie verschiedene Verfahren nebeneinander bestehen, ist Gegenstand laufender Arbeit.

Aufgabe 4.1.

(a) Verfolgen Sie einen TCP-Verbindungsaufbau und -abbau.
(b) Berechnen Sie den erreichbaren Durchsatz bei Paketverlust.
(c) Untersuchen Sie, was QUIC anders macht.

Lösung Schritt für Schritt.

(a) Aufbau und Abbau.
Der Aufbau in drei Schritten.

Schritt	Nachricht	Zweck
1	SYN, Folgenr. x	„Ich möchte, ab x“
2	SYN-ACK, y, ack $x+1$	„Einverstanden, ab y“
3	ACK, ack $y+1$	„Bestätigt“

Warum drei und nicht zwei.

> *Beide Seiten müssen ihre Anfangsfolgenummer bekanntgeben* und *deren Empfang bestätigt bekommen. Das sind vier Vorgänge, von denen sich zwei in einer Nachricht zusammenfassen lassen. Drei ist das Minimum.*

Warum die Anfangsnummer zufällig sein muss.

Alte Pakete	Ein verspätetes Paket einer früheren Verbindung darf nicht als gültig durchgehen
Angriff	Wer die Folgenummer erraten kann, kann sich in eine Verbindung einschleusen

Frühe Umsetzungen zählten die Anfangsnummer schlicht hoch. Sie war damit vorhersagbar, und darauf beruhten Angriffe, die bis in die 1990er Jahre reichten. Heute wird sie aus einem Streuwert über die Verbindungsdaten und einem Geheimnis gebildet.

Der Abbau in vier Schritten.

1	FIN von A	„Ich habe nichts mehr“
2	ACK von B	
3	FIN von B	wenn auch B fertig ist
4	ACK von A	

Der Abbau braucht vier Schritte, weil jede Richtung getrennt geschlossen wird. Eine Seite kann fertig sein und weiter empfangen. Genau das benutzt man, wenn ein Programm seine Eingabe abschließt und noch auf die Antwort wartet.

Der Wartezustand am Ende.
Die Seite, die zuerst schließt, verharrt für die doppelte maximale Paketlebensdauer in einem Wartezustand, üblicherweise 60 Sekunden.

Der Grund	Verspätete Pakete der alten Verbindung dürfen eine neue mit denselben Anschlussnummern nicht stören
Die Folge	Ein Server mit vielen kurzen Verbindungen sammelt zehntausende solcher Zustände an

Deshalb schließt bei HTTP möglichst der Client zuerst, damit der Wartezustand bei ihm liegt und nicht beim Server. Eine Kleinigkeit im Protokollentwurf mit erheblicher Wirkung im Betrieb.

(b) Durchsatz bei Verlust.
Die Näherungsformel. Aus dem Sägezahnverhalten ergibt sich

$$\text{Durchsatz} \approx \frac{\text{MSS}}{\text{RTT}} \cdot \frac{C}{\sqrt{p}}, \qquad C \approx 1{,}22.$$

Zahlen für MSS $= 1460$ B und RTT $= 80$ ms.

p	$1/\sqrt{p}$	**Durchsatz**
10^{-2}	10	1,8 Mbit/s
10^{-3}	31,6	5,6 Mbit/s
10^{-4}	100	17,8 Mbit/s
10^{-5}	316	56 Mbit/s
10^{-6}	1000	178 Mbit/s

Die Wurzel ist der entscheidende Punkt. Um den Durchsatz zu verzehnfachen, muss die Verlustrate auf ein Hundertstel sinken. Deshalb sind Leitungen mit geringem, aber nicht verschwindendem Verlust so unangenehm.

Die Wirkung der Entfernung.

Ziel	RTT	bei $p = 10^{-4}$
Im Haus	1 ms	1,4 Gbit/s
Amsterdam	10 ms	142 Mbit/s
New York	80 ms	17,8 Mbit/s
Sydney	250 ms	5,7 Mbit/s

Eine einzelne TCP-Verbindung nach Sydney erreicht bei dieser Verlustrate knapp sechs Megabit je Sekunde, gleichgültig wie dick die Leitung ist. Das ist der Grund, warum Ladeprogramme mehrere Verbindungen parallel öffnen.

> *Zehn parallele Verbindungen erreichen etwa das Zehnfache. Das ist kein Betrug am Verfahren, sondern eine Folge davon, dass die Gerechtigkeit von TCP je Verbindung gilt und nicht je Anwendung.*

(c) Was QUIC anders macht.

	TCP + TLS	QUIC
Läuft auf	IP	**UDP**
Wo umgesetzt	im Kern	**im Programm**
Verschlüsselung	darüber	**eingebaut**
Aufbau	2–3 Umläufe	0–1
Mehrere Ströme	blockieren einander	unabhängig
Netzwechsel	Verbindung bricht	überlebt

Die fünfte Zeile im Detail.
Bei TCP über eine Verbindung mit mehreren Datenströmen blockiert ein verlorenes Paket alles, was danach kommt, auch aus anderen Strömen.

Lage	TCP	QUIC
Paket von Strom A geht verloren	B wartet mit	B läuft weiter

Der Grund ist, dass TCP einen einzigen Bytestrom in Reihenfolge liefert und von den Strömen darüber nichts weiß. QUIC kennt sie und kann jeden für sich in Reihenfolge bringen.

Die letzte Zeile im Detail.
Eine TCP-Verbindung wird durch vier Werte bestimmt: beide Adressen und beide Anschlussnummern. Wechselt das Gerät vom WLAN ins Mobilnetz, ändert sich die Adresse, und die Verbindung ist tot.

QUIC	benutzt eine eigene Verbindungskennung, die von der Adresse unabhängig ist
Die Folge	Ein Wechsel des Netzes unterbricht nichts; ein laufender Videoanruf läuft weiter

Der Preis.

Prozessorlast	Deutlich höher als TCP, weil alles im Programm und nicht im Kern läuft
Hardwarehilfe	Netzkarten entlasten TCP seit Jahrzehnten; für QUIC kam das erst spät
Fehlersuche	Der Kopf ist verschlüsselt; ein Mitschnitt zeigt fast nichts

Die letzte Zeile ist beabsichtigt und unbequem zugleich. Dass Geräte in der Mitte nichts mehr verändern können, ist der Zweck. Dass auch der eigene Betrieb nichts mehr sieht, ist die Nebenwirkung.

Bemerkenswert ist die zweite Zeile der Vergleichstabelle. Dass QUIC im Programm statt im Kern läuft, ist kein technisches Detail, sondern der eigentliche Grund für seine Verbreitung: Ein Anbieter kann das Verfahren ändern, ohne dass irgendjemand ein Betriebssystem aktualisiert.

Jacobsons Arbeit[14] begründete die Staukontrolle; Stevens[27] beschreibt TCP bis ins Detail; Kurose und Ross[17] entwickeln die Formeln mit Beispielen.

5 Namen und Anwendungen

Warum Namen und nicht Adressen

Merkbar	Menschen behalten Namen, keine Zahlenfolgen
Beweglich	Ein Dienst kann umziehen, ohne den Namen zu ändern
Verteilbar	Ein Name kann je nach Anfragendem verschiedene Adressen liefern
Unabhängig	Der Name gilt für IPv4 und IPv6 gleichermaßen

Die dritte Zeile ist heute die wichtigste. Große Anbieter beantworten dieselbe Anfrage aus verschiedenen Weltgegenden mit verschiedenen Adressen und lenken damit den Verkehr zum nächstgelegenen Rechenzentrum. Die Namensauflösung ist zur Verkehrssteuerung geworden.

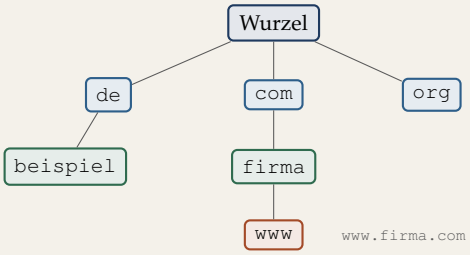
Der Vorläufer und sein Ende.
Bis 1983 gab es eine einzige Datei namens `HOSTS.TXT`, die alle Rechner des ARPANET aufführte. Sie wurde zentral gepflegt und musste von allen regelmäßig abgeholt werden.

Jahr	Rechner	Dateigröße	Aktualität
1973	≈ 40	wenige KB	gut
1983	≈ 500	≈ 100 KB	schlecht
hypothetisch 2026	$> 10^9$	aussichtslos	—

Das Verfahren skalierte nicht, und zwar an drei Stellen zugleich: Die Datei wurde zu groß, die zentrale Pflege zu aufwendig, und die Verteilung an alle zu teuer.

Die Antwort: Zuständigkeit aufteilen

PAUL MOCKAPETRIS entwarf 1983 ein System, das die Namen hierarchisch ordnet und die Zuständigkeit entsprechend aufteilt.



Jede Ebene	verwaltet nur, wer für die nächste zuständig ist
Die Wurzel	kennt die Server für `de`, `com` und die übrigen Endungen
Niemand	kennt alle Namen

Die letzte Zeile ist der Kern. Es gibt kein vollständiges Verzeichnis und keine Stelle, die eines hätte. Wer einen Namen einträgt, muss niemanden weltweit benachrichtigen; er trägt ihn bei seinem zuständigen Server ein, und der Rest ergibt sich durch Nachfragen.

Der Ablauf einer Auflösung.

Frage an	Frage	Antwort
Wurzel	`www.firma.com`?	„Frag die `com`-Server“
`com`	dieselbe	„Frag die Server von `firma.com`“
`firma.com`	dieselbe	**`203.0.113.7`**

Fall	Dauer	
Im Zwischenspeicher	< 1 ms	der Regelfall
Ein Schritt fehlt	≈ 30 ms	
Vollständige Auflösung	≈ 100–300 ms	selten

Das Ganze funktioniert nur, weil fast alles zwischengespeichert wird. Die Server der Wurzel und der Endungen bekommen deshalb sehr viel weniger Anfragen, als die Zahl der Namensauflösungen erwarten ließe.

Gültigkeitsdauer	Vorteil	Nachteil
60 s	schnelle Umschaltung	viele Anfragen
3600 s	üblich	Änderung dauert
86 400 s	sehr wenige Anfragen	Umzug dauert einen Tag

Wer einen Umzug plant, senkt die Gültigkeitsdauer vorher auf wenige Minuten und hebt sie danach wieder an. Wer das vergisst, wartet einen Tag, bis alle Zwischenspeicher den neuen Wert kennen.

Die Schwächen des Systems

DNS entstand in einer Zeit, in der niemand mit Angreifern rechnete.

Unverschlüsselt	Jeder auf dem Weg sieht, welche Namen abgefragt werden
Ungeprüft	Antworten sind nicht signiert; wer schneller antwortet, gewinnt
Verstärkung	Eine kleine Anfrage erzeugt eine große Antwort

Die erste Zeile.
Auch bei vollständig verschlüsseltem Verkehr verrät die Namensauflösung, welche Angebote jemand aufruft. Es ist derselbe Punkt wie bei den Verkehrsdaten in Band 8.

Verfahren	Transport	Bemerkung
DNS klassisch	UDP, offen	alles sichtbar
DoT	über TLS	eigener Anschluss
DoH	über HTTPS	**nicht von Web unterscheidbar**

Die dritte Zeile ist umstritten, und zwar aus einem bemerkenswerten Grund: Sie schützt den Einzelnen und nimmt zugleich Netzbetreibern und Unternehmen die Möglichkeit, Namensauflösungen zu sehen und zu filtern. Beides ist dieselbe Eigenschaft.

Die dritte Zeile.

	Größe	
Anfrage	≈ 60 B	
Antwort	bis 4000 B	mit Signaturen
Verstärkung	≈ **60**-fach	

Wer die Absenderadresse fälscht, lässt die Antworten beim Opfer landen. Aus einem Megabit je Sekunde Anfragen werden sechzig Megabit je Sekunde Angriffsverkehr, und der Angreifer bleibt unsichtbar.

Abhilfe 1	Server antworten nur eigenen Nutzern, nicht jedem
Abhilfe 2	Betreiber verwerfen Pakete mit offensichtlich gefälschter Absenderadresse

Die zweite Abhilfe ist seit 2000 als Empfehlung genormt und wäre die wirksamste. Sie wird bis heute nicht überall umgesetzt, weil sie dem Betreiber Aufwand macht und nur anderen nützt.

Aufgabe 5.1.

(a) Verfolgen Sie das Laden einer Webseite vollständig.
(b) Untersuchen Sie, wie eine falsche Antwort untergeschoben wird.
(c) Rechnen Sie den Nutzen von Zwischenspeichern aus.

Lösung Schritt für Schritt.

(a) Eine Webseite laden.
Die Schritte, erste Verbindung, Ziel in Europa.

	Vorgang	Zeit	Summe
1	Namensauflösung	30 ms	30
2	TCP-Aufbau	20 ms	50
3	TLS-Aushandlung	20 ms	70
4	Anfrage und Antwort	20 ms	90
5	HTML auswerten	10 ms	100
6	Weitere Bestandteile	60 ms	**160**

Von 160 Millisekunden entfallen 90 auf Vorbereitung und nur 20 auf die eigentliche Übertragung des Dokuments. Bei kleinen Seiten ist fast alles Umlaufzeit.

Was die Zeit senkt.

Maßnahme	Ersparnis	
DNS im Speicher	30 ms	fast immer
Verbindung wiederverwenden	40 ms	HTTP-Dauerverbindung
TLS-Wiederaufnahme	20 ms	
QUIC ohne Umlauf	40 ms	bei bekanntem Server
Inhalte in der Nähe	≈ 50 %	Verteilnetz

Die letzte Zeile wirkt am stärksten, weil sie die Umlaufzeit selbst senkt statt der Zahl der Umläufe. Ein Server in derselben Stadt macht jede der obigen Zeilen billiger.

Warum HTTP mehrere Fassungen hat.

Fassung	Neuerung	Problem, das bleibt
1.0	—	je Datei eine Verbindung
1.1	Dauerverbindung	Anfragen der Reihe nach
2	mehrere Ströme	TCP blockiert sie doch
3	über QUIC	gelöst

Die dritte Zeile ist lehrreich. HTTP/2 führte unabhängige Ströme ein und lief weiter über TCP, das davon nichts weiß. Ein einziges verlorenes Paket hielt alle Ströme an. Erst der Wechsel des Transports löste das Problem wirklich.

(b) Eine falsche Antwort unterschieben.
Die Lage. Ein Auflöser fragt und wartet. Wer zuerst eine passende Antwort liefert, gewinnt.
Was passen muss.

Feld	Umfang	
Kennung	16 bit	65 536
Quellport	16 bit	**früher fest**
Adressen und Frage	—	bekannt

Solange der Quellport fest war, blieb nur die Kennung zu erraten: eine von 65 536. Wer im Mittel 32 768 Antworten schickt, trifft. Das ist keine ernsthafte Hürde.

Die Verschärfung von 2008.
DAN KAMINSKY zeigte, dass sich der Versuch beliebig oft wiederholen lässt, indem man nach zufälligen Unternamen fragt.

1. Der Angreifer lässt nach `zufall1.firma.com` fragen.
2. Er antwortet gefälscht und hängt eine Auskunft über `firma.com` selbst an.
3. Trifft er die Kennung nicht, versucht er es mit `zufall2.firma.com`.
4. Beliebig viele Versuche, ohne dass ein Zwischenspeicher ihn ausbremst.

	zu erraten	**Erfolgsdauer**
Fester Quellport	2^{16}	Sekunden
Zufälliger Quellport	2^{32}	Tage
Mit Groß-/Kleinschreibung	$\approx 2^{40}$	Jahre

Die Notmaßnahme von 2008 bestand darin, den Quellport zufällig zu wählen. Das war keine Lösung, sondern eine Verteuerung, und es wurde in einer koordinierten Aktion weltweit an einem Tag ausgeliefert.

Die eigentliche Lösung.
DNSSEC signiert die Antworten mit den Mitteln aus Band 8. Eine Signaturkette reicht von der Wurzel bis zum einzelnen Eintrag.

Was es leistet	Eine gefälschte Antwort wird erkannt
Was es nicht leistet	Vertraulichkeit; die Anfrage bleibt sichtbar
Verbreitung 2026	bei den Endungen hoch, bei den einzelnen Namen gering

Die dritte Zeile hat denselben Grund wie bei IPv6 und bei der Wegewahlsicherung: Wer signiert, hat zunächst keinen Vorteil, solange die Auflöser nicht prüfen, und die Auflöser prüfen nicht, solange kaum jemand signiert.

(c) Der Nutzen von Zwischenspeichern.

Die Formel. Bei einer Trefferquote h ergibt sich die mittlere Zeit als

$$T = h \cdot T_{\text{Treffer}} + (1 - h) \cdot T_{\text{Fehlgriff}}.$$

Zahlen für $T_{\text{Treffer}} = 0{,}5$ ms und $T_{\text{Fehlgriff}} = 80$ ms.

h	T	**Beschleunigung**
0,00	80,0 ms	1,0
0,50	40,3 ms	2,0
0,80	16,4 ms	4,9
0,90	8,5 ms	9,5
0,95	4,5 ms	17,7
0,99	1,3 ms	**62**

Der Sprung von 95 auf 99 Prozent bringt mehr als der von 0 auf 80. Bei hohen Trefferquoten bestimmen allein die verbleibenden Fehlgriffe das Ergebnis, und jede Halbierung ihrer Zahl halbiert die Zeit.

Das ist dieselbe Rechnung wie bei den Caches in Band 4 und beim Verzeichnispuffer in Band 5. Sie taucht überall auf, wo eine schnelle und eine langsame Ebene zusammenwirken, und ihre Form ist immer dieselbe.

Was ein Zwischenspeicher schwer macht

Ungültigmachen	Woher weiß der Speicher, dass sich der Wert geändert hat?
Der Ansturm	Läuft ein beliebter Eintrag ab, fragen tausende gleichzeitig nach
Vergiftung	Ein falscher Eintrag wird lange weitergereicht

Die zweite Zeile hat einen eigenen Namen und eine einfache Abhilfe: Man streut die Ablaufzeiten zufällig, damit nicht alle Einträge gleichzeitig verfallen, und lässt nur eine Anfrage nachladen, während die übrigen warten.

Die erste Zeile ist die schwerste und bleibt es. Es gibt in der Informatik den Spruch, dass es nur zwei wirklich schwere Probleme gebe: Zwischenspeicher ungültig zu machen und Dinge zu benennen. Dieses Kapitel handelt von beiden.

Mockapetris[22] entwarf DNS; Kurose und Ross[17] beschreiben die Anwendungsschicht ausführlich; Peterson und Davie[24] behandeln die Namensauflösung im Zusammenhang.

Part II

Verteiltes Rechnen

6 Was es schwer macht

Drei Eigenschaften, die alles bestimmen

Teilausfall	Ein Teil läuft weiter, ein Teil nicht, und niemand weiß welcher
Keine gemeinsame Zeit	Es gibt keine Uhr, auf die sich alle beziehen können
Unbegrenzte Verzögerung	Eine Nachricht kann beliebig lange unterwegs sein

Aus diesen drei Punkten folgt fast alles, was die folgenden Kapitel behandeln. Sie sind keine technischen Unzulänglichkeiten, die sich beheben ließen, sondern Eigenschaften der Lage.

Der wichtigste Punkt: Man kann Ausfall und Langsamkeit nicht unterscheiden.

Beobachtung	Mögliche Ursache 1	Mögliche Ursache 2
Keine Antwort seit 5 s	Rechner ist tot	**er ist beschäftigt**
	Netz getrennt	Nachricht ist unterwegs

Ein Beobachter kann diese Fälle grundsätzlich nicht auseinanderhalten. Er kann nur eine Frist setzen und danach eine Vermutung anstellen. Jede Frist ist willkürlich, und jede Vermutung kann falsch sein.

Frist	Zu kurz	Zu lang
Fehler	Gesunde für tot erklärt	Tote zu spät bemerkt
Folge	unnötige Umschaltungen	lange Ausfallzeit
Schlimmstenfalls	**zwei Anführer zugleich**	System steht

Die letzte Zeile links beschreibt die gefährlichste Lage überhaupt: Ein Knoten wird für ausgefallen erklärt, ein Ersatz übernimmt, und der Erste läuft doch noch. Kapitel 9 zeigt, was dagegen hilft.

Fehlermodelle

Bevor man ein Verfahren entwirft, muss feststehen, wie ein Knoten ausfallen darf.

Modell	Der Knoten	Nötige Mehrheit
Absturz	hält an und schweigt	$\lceil (n+1)/2 \rceil$
Absturz und Neustart	kommt zurück, evtl. ohne Gedächtnis	dasselbe
Auslassung	verliert Nachrichten	dasselbe
Byzantinisch	**tut Beliebiges, auch Boshaftes**	$\lfloor (n-1)/3 \rfloor$ Fehler

Der Sprung in der letzten Zeile ist erheblich. Um f abstürzende Knoten zu verkraften, braucht man $2f+1$ Rechner. Um f böswillige zu verkraften, braucht man $3f+1$. Kapitel 10 zeigt, warum.

Verkraftet	Absturz	Byzantinisch
1 Fehler	3 Knoten	4 Knoten
2 Fehler	5	7
3 Fehler	7	10

Warum man meist beim Absturzmodell bleibt.

Eigene Rechner	Man traut der eigenen Hardware; boshaftes Verhalten ist unwahrscheinlich
Kosten	Mehr Knoten, mehr Nachrichten, mehr Rechenzeit
Wann doch	Wenn die Teilnehmer einander nicht trauen: über Organisationsgrenzen hinweg

Die dritte Zeile beschreibt genau die Lage bei verteilten Registern. Dort gibt es keinen gemeinsamen Betreiber, und deshalb ist der teure Weg dort der richtige.

Das unangenehmste Fehlerbild

Ein Knoten, der weder läuft noch tot ist, richtet mehr Schaden an als einer, der abstürzt.

Sauberer Absturz	Wird schnell erkannt, Ersatz übernimmt
Halb lebendig	Antwortet auf Lebenszeichen, erledigt aber keine Arbeit; niemand nimmt ihn heraus
Sehr langsam	Verzögert alles, was auf ihn wartet, ohne aufzufallen

Ein Betreiber sagte einmal, ein toter Rechner sei ein guter Rechner. Gemeint ist genau das: Der eindeutige Ausfall ist der handhabbare Fall. Alles Halbe ist schlimmer.

Deshalb prüfen gute Lebenszeichen nicht, ob ein Prozess läuft, sondern ob er seine Aufgabe erfüllt: eine Testabfrage an die Datenbank statt einer Antwort aus einer festen Zeichenkette.

Synchron, asynchron, dazwischen

Die Annahmen über Zeit bestimmen, was überhaupt möglich ist.

Modell	Annahme	Folge
Synchron	Schranken für Laufzeit und Taktdrift bekannt	vieles einfach
Asynchron	**keinerlei Schranken**	Konsens unmöglich
Teilsynchron	Schranken gelten **irgendwann**	Konsens möglich

Das mittlere Modell beschreibt die Wirklichkeit am ehrlichsten und ist zu streng, um damit zu arbeiten. Das dritte ist der Ausweg: Man nimmt an, dass das Netz sich nach einer unbekannten Zeit beruhigt. Alle heutigen Verfahren beruhen darauf.

Sicherheit	Nie etwas Falsches tun. Gilt **immer**, auch während einer Störung
Lebendigkeit	Irgendwann etwas Richtiges tun. Gilt nur, wenn sich das Netz beruhigt

Diese Trennung ist der Schlüssel zum Verständnis von Paxos und Raft. Sie geben niemals ein falsches Ergebnis, und sie geben unter Umständen für unbegrenzte Zeit gar keines. Das ist kein Mangel, sondern die einzige mögliche Antwort auf den Unmöglichkeitssatz aus Kapitel 9.

Aufgabe 6.1.

(a) Zeigen Sie das Problem der zwei Generäle.
(b) Untersuchen Sie, wie man Ausfälle erkennt.
(c) Rechnen Sie aus, wie oft etwas ausfällt.

Lösung Schritt für Schritt.

(a) Die zwei Generäle.
Die Lage. Zwei Heere stehen auf zwei Hügeln, der Feind im Tal dazwischen. Nur ein gemeinsamer Angriff siegt. Boten müssen durch das Tal und können abgefangen werden.
Die Behauptung. Es gibt kein Verfahren, das beide sicher zum gleichzeitigen Angriff bringt.
Der Beweis durch Widerspruch.

1.	Angenommen, es gibt ein Verfahren mit einer endlichen Zahl von Nachrichten.
2.	Von allen solchen Verfahren nehmen wir eines mit der **kleinsten** Zahl, sagen wir k.
3.	Die letzte Nachricht kann verlorengehen.
4.	Der Empfänger muss also auch ohne sie richtig handeln.
5.	Dann war sie überflüssig, und es gibt ein Verfahren mit $k - 1$ Nachrichten.
6.	Widerspruch zur Minimalität.

Also gibt es kein solches Verfahren.

Bemerkenswert ist, dass hier kein Angreifer und keine Böswilligkeit vorkommen. Es genügt, dass Nachrichten verlorengehen können. Gemeinsames Wissen lässt sich über einen unzuverlässigen Kanal nicht mit Sicherheit herstellen.

Was man stattdessen tut.

Wahrscheinlichkeit	Nach zehn bestätigten Nachrichten ist die Sicherheit hoch genug
Umdeuten	Statt Gleichzeitigkeit fordert man, dass jeder irgendwann handelt
Idempotenz	Man baut den Vorgang so, dass mehrfache Ausführung nichts schadet

Die dritte Zeile ist die praktisch wichtigste und die Antwort auf die fünf Fälle aus dem Vorwort. Wenn Wiederholen harmlos ist, muss man die Ungewissheit nicht auflösen.

(b) Ausfälle erkennen.
Der einfachste Weg. Regelmäßige Lebenszeichen, und bei Ausbleiben eine Frist.

Abstand	Frist	Erkennung	Fehlalarme
1 s	3 s	3 s	selten
1 s	10 s	10 s	sehr selten
0,1 s	0,3 s	0,3 s	häufig

Die adaptive Variante.
Statt einer festen Frist schätzt man aus den bisherigen Abständen, wie ungewöhnlich das Ausbleiben ist.

Verdachtsgrad	Statt „tot oder lebendig“ eine Zahl, die mit der Wartezeit steigt
Anpassung	In einem trägen Netz steigt sie langsamer
Vorteil	Die Schwelle lässt sich je nach Anwendung wählen

Die theoretische Einordnung.
CHANDRA und TOUEG zeigten 1996, wie sich Ausfalldetektoren nach ihren Eigenschaften ordnen lassen.

Vollständigkeit	Jeder ausgefallene Knoten wird irgendwann verdächtigt
Genauigkeit	Irgendwann wird ein lebender Knoten nicht mehr verdächtigt

Die erste Eigenschaft ist leicht zu erfüllen: Man verdächtige einfach jeden. Die zweite ist die schwierige. Chandra und Toueg zeigten, dass ein Detektor, der beide in abgeschwächter Form erfüllt, für Konsens genügt, und dass er die schwächste solche Annahme ist.

Damit wird der Unmöglichkeitssatz aus Kapitel 9 umgangen, ohne ihn zu verletzen: Man fügt eine Annahme hinzu, die im asynchronen Modell nicht gilt und in der Praxis meistens zutrifft.

(c) Wie oft etwas ausfällt.
Die Angaben. Ein Server fällt im Mittel alle drei Jahre aus.

$$p_{\text{Ausfall je Tag}} = \frac{1}{3 \cdot 365} = 9{,}1 \cdot 10^{-4}.$$

Bei n Servern.

$$P(\text{mindestens einer fällt heute aus}) = 1 - (1 - p)^n.$$

n	**je Tag**	**Ausfälle je Tag**	
1	0,09 %	0,0009	
100	8,7 %	0,09	
1000	60 %	0,9	
10 000	99,99 %	9,1	
100 000	$\approx$ 100 %	**91**	**täglich**

Bei hunderttausend Servern fallen jeden Tag einundneunzig aus. Das ist kein Störfall, sondern der Normalbetrieb. Ein System, das bei jedem Ausfall Aufmerksamkeit erfordert, ist bei dieser Größe nicht zu betreiben.

Die Folgerung für den Entwurf.

	kleine Anlage	große Anlage
Ausfall ist	ein Vorfall	**Normalbetrieb**
Reaktion	Mensch	**automatisch**
Hardware	hochwertig, teuer	gewöhnlich, ersetzbar
Schutz durch	Redundanz im Gerät	**Redundanz im System**

Die letzte Zeile beschreibt den Umbruch der 2000er Jahre. Statt wenige sehr zuverlässige Maschinen zu kaufen, nimmt man viele gewöhnliche und baut die Zuverlässigkeit in die Software. Das ist billiger und verlangt alles, was in den folgenden Kapiteln steht.

Was sonst noch ausfällt.

Einheit	Häufigkeit	Wirkung
Festplatte	2–4 % je Jahr	einzelne Daten
Server	≈ 30 % je Jahr	ein Knoten
Rack-Switch	selten	**40 Server auf einmal**
Stromkreis	selten	ein ganzer Bereich
Rechenzentrum	sehr selten	alles an einem Ort

Die dritte und vierte Zeile sind der Grund, warum Kopien nicht nur auf verschiedenen Servern liegen dürfen, sondern auf verschiedenen Racks und in verschiedenen Stromkreisen. Ausfälle sind nicht unabhängig, und die Rechnung von oben unterstellt genau das.

Tanenbaum und van Steen[28] ordnen die Modelle systematisch; Birman[2] behandelt Ausfalldetektoren und Gruppenkommunikation; Kleppmann[16] verbindet die Theorie mit der Praxis.

7 Zeit und Ordnung

Warum Uhren nicht genügen

Zwei Rechner haben zwei Uhren, und die gehen nie gleich.

Uhrentyp	Abweichung	je Tag
Quarz im Rechner	$\approx$ 30 ppm	$\approx$ 2,6 s
Nach NTP-Abgleich	$\approx$ 1 ms	
Präzisionszeit-protokoll	$\approx$ 1 µs	im Rechenzentrum
Atomuhr	$\approx 10^{-9}$	$\approx$ 0,1 ms je Jahr

Selbst nach einem Abgleich bleibt eine Unsicherheit von etwa einer Millisekunde. Zwei Ereignisse, die eine halbe Millisekunde auseinanderliegen, lassen sich anhand der Zeitstempel nicht ordnen.

Der Fehler, der daraus entsteht.

Zeit	Vorgang	Uhr A	Uhr B
t_1	A schreibt `x=1`	10,003	—
t_2	B schreibt `x=2`	—	10,001

Nach den Zeitstempeln kam B zuerst, tatsächlich war es A. Wer die letzte Schreibung gewinnen lässt, verliert hier die spätere.

Diese Regel, dass die höhere Zeitmarke gewinnt, ist weit verbreitet und in dieser Form fehlerhaft. Sie verwirft stillschweigend Daten, ohne dass es jemand bemerkt. Kapitel 8 zeigt, was man stattdessen tut.

Die Uhr kann rückwärts laufen

Ein Abgleich kann die Uhr zurückstellen. Ein Programm, das Zeitdifferenzen bildet, bekommt dann negative Werte.

Wanduhr	Kann springen, vor und zurück; für Zeitpunkte gedacht
Monotone Uhr	Läuft nur vorwärts, hat keinen Bezug zur Tageszeit; **für Zeitspannen**

Wer eine Frist misst, muss die monotone Uhr benutzen. Wer die Wanduhr nimmt, bekommt bei einer Zeitumstellung oder einem Abgleich eine Frist, die nie abläuft oder sofort abgelaufen ist.

Dazu kommen Schaltsekunden. Am 30. Juni 2012 führte eine solche zu Ausfällen bei mehreren großen Anbietern, weil der Kern in eine Schleife geriet. Seither verteilen viele Betreiber die Sekunde über einen ganzen Tag, statt sie einzufügen.

Lamports Ausweg: Ordnung ohne Uhren

1978 zeigte Leslie Lamport, dass man für die meisten Zwecke keine Uhrzeit braucht, sondern nur eine Ordnung.

Definition 7.1. Die Relation **geschieht vor**, geschrieben $a \to b$, ist die kleinste Relation mit:

- Liegen a und b im selben Prozess und a vor b, so gilt $a \to b$.
- Ist a das Senden und b das Empfangen derselben Nachricht, so gilt $a \to b$.
- Aus $a \to b$ und $b \to c$ folgt $a \to c$.

*Gilt weder $a \to b$ noch $b \to a$, so heißen die Ereignisse **nebenläufig**. Sie haben keine feste Reihenfolge, und zwar nicht, weil man sie nicht kennt, sondern weil es keine gibt.*

Die Lamport-Uhr.
Jeder Prozess führt einen Zähler C.

1.	Vor jedem Ereignis: $C \mathrel{+}= 1$.
2.	Beim Senden: den Wert C mitschicken.
3.	Beim Empfangen von t: $C \leftarrow \max(C, t) + 1$.

$$a \to b \quad \Longrightarrow \quad C(a) < C(b).$$

Die Umkehrung gilt nicht. *Aus $C(a) < C(b)$ folgt nicht, dass a vor b geschah; die beiden können nebenläufig sein. Die Lamport-Uhr erkennt also Abhängigkeit nicht, sie bewahrt sie nur.*

Vektoruhren schließen die Lücke.
Jeder Prozess führt einen Vektor mit einem Eintrag je Prozess.

1. Vor jedem eigenen Ereignis: den eigenen Eintrag erhöhen.
2. Beim Senden: den ganzen Vektor mitschicken.
3. Beim Empfangen: eintragsweise das Maximum bilden, dann den eigenen erhöhen.

$$a \to b \quad \Longleftrightarrow \quad V(a) \leq V(b) \text{ und } V(a) \neq V(b).$$

	Lamport	**Vektor**
Größe	eine Zahl	n **Zahlen**
Erkennt Nebenläufigkeit	nein	ja
Bei 1000 Knoten	8 B	8 KB je Nachricht

Die letzte Zeile ist der Grund, warum Vektoruhren in großen Systemen nicht unmittelbar benutzbar sind. Es gibt Verkürzungen, die nur die tatsächlich beteiligten Knoten führen, und sie helfen erheblich.

Die Uhr mit Fehlerschranke

Google verfolgte bei Spanner 2012 einen anderen Weg: Man macht die Uhrunsicherheit nicht kleiner, sondern *bekannt*.

Übliche Uhr	liefert einen Zeitpunkt und verschweigt den Fehler
TrueTime	liefert ein **Intervall** [früh, spät] und garantiert, dass die wahre Zeit darin liegt

Der Unterschied ist entscheidend. Eine Uhr, die sagt „es ist 10:00:00,000“ ist nicht überprüfbar. Eine, die sagt „es ist zwischen 10:00:00,000 und 10:00:00,007“ erlaubt eine Schlussfolgerung: Warte sieben Millisekunden, dann ist der Zeitpunkt sicher vorbei.

Wie man daraus eine Ordnung gewinnt.

1. Eine Transaktion bekommt als Zeitmarke die obere Schranke des Intervalls.
2. Vor dem Abschluss wartet sie, bis die untere Schranke diese Marke überschritten hat.
3. Danach ist sicher, dass jede spätere Transaktion eine größere Marke bekommt.

Unsicherheit	**Wartezeit**	**Transaktionen/s**
1 ms	1 ms	begrenzt durch Nebenläufigkeit
7 ms	7 ms	
100 ms	100 ms	unbrauchbar

Deshalb der Aufwand mit Atomuhren und Satellitenempfängern in jedem Rechenzentrum. Man kauft nicht Genauigkeit um ihrer selbst willen, sondern eine kleine Fehlerschranke, weil sie unmittelbar die Wartezeit bestimmt.

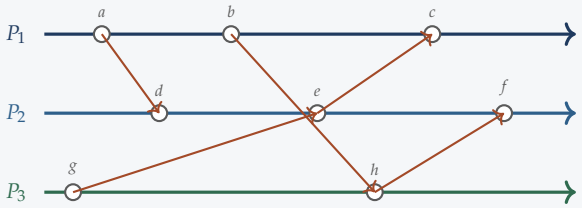
Das ist ein bemerkenswerter Entwurf: Ein Problem der verteilten Algorithmik wird teilweise durch Hardware gelöst. Die meisten Systeme können sich das nicht leisten und arbeiten deshalb mit logischen Uhren.

Aufgabe 7.1.

(a) Rechnen Sie Lamport- und Vektoruhren an einem Ablauf durch.
(b) Untersuchen Sie, wie NTP die Uhr stellt.
(c) Zeigen Sie, wo die Ordnung praktisch gebraucht wird.

Lösung Schritt für Schritt.

(a) Uhren an einem Ablauf.
Der Ablauf. Drei Prozesse, fünf Nachrichten.



Lamport-Uhren.

Ereignis	C	Begründung
g	1	erstes in P_3
a	1	erstes in P_1
d	2	$\max(0,1)+1$
b	2	$1+1$
e	3	$\max(2,1)+1$
h	3	$\max(1,2)+1$
c	4	$\max(2,3)+1$
f	4	$\max(3,3)+1$

Beachten Sie c und f: Beide haben den Wert 4 und sind nebenläufig. Die Lamport-Uhr kann das nicht anzeigen; sie liefert nur gleiche Zahlen, was man auch als „unbekannt“ lesen könnte.

Vektoruhren.

Ereignis	V	Begründung
a	$(1,0,0)$	
g	$(0,0,1)$	
d	$(1,1,0)$	empfängt von a
b	$(2,0,0)$	
e	$(1,2,1)$	empfängt von g
h	$(2,0,2)$	empfängt von b
c	$(3,2,1)$	empfängt von e
f	$(2,3,2)$	empfängt von h

Die Probe auf Nebenläufigkeit.

$$V(c) = (3,2,1), \qquad V(f) = (2,3,2).$$

$3 > 2$ in der ersten Stelle, $\qquad 2 < 3$ in der zweiten.

Keiner der beiden Vektoren ist überall kleiner oder gleich. Also sind c und f nebenläufig, und zwar nachweislich. ✓

Genau diese Auskunft braucht man bei der Zusammenführung von Kopien. Wenn zwei Änderungen nebenläufig sind, darf man nicht einfach die spätere nehmen; es gibt keine spätere. Man muss sie zusammenführen oder den Konflikt melden.

(b) Wie NTP die Uhr stellt.
Die vier Zeitstempel.

t_1	Client sendet	nach seiner Uhr
t_2	Server empfängt	nach dessen Uhr
t_3	Server antwortet	nach dessen Uhr
t_4	Client empfängt	nach seiner Uhr

Die Rechnung.

$$\text{Umlaufzeit } \delta = (t_4 - t_1) - (t_3 - t_2).$$

$$\text{Versatz } \theta = \frac{(t_2 - t_1) + (t_3 - t_4)}{2}.$$

Ein Zahlenbeispiel.

t_1	10,000	Clientuhr
t_2	10,028	Serveruhr
t_3	10,030	Serveruhr
t_4	10,018	Clientuhr

$$\delta = (10{,}018 - 10{,}000) - (10{,}030 - 10{,}028) = 0{,}018 - 0{,}002 = 0{,}016\ \text{s}.$$

$$\theta = \frac{(10{,}028 - 10{,}000) + (10{,}030 - 10{,}018)}{2} = \frac{0{,}028 + 0{,}012}{2} = 0{,}020\ \text{s}.$$

Die Clientuhr geht also 20 Millisekunden nach und wird entsprechend nachgeführt.

Die entscheidende Annahme.

Die Formel für θ unterstellt, dass Hin- und Rückweg gleich lange dauern. Trifft das nicht zu, entsteht ein Fehler von bis zu der halben Wegdifferenz, und er ist nicht erkennbar.

Lage	Fehler	
Symmetrisch	≈ 0	
10 ms Unterschied	5 ms	
Asymmetrische Strecke	beliebig	nicht feststellbar

Deshalb bleibt die Unsicherheit bei einer Millisekunde, obwohl die Uhren selbst viel genauer wären. Der Engpass ist nicht die Uhr, sondern das Netz, und das ist eine Aussage, die für den ganzen Band gilt.

(c) Wo die Ordnung gebraucht wird.

Ursache und Wirkung	Eine Antwort darf nicht vor der Frage erscheinen
Kopien abgleichen	Welche Änderung ist neuer, welche nebenläufig
Momentaufnahmen	Ein widerspruchsfreies Bild aller Knoten
Fehlersuche	Ablaufverfolgung über mehrere Dienste hinweg

Die erste Zeile an einem Beispiel.
In einem Forum schreibt *A* eine Frage, *B* liest sie und antwortet. Ein dritter Leser *C* sieht die Antwort vor der Frage.

> *Das ist nicht nur verwirrend, sondern verletzt eine Erwartung, die jeder Benutzer hat. Ein System, das die Beziehung „geschieht vor“ achtet, kann das ausschließen, und es braucht dafür keine synchronisierten Uhren.*

Garantie	Kosten	
Keine	null	Antwort vor Frage möglich
Kausale Ordnung	Zusatzangaben je Nachricht	schließt das aus
Vollständige Ordnung	**Konsens nötig**	teuer

Die mittlere Zeile ist oft genau richtig. Man braucht nicht, dass sich alle über die Reihenfolge aller *Ereignisse einigen, sondern nur, dass abhängige Ereignisse in der richtigen Folge erscheinen. Das ist erheblich billiger.*

Lamports Aufsatz von 1978[18] begründete das Gebiet; Corbett und Mitautoren[8] beschreiben TrueTime; Tanenbaum und van Steen[28] behandeln Uhren und Momentaufnahmen ausführlich.

8 Kopien und Konsistenz

Warum man Kopien hält

Verfügbarkeit	Fällt eine Kopie aus, übernimmt eine andere
Nähe	Leser werden von der nächstgelegenen Kopie bedient
Durchsatz	Lesende Zugriffe verteilen sich

Alle drei Gründe sind gut, und alle drei erzeugen dasselbe Problem: Sobald es mehrere Kopien gibt, können sie voneinander abweichen, und man muss festlegen, was ein Leser dann zu sehen bekommt.

Die Frage, um die es geht.

Zeit	Vorgang	Kopie A	Kopie B
t_1	Schreibe `x=1` an A	1	0
t_2	Lese `x` von B	1	**0**
t_3	Abgleich	1	1

Der Leser bei t_2 sieht einen veralteten Wert. Ob das ein Fehler ist, hängt davon ab, was das System versprochen hat.

Eine Leiter von Versprechen

Linearisierbarkeit	Es sieht aus, als gäbe es eine einzige Kopie; jeder Lesevorgang sieht die letzte abgeschlossene Schreibung
Sequentiell	Alle sehen dieselbe Reihenfolge, die aber nicht der Echtzeit entsprechen muss
Kausal	Abhängige Vorgänge erscheinen in der richtigen Folge; nebenläufige beliebig
Letztendlich	Irgendwann sind alle Kopien gleich, wenn nichts mehr geschrieben wird

Von oben nach unten werden die Versprechen schwächer und die Systeme schneller und verfügbarer. Es gibt keine richtige Wahl; es gibt nur die Frage, was die Anwendung braucht.

Modell	Kosten	Verfügbar bei Trennung	Beispiel
Linearisierbar	hoch	nein	etcd, Spanner
Sequentiell	hoch	nein	
Kausal	mittel	ja	
Letztendlich	gering	ja	Dynamo, DNS

Der Unterschied zwischen den ersten beiden.

Bei sequentieller Konsistenz einigen sich alle auf eine Reihenfolge. Bei Linearisierbarkeit muss diese Reihenfolge zusätzlich die tatsächliche Zeit achten: Was abgeschlossen war, bevor ein anderer Vorgang begann, muss auch davor einsortiert werden.

Nur die stärkere Fassung erlaubt Aussagen über Kanäle außerhalb des Systems. Wenn jemand am Telefon sagt „ich habe gerade gespeichert“, muss der andere es sehen können. Bei bloß sequentieller Konsistenz gilt das nicht.

Der Satz von Brewer

2000 stellte ERIC BREWER eine Vermutung auf, die GILBERT und LYNCH 2002 bewiesen.

Definition 8.1. Bei einer Netztrennung kann ein System nicht zugleich linearisierbar und für alle Anfragen verfügbar sein. Man muss eines von beiden aufgeben.

Der Beweis in fünf Zeilen.

1. Zwei Knoten A und B, das Netz zwischen ihnen ist getrennt.
2. Jemand schreibt an A und bekommt eine Bestätigung (Verfügbarkeit).
3. Jemand liest von B und bekommt eine Antwort (Verfügbarkeit).
4. B kann von der Schreibung nichts wissen; keine Nachricht kam durch.
5. Also ist die Antwort veraltet, und das widerspricht der Linearisierbarkeit.

Wer beides will, muss in Schritt 2 oder 3 eine Antwort verweigern.

Die verbreitete Fehldeutung

Man liest oft, man könne „zwei von drei“ wählen. Das ist irreführend.

Falsch	„Wir sind ein AP-System, also verzichten wir auf Konsistenz“
Richtig	„Wenn das Netz getrennt ist, entscheiden wir uns für Verfügbarkeit“

Die Netztrennung ist keine Eigenschaft, die man wählt, sondern ein Ereignis, das eintritt oder nicht. Solange nichts getrennt ist, kann ein System beides bieten, und die meisten tun das auch.

Die Erweiterung: PACELC.

Bei Trennung	Verfügbarkeit oder Konsistenz
Sonst	**Verzögerung oder Konsistenz**

Die zweite Zeile ist im Alltag wichtiger. Netztrennungen sind selten; die Abwägung zwischen Antwortzeit und Aktualität besteht dagegen ununterbrochen. Wer starke Konsistenz will, zahlt bei jedem einzelnen Zugriff mit Umläufen.

Verfahren	Umläufe je Schreibung	Zeit über Kontinente
Lokal, ohne Kopien	0	< 1 ms
Mehrheit im Rechenzentrum	1	≈ 1 ms
Mehrheit über Kontinente	1	≈ 100 ms

Die letzte Zeile begrenzt die Zahl der Schreibvorgänge je Sekunde und Datensatz auf etwa zehn. Das ist keine Frage der Rechenleistung, sondern der Lichtgeschwindigkeit.

Aufgabe 8.1.

(a) Rechnen Sie mit Quoren.
(b) Untersuchen Sie, wie man nebenläufige Änderungen zusammenführt.
(c) Prüfen Sie einen Ablauf auf Linearisierbarkeit.

Lösung Schritt für Schritt.

(a) Quoren.
Die Regel. Bei n Kopien, w für eine Schreibung und r für eine Lesung gilt:

$$w + r > n \quad \Longrightarrow \quad \text{jede Lesung sieht mindestens eine aktuelle Kopie.}$$

Die Begründung ist ein Abzählargument: Zwei Mengen mit zusammen mehr als n Elementen aus einer n-elementigen Grundmenge müssen sich überschneiden.

Die üblichen Einstellungen bei $n = 3$.

w	r	**konsistent?**	**Eigenschaft**
3	1	ja	schnelles Lesen, empfindliches Schreiben
2	2	ja	**ausgewogen**
1	3	ja	schnelles Schreiben
1	1	nein	schnell, letztendlich

Die Verfügbarkeit dazu.
Fällt ein Knoten aus, so ist bei $n = 3$:

w	r	**Schreiben geht?**	**Lesen geht?**
3	1	nein	ja
2	2	ja	ja
1	3	ja	nein

Die mittlere Zeile verkraftet als einzige einen Ausfall in beiden Richtungen. Deshalb ist die Mehrheit die übliche Wahl, und deshalb nimmt man ungerade Knotenzahlen.

Warum ungerade Zahlen.

n	Mehrheit	verkraftet	
3	2	1	
4	3	1	kein Gewinn
5	3	2	
6	4	2	kein Gewinn

Ein vierter Knoten erhöht die Zahl der verkrafteten Ausfälle nicht und vergrößert das nötige Quorum. Er kostet also Aufwand und bringt nichts. Deshalb haben Konsensgruppen fast immer drei, fünf oder sieben Mitglieder.

(b) Nebenläufige Änderungen zusammenführen.
Der schlechte Weg: die höhere Zeitmarke gewinnt.

Zeit	Vorgang	Marke	Ergebnis
t_1	*A* legt Milch in den Korb	10,003	
t_2	*B* legt Brot in den Korb	10,001	
t_3	Zusammenführung—		nur Milch

Das Brot ist verloren, ohne dass jemand es merkt. Bei einem Warenkorb ist das ärgerlich; bei einer Bestandsführung ist es ein Schaden.

Der bessere Weg: Konflikte erkennen und zusammenführen.

Erkennen	Mit Vektoruhren aus Kapitel 7 feststellen, ob die Änderungen nebenläufig waren
Beide behalten	Bei Nebenläufigkeit werden beide Fassungen aufbewahrt
Zusammenführen	Die Anwendung entscheidet, wie sie vereinigt werden

Beim Warenkorb ist die Vereinigung offensichtlich: beide Waren hineinlegen. Der Kunde findet im schlimmsten Fall etwas im Korb, das er entfernt hatte, und das ist besser als etwas zu verlieren, das er hineingelegt hat.

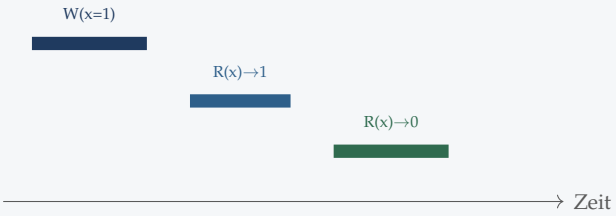
Datentypen, die von selbst zusammenpassen.
Es gibt Strukturen, bei denen die Zusammenführung immer eindeutig ist, ohne Rückfrage.

Typ	Zusammenführung	Beispiel
Zähler	Summe der Teilzähler	Aufrufzahlen
Menge, nur hinzufügen	Vereinigung	Empfängerliste
Registrierung mit Marke	höchste Marke	Einstellung
Text	Positionen zwischen Zeichen	**gemeinsames Bearbeiten**

Die Bedingung ist, dass die Zusammenführung kommutativ, assoziativ und idempotent ist. Dann kommt es weder auf die Reihenfolge noch auf Wiederholungen an, und die Kopien nähern sich ohne Absprache demselben Zustand.

Die letzte Zeile der Tabelle ist die Grundlage der Werkzeuge, bei denen mehrere Personen gleichzeitig in einem Dokument schreiben. Dort funktioniert etwas ohne Konsens, was auf den ersten Blick Konsens zu erfordern scheint.

(c) Einen Ablauf prüfen.
Der Ablauf. Drei Beteiligte, die Balken zeigen Beginn und Ende.

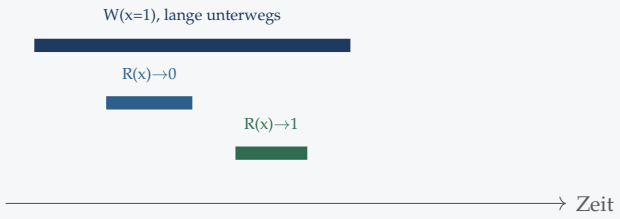


Die Prüfung.

1.	Die Schreibung endete bei 2,6, also vor beiden Lesungen.
2.	Die erste Lesung liefert 1: richtig.
3.	Die zweite Lesung beginnt bei 5,2 und liefert 0.
4.	Das ist ein **älterer** Wert, obwohl die Schreibung längst abgeschlossen war.

Der Ablauf ist nicht linearisierbar. Ein Wert, der einmal gelesen wurde, darf nicht wieder verschwinden.

Ein zulässiger Ablauf zum Vergleich.



Hier überschneidet sich die Schreibung mit beiden Lesungen. Solange sie läuft, darf jede Lesung den alten oder den neuen Wert liefern. Man kann den Zeitpunkt des Wirksamwerdens zwischen die beiden Lesungen legen, und dann ist alles widerspruchsfrei.

Die Regel	Ein überschneidender Zugriff darf jedes Ergebnis liefern
Die Bedingung	Es muss **einen** Zeitpunkt geben, an dem der Vorgang wirksam wurde, und alle Beobachtungen müssen dazu passen

Genau deshalb ist Linearisierbarkeit so nützlich: Sie erlaubt es, über ein verteiltes System nachzudenken, als wäre es ein einzelner Rechner. Und genau deshalb ist sie teuer.

Gilbert und Lynch[12] bewiesen den Satz; Brewer[3] ordnet ihn zwölf Jahre später selbst ein; Bailis und Ghodsi[1] erklären die schwächeren Modelle sehr klar.

9 Konsens

Die Aufgabe

Mehrere Knoten sollen sich auf einen Wert einigen.

Übereinstimmung	Keine zwei Knoten entscheiden sich verschieden
Gültigkeit	Der entschiedene Wert wurde von jemandem vorgeschlagen
Terminierung	Jeder fehlerfreie Knoten entscheidet sich irgendwann

Die ersten beiden sind Sicherheitseigenschaften: Es darf nie etwas Falsches geschehen. Die dritte ist eine Lebendigkeitseigenschaft: Irgendwann muss etwas geschehen. Der folgende Satz sagt, dass man nicht alle drei haben kann.

Wozu man das braucht.

Anführerwahl	Genau einer soll die Schreibvorgänge annehmen
Reihenfolge	Alle Kopien sollen dieselben Änderungen in derselben Folge anwenden
Sperren	Nur einer darf einen Bereich bearbeiten
Mitgliedschaft	Wer gehört gerade zur Gruppe

Alle vier sind dasselbe Problem in verschiedener Gestalt. Wer eines löst, hat alle gelöst, und deshalb steht der Konsens im Mittelpunkt dieses Bandes.

Der Unmöglichkeitssatz

Definition 9.1 (Fischer, Lynch, Paterson 1985)**.** In einem asynchronen System gibt es kein Verfahren, das Konsens erreicht, wenn auch nur ein einziger Knoten ausfallen kann.

> *Das ist eine der schärfsten negativen Aussagen der Informatik. Nicht „es ist teuer“ oder „es dauert lange“, sondern: Es gibt kein solches Verfahren, und es kann keines geben.*

Der Beweisgedanke.

1. Ein Zustand heißt **unentschieden**, wenn von ihm aus noch beide Ergebnisse erreichbar sind.
2. Man zeigt: Es gibt einen unentschiedenen Anfangszustand.
3. Man zeigt: Von jedem unentschiedenen Zustand führt eine Nachrichtenzustellung wieder in einen unentschiedenen.
4. Also gibt es einen unendlichen Ablauf, in dem nie entschieden wird.

Schritt 3 ist der Kern. Der Beweis konstruiert einen Widersacher, der die Nachrichten so zustellt, dass die Entscheidung immer wieder aufgeschoben wird. Er muss dabei keine einzige Nachricht verlieren, sondern nur verzögern.

Was der Satz sagt	Kein Verfahren garantiert alle drei Eigenschaften in einem asynchronen System
Was er nicht sagt	Dass Konsens in der Praxis unmöglich ist

Die Auswege.

Teilsynchronie	Man nimmt an, dass sich das Netz irgendwann beruhigt (Kap. 6)
Ausfalldetektor	Eine zusätzliche, nicht perfekte Auskunft über Ausfälle
Zufall	Verfahren, die mit Wahrscheinlichkeit eins terminieren
Lebendigkeit opfern	Sicherheit immer, Fortschritt nur meistens

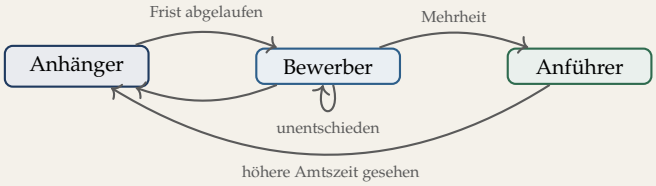
Paxos und Raft gehen den letzten Weg. Sie geben niemals ein falsches Ergebnis. Wenn das Netz sich pathologisch verhält, geben sie unter Umständen gar keines, und zwar beliebig lange. In der Praxis beruhigt sich das Netz nach Millisekunden.

Raft: Konsens zum Nachvollziehen

DIEGO ONGARO und JOHN OUSTERHOUT entwarfen 2014 ein Verfahren mit einem ungewöhnlichen Ziel: verständlich zu sein.

Paxos gilt seit 1998 als das maßgebliche Verfahren und zugleich als schwer zugänglich. Lamport hatte es als Bericht über die Gepflogenheiten eines fiktiven griechischen Parlaments geschrieben, was der Verbreitung nicht half.

Die drei Zustände.



Amtszeit	Eine fortlaufende Nummer; jede Wahl beginnt eine neue
Anhänger	Antwortet nur; wählt und nimmt Einträge entgegen
Bewerber	Bittet um Stimmen, nachdem er zu lange nichts gehört hat
Anführer	Nimmt alle Schreibvorgänge an und verteilt sie

Warum es höchstens einen Anführer je Amtszeit gibt.

Jeder Knoten gibt je Amtszeit höchstens eine Stimme ab. Ein Anführer braucht die Mehrheit. Zwei Mehrheiten müssen sich überschneiden, und der gemeinsame Knoten hätte zweimal gestimmt. Also kann es keine zwei geben.

Es ist dasselbe Abzählargument wie bei den Quoren in Kapitel 8. Zwei Mengen mit je mehr als der Hälfte der Elemente haben ein gemeinsames Element.

Wie die Wahl beendet wird.

Zufällige Fristen	Jeder wartet zwischen 150 und 300 Millisekunden; der erste bewirbt sich
Die Wirkung	Meist bewirbt sich nur einer, und die Wahl ist nach einem Umlauf entschieden
Bei Gleichstand	Neue Amtszeit, neue zufällige Fristen

Der Zufall ist hier kein Notbehelf, sondern die Lösung. Bei festen Fristen bewerben sich alle gleichzeitig, keiner bekommt die Mehrheit, und das wiederholt sich. Es ist dieselbe Einsicht wie beim Zurückweichen in Ethernet.

Aufgabe 9.1.

(a) Spielen Sie eine Anführerwahl mit Ausfall durch.
(b) Zeigen Sie, warum ein Zwei-Phasen-Abschluss blockiert.
(c) Rechnen Sie die Kosten des Konsenses aus.

Lösung Schritt für Schritt.

(a) Eine Anführerwahl.
Die Lage. Fünf Knoten S_1 bis S_5, Amtszeit 4, Anführer ist S_1.
Der Ablauf.

Zeit	Ereignis	Folge
t_1	S_1 fällt aus	keine Lebenszeichen mehr
t_2	S_3 Frist abgelaufen	Amtszeit 5, bewirbt sich
t_3	S_2, S_4 stimmen zu	mit S_3 selbst: drei Stimmen
t_4	S_3 hat die Mehrheit	**Anführer, Amtszeit 5**
t_5	S_5 Frist abgelaufen	Amtszeit 5, bewirbt sich
t_6	S_2 lehnt ab	hat in Amtszeit 5 schon gestimmt
t_7	S_5 hört von S_3	wird Anhänger

Der Rückkehrer.

t_8	S_1 kommt zurück	glaubt, Anführer in Amtszeit 4 zu sein
t_9	S_1 schickt Einträge	mit Amtszeit 4
t_{10}	Alle lehnen ab	ihre Amtszeit ist 5
t_{11}	S_1 sieht die 5	**wird Anhänger**

Die Amtszeitnummer erledigt das Problem der zwei Anführer aus Kapitel 6. S_1 kann keinen Schaden anrichten, weil niemand Anweisungen aus einer alten Amtszeit annimmt. Er merkt selbst, dass er abgelöst wurde, und fügt sich ein.

Die zusätzliche Regel bei der Stimmabgabe.
Ein Knoten stimmt nur zu, wenn der Bewerber ein mindestens so aktuelles Protokoll hat wie er selbst.

Ohne diese Regel könnte ein Knoten mit einem veralteten Protokoll gewählt werden und bereits bestätigte Einträge überschreiben. Die Regel sorgt dafür, dass ein neuer Anführer alles enthält, was jemals einer Mehrheit bekannt war.

(b) Warum der Zwei-Phasen-Abschluss blockiert.
Der Ablauf.

1.	Der Koordinator fragt alle: „Könnt ihr abschließen?“
2.	Jeder antwortet mit Ja oder Nein und ist danach **gebunden**.
3.	Bei lauter Ja: Der Koordinator weist den Abschluss an.
4.	Sonst: Er weist den Abbruch an.

Der schlimme Fall.

t_1	Alle antworten mit Ja	
t_2	Der Koordinator fällt aus	**bevor er etwas anweist**
t_3	Die Teilnehmer warten	unbegrenzt

Die Teilnehmer haben zugesagt und dürfen deshalb weder abschließen noch abbrechen. Sie halten ihre Sperren und blockieren alle anderen, die dieselben Daten brauchen. Ein einziger ausgefallener Koordinator legt das System still.

Warum sie sich nicht selbst einigen können.

Sie fragen einander	Alle sagen „ich habe Ja gesagt“
Sie wissen nicht	ob ein Teilnehmer, den sie nicht erreichen, Nein gesagt hat

Solange ein Teilnehmer nicht erreichbar ist, kann keiner entscheiden, ohne zu riskieren, dass er sich anders entscheidet als der Ausgefallene. Genau das ist der Fall, den der Satz von Fischer, Lynch und Paterson beschreibt.

Die Abhilfen.

Drei Phasen	Eine Zwischenstufe verhindert die Blockade; setzt aber ein synchrones System voraus und ist deshalb selten
Konsens darunter	Der Koordinator wird selbst durch eine Raft-Gruppe ersetzt und ist damit ausfallsicher

Die zweite Zeile ist der heute übliche Weg. Man löst das Problem nicht, sondern verschiebt es an eine Stelle, an der es gelöst ist. Das Entscheidungsprotokoll wird in ein repliziertes Protokoll geschrieben, und ein Nachfolger kann es lesen und fortsetzen.

(c) Die Kosten des Konsenses.
Nachrichten je Entscheidung bei n Knoten.

Schritt	Nachrichten	
Anführer an alle	$n-1$	
Antworten	$n-1$	
Zusammen	$\mathbf{2(n-1)}$	je Eintrag

n	Nachrichten	Umläufe	verkraftet
3	4	1	1
5	8	1	2
7	12	1	3
99	196	1	49

Bemerkenswert: Die Zahl der Umläufe bleibt eins, gleichgültig wie groß die Gruppe ist. Der Anführer wartet nicht auf alle, sondern auf die Mehrheit, und die antwortet, sobald die schnellere Hälfte geantwortet hat.

Die Verzögerung.

$T =$ Umlaufzeit zur $\lceil \frac{n-1}{2} \rceil$-schnellsten Antwort $+$ Schreiben auf Platte.

Aufstellung	Umlauf	Platte	Summe
Ein Rack	0,2 ms	0,3 ms	0,5 ms
Ein Rechenzentrum	0,5 ms	0,3 ms	0,8 ms
Drei Zonen einer Region	2 ms	0,3 ms	2,3 ms
Drei Kontinente	150 ms	0,3 ms	150 ms

Die letzte Zeile begrenzt die Schreibrate auf etwa sieben je Sekunde, wenn jeder Vorgang auf den vorigen wartet. Wer mehr braucht, muss bündeln oder die Daten so aufteilen, dass jede Gruppe in einer Region liegt.

Das Bündeln ist hier besonders wirksam: Hundert Einträge in einer Nachricht kosten fast dasselbe wie einer. Aus sieben Vorgängen je Sekunde werden siebenhundert, und die Verzögerung des einzelnen bleibt gleich.

Konsens ist kein allgemeines Werkzeug

Ein Konsensverfahren ist für kleine Datenmengen und wichtige Entscheidungen gedacht, nicht für den Hauptdatenstrom.

Richtig	Anführerwahl, Konfiguration, Mitgliedschaft, Sperren
Falsch	Alle Nutzdaten durch dieselbe Gruppe schicken

Der übliche Aufbau trennt beides: Eine kleine Konsensgruppe entscheidet, wer wofür zuständig ist, und die eigentliche Arbeit geschieht ohne sie. So wurde es bei Chubby, ZooKeeper und etcd gemacht, und so bauen es heute fast alle.

Fischer, Lynch und Paterson[11] bewiesen die Unmöglichkeit; Lamport[19] entwarf Paxos; Ongaro und Ousterhout[23] schrieben Raft mit dem Ziel der Verständlichkeit; Chandra und Mitautoren[6] berichten, was beim Bau tatsächlich schwer war; Chandra und Toueg[7] klärten die nötigen Annahmen.

10 Wenn Knoten lügen

Das Problem der byzantinischen Generäle

LAMPORT, SHOSTAK und PEASE kleideten 1982 ein Fehlermodell in ein Gleichnis.

> *Mehrere Divisionen belagern eine Stadt. Die Generäle können nur über Boten reden. Sie müssen sich auf Angriff oder Rückzug einigen. Einige von ihnen sind Verräter und können beliebig lügen, auch verschiedenen Empfängern Verschiedenes sagen.*

Übereinstimmung	Alle treuen Generäle entscheiden sich gleich
Gültigkeit	Ist der Befehlshaber treu, folgen alle seinem Befehl

Warum das kein Gleichnis für Bosheit sein muss.

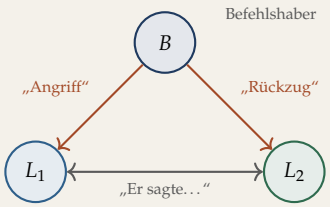
Angriff	Ein übernommener Rechner tut, was der Angreifer will
Fehlerhafte Hardware	Ein defekter Speicher liefert verschiedenen Lesern Verschiedenes
Programmfehler	Ein Knoten sendet Unsinn, ohne es zu bemerken
Verschiedene Fassungen	Zwei Umsetzungen desselben Protokolls verhalten sich ungleich

Die letzte Zeile ist der häufigste praktische Fall. Ein Knoten mit einer älteren Software ist nicht böswillig und verhält sich aus Sicht der anderen dennoch beliebig.

Warum drei nicht genügen

Definition 10.1. Mit n Knoten und f byzantinischen Fehlern ist das Problem genau dann lösbar, wenn $n \geq 3f + 1$.

Der Fall $n = 3$, $f = 1$.



Aus Sicht von L_1.

	Fall A	**Fall B**
B ist	**Verräter**	treu
L_2 ist	treu	**Verräter**
B sagte L_1	„Angriff“	„Angriff“
L_2 meldet	„Rückzug“	„Rückzug“ (gelogen)
Richtige Antwort	beliebig, aber gleich	**Angriff**

L_1 sieht in beiden Fällen genau dasselbe und muss verschieden handeln. Das ist unmöglich. Bei drei Knoten und einem Verräter gibt es also kein Verfahren, und zwar unabhängig davon, wie klug es ist.

Warum $3f + 1$ genügt.

1. Man kann höchstens auf $n - f$ Antworten warten; die übrigen f könnten ausgefallen sein.
2. Unter diesen $n - f$ können bis zu f von Verrätern stammen.
3. Es bleiben mindestens $n - 2f$ ehrliche Antworten.
4. Damit diese die Mehrheit bilden, muss $n - 2f > f$ gelten, also $n > 3f$.

Die Rechnung ist der ganze Grund für die Zahl drei. Zwei Verlustquellen zugleich, Ausfall und Lüge, verlangen den doppelten Aufschlag gegenüber dem einfachen Fall.

f	Absturz	Byzantinisch	Aufschlag
1	3	4	+33 %
2	5	7	+40 %
3	7	10	+43 %
10	21	31	+48 %

Der Aufschlag an Knoten ist überschaubar. Teurer ist der Nachrichtenaufwand: Beim klassischen Verfahren wächst er quadratisch mit der Knotenzahl, weil jeder jedem mitteilen muss, was er gehört hat.

Signaturen ändern die Lage

Kann ein Verräter Nachrichten nicht fälschen, so wird das Problem leichter.

	ohne Signaturen	mit Signaturen
Nötige Knoten	$3f + 1$	$\mathbf{f + 2}$
Weitersagen	Aussage über Aussage	**nachprüfbares Zeugnis**
Lügen gegenüber zweien	möglich	nachweisbar

Der Grund ist einleuchtend. Ohne Signaturen kann L_2 behaupten, B habe etwas gesagt, und L_1 kann es nicht prüfen. Mit Signaturen reicht L_2 die unterschriebene Nachricht weiter, und die Behauptung ist nachprüfbar.

Ein Befehlshaber, der zwei verschiedene Befehle unterschreibt, überführt sich damit selbst. Beide Zeugnisse tragen seine Unterschrift, und wer beide sieht, hat den Beweis. Das ist die Verbindung dieses Kapitels zu Band 8.

Das praktische Verfahren.
CASTRO und LISKOV zeigten 1999 mit PBFT, dass byzantinische Fehlertoleranz auch in einem asynchronen System praktikabel ist.

Phase	Zweck	Nachrichten
Vorschlag	Der Anführer schlägt eine Reihenfolge vor	$\mathcal{O}(n)$
Vorbereitung	Alle bestätigen einander	$\mathcal{O}(n^2)$
Festlegung	Alle bestätigen die Bestätigung	$\mathcal{O}(n^2)$

Warum zwei Bestätigungsrunden? Die erste sichert, dass sich alle über die Reihenfolge innerhalb einer Sicht einig sind. Die zweite sichert, dass diese Einigkeit einen Wechsel des Anführers übersteht. Ohne sie könnte ein neuer Anführer eine andere Reihenfolge durchsetzen.

n	Raft	PBFT	Faktor
4	6	≈ 36	6
10	18	≈ 210	12
100	198	$\approx 20\,100$	101

Der quadratische Aufwand begrenzt die Gruppengröße auf einige Dutzend. Für ein offenes, weltweites System ist das zu wenig, und genau daran arbeiten die Verfahren, die seit 2018 entstanden sind.

Aufgabe 10.1.

(a) Spielen Sie den Fall $n = 4$, $f = 1$ vollständig durch.
(b) Vergleichen Sie die Ansätze offener Register.
(c) Untersuchen Sie, wann sich der Aufwand lohnt.

Lösung Schritt für Schritt.

(a) Vier Generäle, ein Verräter.
Fall 1: Der Befehlshaber ist treu, ein Leutnant ist Verräter.
B sendet allen „Angriff“. L_3 ist Verräter.

Empfänger	von L_1	von L_2	von L_3
L_1	—	Angriff	Rückzug
L_2	Angriff	—	Rückzug

Die Auswertung bei L_1.

Quelle	Wert	
Direkt von B	Angriff	
Über L_2	Angriff	
Über L_3	Rückzug	
Mehrheit	**Angriff**	2 : 1

L_2 rechnet ebenso und kommt zum selben Ergebnis. Beide treuen Leutnants greifen an, wie der treue Befehlshaber es befahl. ✓

Fall 2: Der Befehlshaber ist der Verräter.
B sendet L_1 „Angriff“, L_2 „Rückzug“, L_3 „Angriff“.

Bei	direkt	gemeldet	Mehrheit
L_1	Angriff	Rückzug, Angriff	**Angriff** (2:1)
L_2	Rückzug	Angriff, Angriff	**Angriff** (2:1)
L_3	Angriff	Angriff, Rückzug	**Angriff** (2:1)

Alle drei treuen Leutnants kommen zum selben Ergebnis, obwohl sie verschiedene Befehle erhalten haben. Die Übereinstimmung ist erfüllt. Die Gültigkeit fordert hier nichts, weil der Befehlshaber nicht treu war.

Man beachte, dass jeder Leutnant dasselbe Mehrheitsverfahren auf seine drei Werte anwendet. Dass dabei bei allen dasselbe herauskommt, obwohl die Eingaben verschieden sind, ist der eigentliche Inhalt des Satzes.

(b) Offene Register.

Das neue Problem. In einem offenen System weiß niemand, wie viele Teilnehmer es gibt. Damit versagt jede Mehrheitsrechnung: Wer sich tausend Kennungen zulegt, hat die Mehrheit.

Ansatz	Stimmgewicht durch	Kosten
Arbeitsnachweis	Rechenleistung	Energie
Anteilsnachweis	hinterlegtes Guthaben	Kapital
Zulassung	bekannte Mitglieder	gering, nicht offen

Alle drei binden das Stimmgewicht an eine Ressource, die sich nicht beliebig vervielfachen lässt. Das ist die Antwort auf die Frage, wie man ohne Mitgliederliste abstimmt.

Der Unterschied zu PBFT.

	PBFT	Arbeitsnachweis
Teilnehmer	bekannt	offen
Endgültigkeit	sofort	nur wahrscheinlich
Durchsatz	Tausende/s	wenige/s
Energie	gering	sehr hoch

Die zweite Zeile ist der grundlegendste Unterschied. Bei PBFT ist eine Entscheidung endgültig. Beim Arbeitsnachweis wird sie mit jedem weiteren Block unwahrscheinlicher rückgängig zu machen, aber nie unmöglich.

Deshalb wartet man dort mehrere Bestätigungen ab. Es ist eine Sicherheit im Sinne der Wahrscheinlichkeit, nicht im Sinne eines Beweises, und dieser Unterschied wird in der öffentlichen Darstellung oft übergangen.

(c) Wann sich der Aufwand lohnt.

Lage	Modell	Begründung
Eigenes Rechenzentrum	Absturz	alle Knoten unter eigener Kontrolle
Mehrere Anbieter	Absturz	Verträge, Prüfung
Flugsteuerung	**byzantinisch**	Hardwarefehler kann alles bewirken
Register ohne Betreiber	**byzantinisch**	niemand traut niemandem

Die dritte Zeile im Detail.
In Flugzeugen und Raumfahrzeugen rechnen mehrere Rechner dasselbe, und ihre Ergebnisse werden verglichen.

Warum nicht Absturz	Ein defekter Sensor liefert keinen Ausfall, sondern einen falschen Wert
Warum verschiedene Bauarten	Ein Entwurfsfehler träfe sonst alle Rechner gleichzeitig

Die zweite Zeile beschreibt die Grenze aller Redundanz. Drei gleiche Rechner mit derselben Software machen denselben Fehler zur selben Zeit. Deshalb werden bei kritischen Systemen mehrere unabhängige Umsetzungen derselben Spezifikation gebaut.

Auch das hat Grenzen. Untersuchungen haben gezeigt, dass unabhängige Teams bei denselben schwierigen Stellen ähnliche Fehler machen, weil die Schwierigkeit in der Aufgabe liegt und nicht im Team.

Der häufigste Grund, es nicht zu tun

Byzantinische Fehlertoleranz schützt gegen einen Angreifer, der einzelne Knoten übernimmt. Sie schützt nicht gegen den häufigeren Fall.

Geschützt	Ein Drittel der Knoten ist übernommen
Nicht geschützt	Ein Fehler in der gemeinsamen Software auf allen Knoten
Nicht geschützt	Der Betreiber selbst handelt falsch

Wenn alle Knoten dieselbe Software ausführen, ist ein Fehler darin ein gemeinsamer Fehler, und keine Mehrheit hilft. Das ist der Grund, warum der Aufwand in der Praxis oft in besseres Prüfen fließt statt in mehr Knoten.

Lamport, Shostak und Pease[20] stellten das Problem 1982; Castro und Liskov[4] machten es praktikabel; Tanenbaum und van Steen[28] ordnen es ein.

Part III

Bauen im Großen

11 Daten verteilen

Zwei Arten, Daten aufzuteilen

	Kopieren	**Aufteilen**
Jeder Knoten hat	**alles**	**einen Teil**
Löst	Ausfälle, Leseleistung	Datenmenge, Schreibleistung
Grenze	Schreibleistung, Speicher	Ausfall trifft einen Teil

Beides zusammen ist der Regelfall: Die Daten werden in Abschnitte zerlegt, und jeder Abschnitt liegt in mehreren Kopien. Damit löst man beide Probleme und handelt sich die Schwierigkeiten beider ein.

Wie man aufteilt.

Verfahren	**Vorteil**	**Nachteil**
Nach Bereichen	Bereichsabfragen möglich	ungleiche Last
Nach Streuwert	gleichmäßig	keine Bereichsabfragen
Nach Verzeichnis	beliebig steuerbar	zusätzliche Nachschlagestelle

Der Gegensatz in den ersten beiden Zeilen ist grundsätzlich. Wer nach Streuwert aufteilt, zerstreut benachbarte Schlüssel absichtlich über alle Knoten. Damit wird die Last gleichmäßig, und eine Abfrage nach allen Werten zwischen zwei Schlüsseln muss jeden Knoten fragen.

Der heiße Schlüssel.

Fall	Verteilung	
Gleichmäßige Schlüssel	gut	
Zeitstempel als Schlüssel	alles auf einen	neue Daten immer rechts
Ein Prominenter	ein Knoten überlastet	Streuwert hilft nicht

Gegen die letzte Zeile hilft kein Aufteilungsverfahren, denn der Schlüssel ist einer. Man behilft sich, indem man ihm künstlich mehrere Unterschlüssel gibt und die Anfragen darauf verteilt. Das verlagert die Arbeit in die Anwendung, die die Teile wieder zusammenführen muss.

Konsistentes Hashing

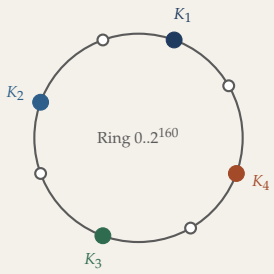
Das Problem mit der einfachen Lösung.
Der naheliegende Weg lautet

$$\text{Knoten} = h(\text{Schlüssel}) \bmod n.$$

Er funktioniert, solange n sich nicht ändert. Kommt ein Knoten hinzu, so ändert sich n, und damit ändert sich die Zuordnung für fast alle Schlüssel. Praktisch alle Daten müssten umziehen.

Änderung	Umzug bei Modulo	bei konsistentem Hashing
$10 \to 11$ Knoten	$\approx 91\,\%$	$\approx 9\,\%$
$100 \to 101$	$\approx 99\,\%$	$\approx 1\,\%$

Die Idee von Karger und Mitautoren, 1997.



1. Knoten und Schlüssel werden auf denselben Ring abgebildet.
2. Ein Schlüssel gehört dem **nächsten Knoten im Uhrzeigersinn**.
3. Fällt ein Knoten weg, so übernimmt sein Nachfolger dessen Bereich.
4. Kommt einer hinzu, so übernimmt er einen Teil von seinem Nachfolger.

Nur der Bereich zwischen dem neuen Knoten und seinem Vorgänger zieht um. Alles andere bleibt, wo es ist. Bei n Knoten sind das im Mittel $1/n$ der Daten.

Die Verfeinerung mit virtuellen Knoten.

Punkte je Knoten	**Abweichung der Last**	
1	$\approx 100\,\%$	sehr ungleich
10	$\approx 32\,\%$	
100	$\approx 10\,\%$	
1000	$\approx 3\,\%$	üblich

Jeder Knoten wird an vielen Stellen des Rings eingetragen. Damit gleichen sich die Bereichsgrößen aus, und ein ausfallender Knoten verteilt seine Last auf viele statt auf einen. Das ist wichtiger als die Gleichverteilung selbst.

Aufgabe 11.1.

(a) Rechnen Sie das Umzugsvolumen bei beiden Verfahren aus.
(b) Untersuchen Sie, wie Dynamo Verfügbarkeit erkauft.
(c) Vergleichen Sie Spiegelung und Löschcodes.

Lösung Schritt für Schritt.

(a) Umzugsvolumen.
Bei Modulo. Ein Schlüssel bleibt genau dann liegen, wenn

$$h(k) \bmod n = h(k) \bmod (n+1).$$

Für gleichverteilte Streuwerte trifft das mit Wahrscheinlichkeit etwa $1/(n+1)$ zu.

$n \to n{+}1$	**bleibt**	**zieht um**
$2 \to 3$	33 %	67 %
$10 \to 11$	9 %	91 %
$100 \to 101$	1 %	99 %

Bei konsistentem Hashing. Nur der Bereich des neuen Knotens zieht um:

$$\text{Anteil} = \frac{1}{n+1}.$$

$n \to n{+}1$	**Modulo**	**Konsistent**	**Faktor**
$10 \to 11$	91 %	9 %	10
$100 \to 101$	99 %	1 %	100
$1000 \to 1001$	99,9 %	0,1 %	1000

Was das in Byte bedeutet.

	Modulo	**Konsistent**
100 TB, 100 Knoten	99 TB	1 TB
Bei 10 Gbit/s	22 Stunden	13 Minuten

Der Unterschied entscheidet darüber, ob man einen Knoten im laufenden Betrieb hinzufügen kann oder nicht. Zweiundzwanzig Stunden Umzugsverkehr sind keine Erweiterung, sondern ein Ausfall mit Ankündigung.

(b) Wie Dynamo Verfügbarkeit erkauft.
Amazon beschrieb 2007 ein System, das eine ungewöhnliche Entscheidung trifft: Es nimmt Schreibvorgänge immer an.

Der Grund	Ein Warenkorb, in den man nichts legen kann, kostet unmittelbar Umsatz
Der Preis	Kopien können auseinanderlaufen und müssen zusammengeführt werden

Die Bausteine.

Konsistentes Hashing	mit virtuellen Knoten
Quoren	mit einstellbarem w und r (Kap. 8)
Vektoruhren	zum Erkennen nebenläufiger Änderungen
Zettelübergabe	Ist ein Knoten weg, nimmt ein anderer stellvertretend an
Merkle-Bäume	zum billigen Vergleich zweier Kopien

Die vierte Zeile im Detail.
Ist der zuständige Knoten nicht erreichbar, schreibt ein anderer den Wert mit einem Vermerk und reicht ihn später weiter.

> *Damit ist das System auch dann schreibbar, wenn die eigentlich zuständigen Knoten alle ausgefallen sind. Der Preis ist, dass ein Leser den Wert zunächst nicht findet, weil er beim falschen Knoten liegt.*

Die fünfte Zeile im Detail.
Zwei Knoten wollen prüfen, ob ihre Daten übereinstimmen, ohne alles zu übertragen.

1. Über jeden Datenblock wird ein Streuwert gebildet.
2. Je zwei Streuwerte werden zu einem übergeordneten zusammengefasst.
3. Das wiederholt sich bis zu einer Wurzel.
4. Zwei Knoten vergleichen zuerst die Wurzeln.
5. Bei Gleichheit sind alle Daten gleich; sonst steigt man in den abweichenden Teilbaum.

Lage	**Übertragen**	
Alles gleich	32 B	nur die Wurzel
Ein Block abweichend	$\approx \log_2 n$ Streuwerte	bei 10^6 Blöcken: 20
Alles verschieden	alles	

Der übliche Fall ist der erste oder der zweite. Damit lassen sich zwei Kopien von einem Terabyte mit wenigen hundert Byte Verkehr vergleichen. Es ist dieselbe Struktur, die Merkle 1979 für Signaturen einführte (Band 8).

(c) Spiegelung oder Löschcodes.
Die Aufgabe. Ein Petabyte soll zwei gleichzeitige Ausfälle überstehen.

	Dreifache Kopie	**Löschcode (10,4)**
Speicherbedarf	3 PB	1,4 PB
Verkraftet	2 Ausfälle	4 Ausfälle
Lesen normal	ein Knoten	ein Knoten
Lesen bei Ausfall	ein anderer	10 Knoten
Wiederherstellung	1× Datenmenge	10×

Die Löschcodes aus Band 7 sparen mehr als die Hälfte des Speichers und verkraften mehr Ausfälle. Die beiden letzten Zeilen sind der Preis: Um einen einzigen verlorenen Block wiederherzustellen, müssen zehn andere gelesen werden.

Wiederherstellung	**Kopie**	**Löschcode**
10 TB verloren	10 TB lesen	100 TB lesen
Bei 10 Gbit/s	2,2 h	22 h
Risiko währenddessen	gering	länger ungeschützt

Die letzte Zeile ist der eigentliche Einwand. Eine lange Wiederherstellung ist ein Zeitraum erhöhter Verwundbarkeit. Deshalb gibt es Codes, die eigens auf billige Reparatur ausgelegt sind und dafür etwas mehr Speicher brauchen.

Die übliche Aufteilung.

Heiße Daten	Dreifache Kopie; schnelles Lesen, billige Reparatur
Kalte Daten	Löschcodes; der Speicher überwiegt alles andere
Umschichtung	Nach einigen Tagen wandern Daten von der einen in die andere Form

Die dritte Zeile ist der Kompromiss, den fast alle großen Speichersysteme fahren. Er beruht darauf, dass frische Daten häufig gelesen werden und alte fast nie, und diese Beobachtung ist erstaunlich stabil.

Karger und Mitautoren[15] führten das konsistente Hashing ein; DeCandia und Mitautoren[10] beschreiben Dynamo; Kleppmann[16] behandelt Aufteilung und Kopien mit vielen Beispielen.

12 Nachrichten und Ströme

Warum man Nachrichten dazwischenschaltet

Zwei Dienste können unmittelbar miteinander reden. Meist ist es besser, wenn sie es nicht tun.

	unmittelbar	über eine Warteschlange
Verfügbarkeit	beide müssen laufen	Empfänger darf fehlen
Lastspitzen	schlagen durch	werden gepuffert
Antwortzeit	gering	höher
Kopplung	eng	lose
Fehlersuche	einfach	schwieriger

Die zweite Zeile ist der häufigste Grund. Wenn zehntausend Bestellungen in einer Minute eingehen und die Abrechnung nur hundert je Sekunde schafft, kann man entweder Bestellungen ablehnen oder sie in eine Schlange stellen.

Zwei Bauformen.

	Warteschlange	Protokoll
Nachricht wird	nach dem Lesen entfernt	**behalten**
Mehrere Leser	teilen sich die Arbeit	**lesen alle alles**
Nachlesen	unmöglich	jederzeit
Reihenfolge	je Schlange	je Abschnitt
Beispiel	RabbitMQ	Kafka

Die dritte Zeile ist der eigentliche Unterschied. Ein Protokoll bewahrt die Nachrichten für eine festgelegte Zeit auf, und jeder Leser merkt sich selbst, wie weit er gekommen ist. Damit kann ein neuer Dienst die Geschichte von vorn lesen, und ein fehlerhafter kann nach einer Korrektur noch einmal beginnen.

Drei Zustellgarantien

Garantie	Bedeutung	Umsetzung
Höchstens einmal	kann verlorengehen	senden und vergessen
Mindestens einmal	kann doppelt kommen	**wiederholen bis bestätigt**
Genau einmal	klingt am besten	gibt es nicht wirklich

Die dritte Zeile widerspricht dem Vorwort dieses Bandes. Weil sich die fünf Fälle nicht unterscheiden lassen, kann kein Protokoll garantieren, dass eine Nachricht genau einmal ankommt. Was es geben kann, ist eine genau einmalige Wirkung.

Was unmöglich ist	Die Nachricht kommt genau einmal an
Was möglich ist	Sie kommt mehrfach an, und die Wirkung tritt einmal ein

Idempotenz: die eigentliche Antwort.

Definition 12.1. Ein Vorgang heißt **idempotent**, wenn seine mehrfache Ausführung dasselbe bewirkt wie die einmalige.

Vorgang	idempotent?	
`Setze Stand auf 100`	ja	
`Erhöhe Stand um 10`	nein	
`Lösche Datensatz 42`	ja	
`Sende E-Mail`	nein	
`Buche 50 EUR ab`	nein	**teuer**

Wie man nicht-idempotente Vorgänge behandelt.

1. Der Absender vergibt eine eindeutige Vorgangsnummer.
2. Der Empfänger führt eine Tabelle der bereits erledigten Nummern.
3. Kommt eine bekannte Nummer erneut, wird das alte Ergebnis zurückgegeben.
4. Prüfung und Ausführung geschehen in derselben Transaktion.

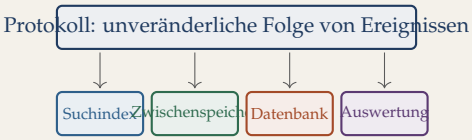
Punkt 4 ist entscheidend und wird oft übersehen. Wer zuerst prüft, dann ausführt und dann einträgt, hat zwischen den Schritten dieselbe Lücke wie beim Zähler in Band 5, nur über Rechnergrenzen hinweg.

Der Aufwand	Die Tabelle muss aufbewahrt werden, solange Wiederholungen möglich sind
Die Größe	Bei 10^6 Vorgängen je Tag und sieben Tagen Aufbewahrung: $7 \cdot 10^6$ Einträge

Das ist überschaubar und muss bedacht werden. Wer die Tabelle zu früh leert, führt einen alten Vorgang ein zweites Mal aus, und zwar genau in dem seltenen Fall, für den die ganze Vorkehrung gedacht war.

Das Protokoll als einzige Wahrheit

Eine Bauweise, die sich seit etwa 2013 durchgesetzt hat: Man behandelt die Folge der Änderungen als das eigentliche Datum und alle Zustände als daraus abgeleitet.



Eine Reihenfolge	Alle Abnehmer sehen dieselbe Folge und kommen zum selben Ergebnis
Neu aufbauen	Ein beschädigter Index wird verworfen und aus dem Protokoll neu erzeugt
Neue Abnehmer	Ein neuer Dienst liest die Geschichte von vorn
Nachvollziehbar	Man kann fragen, warum ein Zustand so ist

Es ist derselbe Gedanke wie beim Journal eines Dateisystems in Band 5 und beim protokollstrukturierten Dateisystem: Schreibe auf, was geschehen soll, und leite den Zustand daraus ab. Die Änderungen sind die Wahrheit, der Zustand ist eine Bequemlichkeit.

Was dabei unbequem wird

Löschen	Ein unveränderliches Protokoll widerspricht dem Recht auf Löschung
Formatänderungen	Alte Einträge haben ein altes Format und müssen weiter lesbar bleiben
Verzögerung	Die abgeleiteten Sichten hinken hinterher

Die erste Zeile ist ernst. Der übliche Ausweg besteht darin, personenbezogene Daten verschlüsselt abzulegen und beim Löschen den Schlüssel zu vernichten. Das Protokoll bleibt unverändert, und der Inhalt ist unlesbar geworden.

Die dritte Zeile ist der Alltag: Ein Benutzer ändert etwas und sieht die Änderung im Suchergebnis erst nach einer Sekunde. Man kann das verbergen, indem man die eigene Änderung im Browser schon anzeigt, bevor sie überall angekommen ist.

Aufgabe 12.1.

(a) Bauen Sie eine Abbuchung so, dass Wiederholung unschädlich ist.
(b) Untersuchen Sie, wie man ohne verteilte Transaktion auskommt.
(c) Rechnen Sie das Verhalten einer Warteschlange unter Last aus.

Lösung Schritt für Schritt.

(a) Eine wiederholungsfeste Abbuchung.
Der naive Weg.

```
UPDATE konto SET stand = stand - 50
WHERE nr = 4711;
```

Bei zweifacher Ausführung werden hundert abgebucht. Der Vorgang ist nicht idempotent, und das Netz liefert keine Garantie, dass er nur einmal ankommt.

Der Weg mit Vorgangsnummer.

```
BEGIN;
INSERT INTO erledigt(vorgang) VALUES
('abc-123');
- schlägt bei Wiederholung fehl:
eindeutiger Schlüssel
UPDATE konto SET stand = stand - 50
WHERE nr = 4711;
COMMIT;
```

Durchgang	Einfügen	Ergebnis
Erster	gelingt	Stand −50
Zweiter	**schlägt fehl**	Transaktion abgebrochen

Weil beides in einer Transaktion steht, geschieht entweder alles oder nichts. Die Eindeutigkeitsbedingung der Datenbank übernimmt die Prüfung, und zwar atomar. Es gibt kein Zeitfenster dazwischen.

Woher die Vorgangsnummer kommt.

Vom Absender	Er erzeugt sie einmal und benutzt sie bei allen Wiederholungen
Nicht vom Empfänger	Sonst bekäme jede Wiederholung eine neue Nummer
Nicht aus dem Inhalt	Zwei echte Abbuchungen über denselben Betrag wären sonst nicht unterscheidbar

Die dritte Zeile ist ein häufiger Fehler. Wer den Streuwert des Inhalts als Nummer nimmt, unterdrückt eine zweite, echte Zahlung über denselben Betrag am selben Tag.

(b) Ohne verteilte Transaktion.
Die Lage. Eine Bestellung soll den Bestand mindern und eine Zahlung auslösen. Beides liegt in verschiedenen Diensten.

Klassisch	Ein Zwei-Phasen-Abschluss über beide Dienste
Das Problem	Blockade bei Ausfall (Kap. 9), enge Kopplung

Die Alternative: eine Folge mit Ausgleichsschritten.

Schritt	Vorgang	Ausgleich bei Fehlschlag
1	Bestellung anlegen	Bestellung stornieren
2	Bestand reservieren	Reservierung aufheben
3	Zahlung einziehen	Zahlung erstatten
4	Versand auslösen	schwierig

Jeder Schritt schließt für sich ab. Scheitert ein späterer, so werden die früheren durch Gegenbuchungen ausgeglichen. Das ist kein Rückgängigmachen, sondern ein nachträglicher Ausgleich, und der Unterschied ist wesentlich.

	Transaktion	Ausgleichsfolge
Zwischenzustand	unsichtbar	sichtbar
Isolation	ja	nein
Blockade	möglich	nein
Kopplung	eng	lose

Die erste Zeile ist der Preis. Zwischen Schritt 2 und dem Ausgleich sieht ein anderer Kunde den Artikel als reserviert, obwohl die Bestellung scheitern wird. Das ist ein fachliches Problem und muss fachlich beantwortet werden, nicht technisch.

Genau deshalb hat dieses Vorgehen seine natürliche Entsprechung im Geschäftsleben. Eine Stornierung ist dort seit jeher eine Gegenbuchung und kein Ausradieren. Die Buchhaltung hat das Problem vor Jahrhunderten gelöst.

(c) Eine Warteschlange unter Last.
Der Satz von Little.

$$L = \lambda \cdot W,$$

mit L als mittlerer Zahl im System, λ als Ankunftsrate und W als mittlerer Verweildauer.

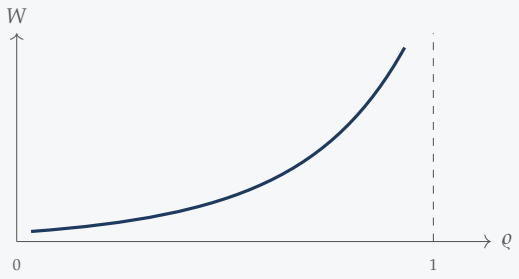
Die Wartezeit bei einem Bediener.

$$W = \frac{1}{\mu - \lambda} = \frac{1/\mu}{1 - \varrho}, \qquad \varrho = \frac{\lambda}{\mu}.$$

Zahlen für $\mu = 100$ Aufträge je Sekunde.

λ	ϱ	W	**Faktor**
50	0,50	20 ms	2
80	0,80	50 ms	5
90	0,90	100 ms	10
95	0,95	200 ms	20
99	0,99	1000 ms	**100**
100	1,00	∞	—

Die Bearbeitung eines Auftrags dauert unverändert zehn Millisekunden. Bei neunundneunzig Prozent Auslastung wartet er eine Sekunde. Die Wartezeit wächst nicht linear mit der Last, sondern explodiert kurz vor der Sättigung.



Die Folgerung	Man plant Systeme auf 60 bis 70 Prozent Auslastung, nicht auf 95
Der Grund	Der Rest ist Reserve für Schwankungen, nicht Verschwendung

Wer eine Anlage auf neunzig Prozent auslastet, um Kosten zu sparen, kauft sich eine zehnfache Wartezeit ein. Das ist selten der bessere Tausch, und es ist eine der am häufigsten übersehenen Rechnungen im Betrieb.

Gray und Reuter[13] behandeln Transaktionen und Ausgleichsfolgen; Kleppmann[16] entwickelt die protokollbasierte Bauweise; Shapiro und Mitautoren[26] beschreiben Datentypen, die sich ohne Absprache zusammenführen lassen.

13 Ausfälle und Betrieb

Wiederholen will gelernt sein

Eine Wiederholung ist die naheliegendste Reaktion auf einen Fehler und zugleich die gefährlichste.

Sofort	Trifft denselben überlasteten Dienst
Ohne Obergrenze	Verlängert den Ausfall beliebig
Gleichzeitig von allen	Alle Aufrufer wiederholen im Takt
Über mehrere Ebenen	Multipliziert sich

Die letzte Zeile in Zahlen.
Drei Ebenen, jede mit drei Versuchen:

$$3 \times 3 \times 3 = 27 \text{ Aufrufe für eine Anfrage.}$$

Ebenen	Versuche je Ebene	Gesamtaufrufe	
1	3	3	
2	3	9	
3	3	27	
4	3	81	Verstärkung

Genau in dem Augenblick, in dem ein Dienst überlastet ist, verachtzigfacht sich die Last auf ihn. Das ist dieselbe Rückkopplung mit falschem Vorzeichen wie beim Stauzusammenbruch von 1986 in Kapitel 4.

Nur eine Ebene wiederholt	Meist die äußerste oder eine mittlere, nie alle
Wachsende Pausen	Verdoppeln nach jedem Versuch
Zufällige Streuung	Damit nicht alle gleichzeitig wiederholen
Wiederholungs-budget	Höchstens fünf Prozent aller Aufrufe dürfen Wiederholungen sein

Die letzte Zeile ist die wirksamste und die am wenigsten verbreitete. Sie begrenzt die Verstärkung unmittelbar: Solange nur wenige Aufrufe fehlschlagen, wird großzügig wiederholt; sobald viele fehlschlagen, hört es auf.

Die synchronisierte Herde

Wiederholt jeder nach genau einer Sekunde, so treffen alle Wiederholungen gleichzeitig ein.

Verfahren	Spitzenlast
Feste Pause	100 % auf einmal
Verdoppelnde Pause	dasselbe, nur später
Mit Zufall	gleichmäßig verteilt

Die zweite Zeile ist der häufige Irrtum. Wachsende Pausen allein helfen nicht, wenn alle Aufrufer zur selben Zeit begonnen haben; sie kommen dann eben nach zwei, vier und acht Sekunden gemeinsam. Erst der Zufall bricht den Gleichtakt.

Dasselbe gilt für regelmäßige Aufgaben. Wenn tausend Rechner ihre Aufgabe zur vollen Stunde starten, entsteht jede Stunde eine Lastspitze. Man streut die Startzeiten, und das Problem verschwindet.

Fristen und der Sicherungsschalter

Jeder Aufruf braucht eine Frist.

Ohne Frist	Ein hängender Aufruf hält einen Faden für immer fest
Die Folge	Der Vorrat an Fäden erschöpft sich, und der ganze Dienst steht

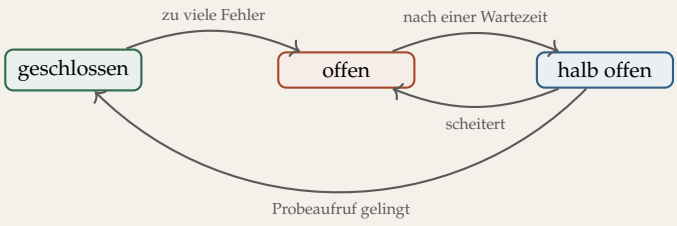
Ein Dienst, der auf einen langsamen Nachbarn wartet, wird selbst langsam und reißt seine eigenen Aufrufer mit. So pflanzt sich ein Ausfall durch ein ganzes System fort, obwohl nur ein einzelner Bestandteil betroffen war.

Fristen müssen weitergereicht werden.

	feste Fristen	**weitergereichte Frist**
Ebene 1	10 s	noch 10 s
Ebene 2	10 s	noch 8 s
Ebene 3	10 s	noch 6 s
Aufwand nach Abbruch	läuft weiter	wird abgebrochen

Wenn der oberste Aufrufer nach zehn Sekunden aufgibt, ist alle Arbeit der unteren Ebenen danach vergeudet. Reicht man die verbleibende Zeit mit, so hören sie von selbst auf, und das entlastet ein überlastetes System spürbar.

Der Sicherungsschalter.



Geschlossen	Alles läuft; Fehler werden gezählt
Offen	Aufrufe scheitern **sofort**, ohne den Dienst zu behelligen
Halb offen	Ein einzelner Probeaufruf entscheidet, ob es wieder geht

Der Nutzen liegt in beide Richtungen. Der Aufrufer wartet nicht mehr auf einen Dienst, von dem er weiß, dass er nicht antwortet. Und der überlastete Dienst bekommt Ruhe, um sich zu erholen, statt weiter mit Anfragen überschüttet zu werden.

Ein offener Schalter setzt allerdings voraus, dass es eine sinnvolle Ersatzantwort gibt: ein veralteter Wert aus dem Zwischenspeicher, ein eingeschränkter Dienst, eine ehrliche Fehlermeldung. Ohne das verschiebt er den Fehler nur nach oben.

Wie viel Ausfall erlaubt ist

Verfügbarkeit	je Jahr	je Monat	je Woche
99 %	3,65 Tage	7,2 h	1,7 h
99,9 %	8,8 h	43 min	10 min
99,99 %	53 min	4,3 min	1 min
99,999 %	5,3 min	26 s	6 s

Fünf Neunen bedeuten, dass ein Mensch nicht mehr eingreifen kann. Fünf Minuten im Jahr reichen nicht einmal, um sich einzuloggen und nachzusehen. Alles muss automatisch geschehen.

Die Rechnung bei mehreren Bestandteilen.
Hängt ein Dienst von n anderen ab und braucht alle, so gilt

$$V_{\text{gesamt}} = \prod_{i=1}^{n} V_i.$$

n	**je** 99,9 %	**Gesamt**	**Ausfall/Jahr**
1	0,999	99,90 %	8,8 h
5	0,999	99,50 %	44 h
20	0,999	98,02 %	174 h
100	0,999	90,5 %	832 h

Hundert Dienste mit je drei Neunen ergeben zusammen nur noch neunzig Prozent. Das ist der stärkste Einwand gegen eine feingliedrige Aufteilung in viele Dienste, und er wird beim Entwurf regelmäßig übersehen.

Abhängigkeiten senken	Nicht jeder Dienst muss jeden fragen
Sanft abfallen	Ohne den Empfehlungsdienst geht die Seite trotzdem
Kopien	Zwei unabhängige Kopien mit je 99 % ergeben zusammen 99,99 %

Die zweite Zeile ist die wirksamste. Ein Dienst, dessen Ausfall nur einen Teil der Seite leer lässt, gehört nicht in die Produktkette und darf die Gesamtverfügbarkeit nicht senken. Das ist eine Entwurfsentscheidung und keine Betriebsfrage.

Aufgabe 13.1.

(a) Rechnen Sie ein Fehlerbudget durch.
(b) Untersuchen Sie, wie sich ein Ausfall fortpflanzt.
(c) Diskutieren Sie das absichtliche Herbeiführen von Fehlern.

Lösung Schritt für Schritt.

(a) Das Fehlerbudget.
Der Gedanke. Statt Ausfälle zu verbieten, gibt man ein Kontingent vor.

Ziel	99,9 %	im Quartal
Erlaubte Ausfallzeit	2,16 h	
Verbraucht nach 6 Wochen	1,80 h	83 %
Rest	0,36 h	knapp

Was daraus folgt.

Budget	**Entscheidung**	
Reichlich vorhanden	zügig ausliefern	Risiko ist bezahlbar
Fast verbraucht	**nur noch Stabilisierung**	keine neuen Merkmale
Überzogen	Änderungsstopp	Ursachen beheben

Der eigentliche Wert liegt darin, dass ein alter Streit damit entschieden wird. Entwicklung will schnell ausliefern, Betrieb will Stabilität. Das Budget macht daraus eine gemeinsame, nachrechenbare Größe statt einer Auseinandersetzung über Haltungen.

Warum ein unverbrauchtes Budget ein Zeichen ist.

Deutlich unterschritten	Entweder ist das Ziel zu niedrig gesetzt, oder man liefert zu vorsichtig aus
Die Folgerung	Ein Budget soll verbraucht werden; höhere Verfügbarkeit als nötig kostet und nützt niemandem

Das klingt zunächst befremdlich und ist konsequent. Wenn Benutzer über eine Mobilfunkverbindung mit 99,5 Prozent Verfügbarkeit zugreifen, merkt niemand den Unterschied zwischen vier und fünf Neunen im Rechenzentrum.

(b) Wie sich ein Ausfall fortpflanzt.
Der Ablauf.

Zeit	Ereignis	Wirkung
t_0	Datenbank wird langsam	Antworten dauern 2 s statt 20 ms
t_1	Dienst A wartet	Fäden belegt
t_2	Fadenvorrat erschöpft	A nimmt nichts mehr an
t_3	Dienst B wartet auf A	dasselbe bei B
t_4	Alle Aufrufer wiederholen	Last verdreifacht
t_5	Datenbank noch langsamer	Rückkopplung

Beachten Sie, dass die Datenbank nie ausgefallen ist. Sie wurde langsam, und das genügte. Der Zusammenbruch entstand durch die Reaktion des Systems, nicht durch die ursprüngliche Störung.

Wo man den Kreis durchbricht.

Bei	Maßnahme	Wirkung
t_1	Frist von 200 ms	Fäden werden frei
t_2	Lastbegrenzung	A weist früh und schnell ab
t_3	Sicherungsschalter	B fragt A gar nicht mehr
t_4	Wiederholungs-budget	keine Verstärkung

Alle vier Maßnahmen laufen darauf hinaus, früh und deutlich zu scheitern statt spät und unklar. Ein schnelles Nein ist im verteilten System fast immer besser als ein langsames Vielleicht.

Die Lastbegrenzung bei t_2 verdient eine Anmerkung. Ein Dienst sollte eine Obergrenze für gleichzeitige Anfragen haben und darüber hinausgehende sofort abweisen. Damit bleibt er für die angenommenen Anfragen schnell, statt für alle langsam zu werden.

(c) Fehler absichtlich herbeiführen.

Der Gedanke. Ein System, dessen Verhalten im Fehlerfall nie erprobt wurde, verhält sich im Fehlerfall unbekannt.

Das Argument	Der Fehlerpfad ist der am wenigsten geprüfte Code und läuft genau dann, wenn es darauf ankommt
Die Folgerung	Man löst Fehler planmäßig aus, während man zusieht

Es ist dieselbe Überlegung wie bei der Fehlereinstreuung in Kerntreibern aus Band 5. Was nie ausgeführt wird, ist nie geprüft, und Fehlerbehandlung macht in vielen Systemen mehr Code aus als der Normalfall.

Wie man es verantwortlich macht.

1. Eine Vermutung aufschreiben: „Fällt ein Knoten aus, bleibt die Antwortzeit unter 200 ms.“
2. Den kleinstmöglichen Versuch wählen.
3. Zuerst in der Erprobungsumgebung, dann im Betrieb.
4. Einen Abbruchknopf bereithalten und die Auswirkung eingrenzen.
5. Zu Zeiten, in denen Menschen da sind.

Nicht	Zufällig Dinge kaputtmachen und schauen, was passiert
Sondern	Eine Annahme prüfen, die man ohnehin stillschweigend macht

Der Unterschied ist wesentlich. Wer nur Chaos stiftet, lernt, dass sein System kaputtgeht. Wer eine bestimmte Vermutung prüft, lernt, welche seiner Annahmen falsch war, und das ist die eigentlich nützliche Auskunft.

Die Erfahrung derer, die das betreiben, ist übereinstimmend: Die meisten Überraschungen betreffen nicht den Ausfall selbst, sondern die Erholung danach. Ein System, das einen Ausfall verkraftet, bricht beim gleichzeitigen Wiederanlauf aller Bestandteile zusammen.

Dean und Barroso[9] beschreiben das Verhalten großer Systeme unter Last; Birman[2] behandelt Zuverlässigkeit systematisch; Kleppmann[16] verbindet Entwurf und Betrieb.

14 Leistung im Verteilten

Vier Größen, die man auseinanderhalten muss

Verzögerung	Wie lange eine einzelne Anfrage braucht
Durchsatz	Wie viele Anfragen je Zeiteinheit erledigt werden
Auslastung	Welcher Anteil der Kapazität benutzt wird
Nebenläufigkeit	Wie viele Anfragen gleichzeitig unterwegs sind

Die ersten beiden werden ständig verwechselt. Ein System mit hohem Durchsatz kann sehr langsam antworten, und ein System mit kurzer Antwortzeit kann wenig schaffen. Wer die eine Größe verbessert, verschlechtert oft die andere.

Das Gesetz von Little.
Zwischen dreien der vier Größen besteht ein Zusammenhang, der ohne jede Annahme über die Verteilungen gilt:

$$L = \lambda \cdot W.$$

L	Mittlere Zahl der Anfragen im System
λ	Ankunftsrate
W	Mittlere Verweildauer

Der Satz ist bemerkenswert allgemein. Er verlangt weder eine bestimmte Ankunftsverteilung noch eine bestimmte Bedienreihenfolge. Er gilt für einen Server, für ein ganzes Rechenzentrum und für eine Warteschlange an der Kasse.

System	λ	W	L
Webdienst	5000/s	20 ms	100
Datenbank	800/s	5 ms	4
Stapel-verarbeitung	2/s	30 s	60

Die erste Zeile sagt: Es müssen im Mittel hundert Anfragen gleichzeitig in Bearbeitung sein. Wer nur fünfzig Verbindungen zulässt, begrenzt damit den Durchsatz auf die Hälfte, gleichgültig wie schnell die Rechner sind.

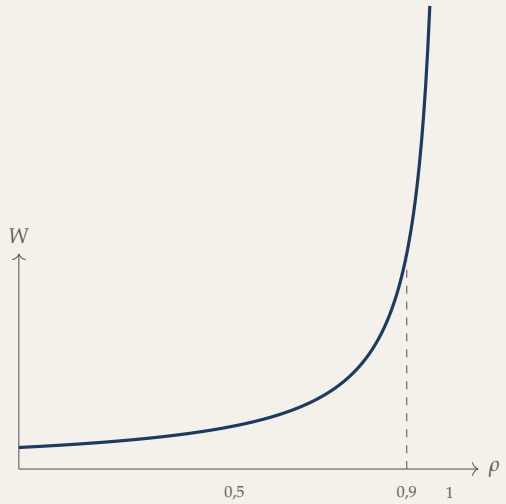
Warum die Wartezeit explodiert

Bei zufällig eintreffenden Anfragen wächst die Wartezeit nicht linear mit der Auslastung, sondern über alle Grenzen.
Für ein einfaches Wartesystem mit zufälliger Ankunft und exponentiell verteilter Bedienzeit gilt

$$W = \frac{S}{1 - \rho},$$

mit der Bedienzeit S und der Auslastung ρ.

ρ	Faktor	W bei $S = 10$ ms
0,10	1,11	11 ms
0,50	2,00	20 ms
0,80	5,00	50 ms
0,90	10,0	100 ms
0,95	20,0	200 ms
0,99	**100**	**1000 ms**



Der Sprung von 90 auf 99 Prozent Auslastung verzehnfacht die Wartezeit. Das ist der Grund, warum niemand ein System dauerhaft nahe an der Kapazitätsgrenze fährt, obwohl die Rechner dann am besten ausgenutzt wären.

Anwendung	Zielauslastung	Grund
Interaktiver Dienst	30–50 %	Antwortzeit
Stapelverarbeitung	80–95 %	Wartezeit egal
Notrufsystem	< 10 %	Spitzen abfangen

Die Formel gilt streng nur für zufällige Ankünfte. Bei gleichmäßigem Eintreffen entstünde gar keine Warteschlange. Der ganze Effekt kommt von den Schwankungen, nicht vom Mittelwert.

Die Schwanzlatenz

In großen Systemen bestimmt nicht der Mittelwert die Erfahrung des Benutzers, sondern das obere Ende der Verteilung.

Maß	Wert	Bedeutung
Mittelwert	12 ms	sagt wenig
Median	9 ms	die Hälfte ist schneller
p_{95}	40 ms	
p_{99}	180 ms	
$p_{99{,}9}$	**900 ms**	jeder Tausendste

Ein Mittelwert von zwölf Millisekunden klingt hervorragend und verbirgt, dass jede tausendste Anfrage fast eine Sekunde braucht. Bei Millionen von Anfragen am Tag sind das tausende unzufriedener Benutzer.

Warum sich das im Verteilten verschärft.
Eine einzelne Benutzeranfrage löst oft viele Teilanfragen aus, und sie ist erst fertig, wenn die *langsamste* zurück ist.

Teilanfragen	P(mind. eine langsam)	bei p_{99}
1	1,0 %	
10	9,6 %	
100	63 %	
500	**99,3** %	fast immer

Bei fünfhundert Teilanfragen wird das Verhalten des 99. Perzentils zum Normalfall. Was auf einem einzelnen Server ein seltener Ausreißer war, ist im Verteilten die Regel.

Was dagegen hilft

Abgesicherte Anfragen	Nach dem 95. Perzentil wird eine zweite Kopie an einen anderen Server geschickt; die erste Antwort zählt
Verzögertes Absichern	Erst warten, dann doppeln; kostet nur wenige Prozent Zusatzlast
Kleinere Einheiten	Viele kleine Aufgaben lassen sich besser ausgleichen als wenige große
Antwort ohne Nachzügler	Nach 99 von 100 Teilantworten weitermachen, wenn das Ergebnis es zulässt

Verfahren	p_{50}	$p_{99{,}9}$	**Zusatzlast**
Ohne	19 ms	1740 ms	0 %
Mit Absicherung	16 ms	**280 ms**	$\approx 2\,\%$

Faktor sechs am oberen Ende für zwei Prozent mehr Last. Das ist eine der wirksamsten Maßnahmen überhaupt, und sie funktioniert nur, weil die Ausreißer nicht am Verfahren liegen, sondern am zufälligen Zustand eines einzelnen Servers: eine Speicherbereinigung, ein Cachefehler, ein anderer Prozess auf derselben Maschine.

Die letzte Zeile der oberen Tabelle verdient eine Warnung. Sie funktioniert nur bei Leseanfragen ohne Nebenwirkung. Eine abgesicherte Schreibanfrage wird zweimal ausgeführt, und damit ist man wieder bei der Idempotenz aus Kapitel 12.

Aufgabe 14.1.

(a) Rechnen Sie mit dem Gesetz von Little eine Kapazität aus.
(b) Bestimmen Sie, wie sich Teilanfragen auf die Antwortzeit auswirken.
(c) Untersuchen Sie, wo Parallelisierung im Netz an ihre Grenze kommt.

Lösung Schritt für Schritt.

(a) Kapazität mit Little.
Die Aufgabe. Ein Dienst soll 20 000 Anfragen je Sekunde bei einer mittleren Antwortzeit von 50 ms bedienen. Wie viele gleichzeitige Bearbeitungen sind nötig?

$$L = \lambda \cdot W = 20\,000 \cdot 0{,}050 = \mathbf{1000}.$$

Die Aufteilung auf Server.
Ein Server bediene 200 gleichzeitige Anfragen.

$$\frac{1000}{200} = 5 \text{ Server, rechnerisch.}$$

Zuschlag für	Faktor	Ergebnis
Rechnerisch	1,0	5
Auslastungsgrenze 60 %	1,67	9
Ein Ausfall verkraftbar	+1	10
Tagesspitze	×1,8	**18**

Aus fünf werden achtzehn. Die drei Zuschläge sind keine Vorsicht, sondern folgen aus dem Vorigen: Die Auslastungsgrenze aus der Wartezeitformel, die Reserve aus der Ausfallwahrscheinlichkeit, die Spitze aus der Beobachtung.

Die Probe in der Gegenrichtung.
Bei achtzehn Servern und 20 000 Anfragen je Sekunde:

$$\rho = \frac{20\,000}{18 \cdot 200 / 0{,}050} = \frac{20\,000}{72\,000} = 0{,}28.$$

Achtundzwanzig Prozent Auslastung im Mittel, bei Tagesspitze etwa fünfzig. Das ist genau der Bereich, in dem die Wartezeitformel noch gutmütig ist.

(b) Teilanfragen und Antwortzeit.
Der Ansatz. Bei n unabhängigen Teilanfragen und einer Wahrscheinlichkeit q, dass eine einzelne langsam ist:

$$P(\text{mindestens eine langsam}) = 1 - (1 - q)^n.$$

Die Rechnung für $q = 0{,}01$.

n	$(1-q)^n$	P
1	0,990	1,0 %
5	0,951	4,9 %
10	0,904	9,6 %
50	0,605	39,5 %
100	0,366	63,4 %
500	0,0066	99,3 %

Was das für die Anforderung an die Einzelkomponente bedeutet.
Soll die Gesamtanfrage in 99 Prozent der Fälle schnell sein, so muss gelten

$$(1-q)^n \geq 0{,}99 \quad \Longrightarrow \quad q \leq 1 - 0{,}99^{1/n}.$$

n	**nötiges** q	**entspricht**
1	10^{-2}	p_{99}
10	10^{-3}	$p_{99{,}9}$
100	10^{-4}	$p_{99{,}99}$
500	$2 \cdot 10^{-5}$	$p_{99{,}998}$

Bei fünfhundert Teilanfragen muss jede einzelne Komponente an ihrem 99,998-Perzentil gemessen werden. Diese Anforderung ist mit gewöhnlichen Mitteln nicht zu erfüllen, und deshalb greift man zu den Verfahren aus dem Technikkasten oben.

Der Vergleich mit der Absicherung.
Mit zwei unabhängigen Kopien ist eine Anfrage nur dann langsam, wenn beide es sind:

$$q' = q^2 = 10^{-4}.$$

$$P(\text{mind. eine von 500 langsam}) = 1 - (1 - 10^{-4})^{500} = 4{,}9\,\%.$$

Von 99,3 auf 4,9 Prozent, allein durch Verdopplung der langsamsten fünf Prozent. Das ist die formale Fassung dessen, warum die Absicherung so gut wirkt.

(c) Wo Parallelisierung an ihre Grenze kommt.
Amdahl im Netz.
Band 4 zeigte, dass der sequentielle Anteil die Beschleunigung begrenzt. Im Verteilten kommt ein zweiter Anteil hinzu: die Kommunikation.

$$T(n) = \underbrace{\frac{T_1(1-s)}{n}}_{\text{parallel}} + \underbrace{T_1 s}_{\text{sequentiell}} + \underbrace{c \cdot n}_{\text{Abstimmung}} .$$

Der dritte Term ist der entscheidende Unterschied. Er wächst mit der Knotenzahl, statt zu fallen. Es gibt deshalb ein Optimum: Ab einer bestimmten Zahl von Knoten wird das System durch weitere Knoten langsamer.

Ein Zahlenbeispiel. $T_1 = 1000$ s, $s = 0{,}02$, $c = 0{,}5$ s je Knoten.

n	**parallel**	**seq.**	**Abstimm.**	$T(n)$
1	980	20	0,5	1000,5
10	98	20	5	123
40	24,5	20	20	**64,5**
100	9,8	20	50	79,8
200	4,9	20	100	124,9

Das Optimum liegt bei etwa vierzig Knoten. Bei zweihundert ist das System wieder so langsam wie bei zehn, und man hat zwanzigmal so viel Hardware bezahlt.

Das Optimum allgemein.
Ableiten und null setzen:

$$-\frac{T_1(1-s)}{n^2} + c = 0 \quad \Longrightarrow \quad n_{\text{opt}} = \sqrt{\frac{T_1(1-s)}{c}}.$$

$$n_{\text{opt}} = \sqrt{\frac{980}{0{,}5}} = \sqrt{1960} = 44{,}3.$$

Abstimmkosten c	n_{opt}
5 s	14
0,5 s	44
0,05 s	140
0,005 s	443

Die Wurzel ist die entscheidende Botschaft. Um die sinnvolle Knotenzahl zu verzehnfachen, muss man die Abstimmkosten auf ein Hundertstel senken. Deshalb steckt in großen verteilten Systemen so viel Aufwand darin, Kommunikation zu vermeiden, statt sie zu beschleunigen.

Die Rechnung unterschätzt das Problem

Sie unterstellt, dass die Abstimmkosten linear mit der Knotenzahl wachsen. Bei manchen Verfahren wachsen sie quadratisch.

Muster	Nachrichten	Beispiel
Rundsendung vom Führer	$\mathcal{O}(n)$	Raft
Jeder mit jedem	$\mathcal{O}(n^2)$	**PBFT**
Baumförmig	$\mathcal{O}(n)$, $\mathcal{O}(\log n)$ Tiefe	Aggregation
Gerüchteweise	$\mathcal{O}(n \log n)$	Zustandsabgleich

Die zweite Zeile erklärt, warum byzantinische Verfahren (Kapitel 10) selten mehr als einige Dutzend Teilnehmer haben. Bei hundert Knoten sind es bereits zehntausend Nachrichten je Runde.

Und sie unterstellt Unabhängigkeit. In Wirklichkeit teilen sich die Knoten Netzstrecken, Kühlung und Stromversorgung. Ein Ausfall trifft dann nicht einen Knoten, sondern ein ganzes Gestell, und alle Rechnungen über unabhängige Ausfälle sind hinfällig.

Dean und Barroso[9] beschreiben die Schwanzlatenz und ihre Behandlung; Kleppmann[16] behandelt die Messgrößen sauber; Tanenbaum und van Steen[28] ordnen die Skalierungsfragen ein.

15 Der Stand im Jahr 2026

Was gelöst ist und was nicht

Frage	Stand	Bemerkung
Paketvermittlung	gelöst	seit fünfzig Jahren
Zuverlässiger Transport	gelöst	TCP, QUIC
Konsens bei Absturzfehlern	gelöst	Paxos, Raft
Verteilte Datenhaltung	gelöst	für die üblichen Fälle
Staukontrolle	in Arbeit	**seit 1988 immer wieder**
Konsistenz und Verfügbarkeit	in Arbeit	bessere Zwischenstufen
Betrieb im Großen	in Arbeit	mehr Werkzeug als Theorie
Byzantinische Verfahren	in Arbeit	teuer, aber praktikabel
Konsens ohne Zeitannahmen	unmöglich	**Fischer, Lynch, Paterson**
Beides bei Netztrennung	unmöglich	CAP
Verteilte Fehlersuche	offen	das härteste Alltagsproblem

Die beiden ersten Zeilen des unteren Abschnitts sind keine Forschungslücken, sondern Sätze. Sie werden nicht gelöst werden. Was sich ändert, ist der Umgang mit ihnen.

Die letzte Zeile verdient eine Erläuterung.
Ein Fehler in einem verteilten System hinterlässt seine Spuren auf vielen Maschinen, in vielen Protokollen und ohne gemeinsame Zeitbasis.

	ein Rechner	verteilt
Zustand	einsehbar	nirgends vollständig
Zeit	eine Uhr	viele, ungenau
Reproduzieren	meist möglich	selten
Debugger	ja	nein

Der Ersatz für den Debugger heißt Ablaufverfolgung: Jede Anfrage bekommt eine Kennung, die durch alle Dienste weitergereicht wird, und am Ende setzt man die Teilstücke zusammen. Das ist mühsamer und die einzige Möglichkeit, die es gibt.

Fünf Entwicklungen

Erstens: Der Transport ist in den Benutzerbetrieb gewandert.
QUIC läuft auf UDP und damit vollständig in der Anwendung, nicht mehr im Betriebssystemkern.

	TCP	QUIC
Läuft im	Kern	**Benutzerbetrieb**
Änderung erfordert	Betriebssystem-wechsel	Bibliothek austauschen
Für Geräte in der Mitte	lesbar	verschlüsselt
Ströme	einer	viele, unabhängig
Verbindungs-wechsel	Abbruch	Kennung bleibt

Die zweite Zeile ist die eigentliche Wirkung. Eine Änderung an TCP braucht Jahrzehnte, weil sie in Milliarden von Betriebssystemen ankommen muss. Eine Änderung an QUIC braucht eine neue Programmfassung.

Die letzte Zeile löst ein altes Ärgernis: Wer vom WLAN ins Mobilfunknetz wechselt, behält seine Verbindung, weil sie nicht mehr über Adressen, sondern über eine eigene Kennung identifiziert wird.

Zweitens: Staukontrolle ist wieder ein Forschungsthema.

Jacobsons Verfahren von 1988 deutet jeden Paketverlust als Stau. Diese Annahme war richtig und ist es nicht mehr überall.

Verfahren	Signal	Schwäche
Reno, Cubic	Verlust	Puffer laufen voll
Vegas	Verzögerung	verliert gegen Verlustverfahren
BBR	**Bandbreite und Umlaufzeit**	Fairness umstritten
Mit Markierung	Signal vom Router	braucht Mitwirkung

Die erste Zeile beschreibt das Problem der überfüllten Puffer: Ein Verfahren, das erst bei Verlust bremst, füllt vorher jeden Puffer auf dem Weg. Die Verzögerung steigt dann auf ein Vielfaches, ohne dass der Durchsatz zunimmt.

Puffer	Durchsatz	Verzögerung	
Klein	95 %	**15 ms**	gut
Angemessen	100 %	40 ms	
Zu groß	100 %	1200 ms	

Der große Puffer bringt keinen zusätzlichen Durchsatz und kostet mehr als eine Sekunde Verzögerung. Er wurde eingebaut, weil Speicher billig ist und weil man Verluste vermeiden wollte. Beides war gut gemeint.

Drittens: Konsistenz wird feiner abgestuft.
Die alte Wahl zwischen starker und letztendlicher Konsistenz ist einer Reihe von Zwischenstufen gewichen.

Eigene Schreibvorgänge lesen	Man sieht wenigstens, was man selbst geschrieben hat
Monoton lesend	Man sieht nie einen älteren Stand als vorher
Kausal	Was ursächlich zusammenhängt, erscheint in der richtigen Reihenfolge
Konfliktfreie Datentypen	Kopien laufen auseinander und lassen sich ohne Absprache zusammenführen

Die kausale Konsistenz ist die stärkste, die sich bei einer Netztrennung noch aufrechterhalten lässt. Sie beruht unmittelbar auf Lamports Beziehung „geschieht vor“ aus Kapitel 7 und ist damit fast fünfzig Jahre alt.

Die letzte Zeile hat eine reizvolle Eigenschaft: Wenn die Zusammenführung mathematisch verträglich ist, kommen alle Kopien am Ende beim selben Ergebnis an, ohne dass je ein Konsens nötig war. Man umgeht das Problem, statt es zu lösen.

Viertens: Der Betrieb ist zur eigenen Disziplin geworden.

Verfügbarkeitsziele	Man legt fest, wie viel Ausfall erlaubt ist, statt null anzustreben
Fehlerbudget	Wird es nicht verbraucht, ist man zu vorsichtig; ist es aufgebraucht, wird nichts Neues ausgeliefert
Absichtliche Störungen	Man schaltet Teile im laufenden Betrieb ab, um zu sehen, ob es hält
Ablaufverfolgung	Jede Anfrage trägt eine Kennung durch alle Dienste

Die zweite Zeile ist der interessanteste Gedanke. Ein Fehlerbudget verwandelt Zuverlässigkeit von einem Streitpunkt zwischen Entwicklung und Betrieb in eine gemeinsame, messbare Größe. Wer es nicht ausschöpft, hat zu wenig gewagt.

Fünftens: Die Rückkehr des einzelnen Rechners.
Eine Gegenbewegung zur Zerlegung in viele kleine Dienste hat sich etabliert.

Jahr	Kerne	Speicher	Netz
2005	2	4 GB	1 Gbit/s
2015	24	128 GB	10 Gbit/s
2026	256	**6 TB**	400 Gbit/s

Sehr viele Aufgaben, für die man 2015 einen Verbund gebaut hätte, passen heute auf eine einzige Maschine. Und ein Aufruf innerhalb eines Prozesses kostet Nanosekunden statt Millisekunden und kann nicht zur Hälfte gelingen.

Verteilen, wenn	es nicht auf eine Maschine passt, ein Maschinenausfall überstanden werden muss, oder die Nähe zu Benutzern zählt
Nicht verteilen, weil	es modern ist, weil Teams getrennt arbeiten wollen, oder weil eine Vorlage es so vorsieht

Der zweite Grund in der ersten Zeile ist der ehrlichste. Wer Ausfallsicherheit braucht, braucht mindestens zwei Maschinen, und ab da gelten alle Sätze dieses Bandes.

Drei Einsichten

Erstens: Die Unmöglichkeitssätze sind der Kern.

Satz	sagt	Folge für die Praxis
FLP	Konsens braucht Zeitannahmen	**Zeitüberschreitungen**
CAP	Bei Trennung nur eines	bewusste Wahl
Zwei Generäle	Sichere Einigung unmöglich	Wiederholung, Idempotenz

Alle drei sagen dasselbe von verschiedenen Seiten: Ohne Annahmen über die Zeit lässt sich in einem Netz nichts Sicheres feststellen. Jede Zeitüberschreitung in jedem verteilten System ist eine praktische Antwort auf einen dieser Sätze.

Zweitens: Das Netz kennt nur eine Ordnung, und die ist unvollständig.

Was gesichert ist	Wenn *a* vor *b* geschah und es einen Nachrichtenpfad gibt, dann steht das fest
Was nicht gesichert ist	Bei allem Übrigen gibt es keine Reihenfolge, nur nebenläufige Ereignisse

Das ist der Inhalt des Titelbilds dieses Bandes. Uhren helfen nicht weiter, weil sie auseinanderlaufen; nur Nachrichten stiften Ordnung. Wer eine totale Reihenfolge braucht, muss sie durch zusätzliche Kommunikation herstellen, und die kostet Umläufe.

Drittens: Teilausfall ist der Normalzustand.

Knoten	***P*(alle laufen)**	**bei 99,9 % je Knoten**
1	99,9 %	
10	99,0 %	
100	90,5 %	
1000	**36,8** %	meist ist einer weg

Bei tausend Knoten ist der vollständig gesunde Zustand die Ausnahme. Ein System, das nur dann richtig arbeitet, wenn alles läuft, arbeitet also fast nie richtig. Es muss so gebaut sein, dass der Teilausfall der erwartete Fall ist und nicht der Ausnahmefall.

Band 10 wendet sich den Datenbanken und dem Wiederfinden von Information zu. Vieles aus diesem Band kehrt dort wieder: Transaktionen, Kopien, Aufteilung, und die Frage, welche Zusagen ein System überhaupt geben kann.

Es gibt nur zwei schwierige Probleme in verteilten Systemen:
2. genau einmal zustellen, 1. die richtige Reihenfolge,
und 2. genau einmal zustellen.

— ein umlaufender Scherz unbekannter Herkunft.
Die durcheinandergeratene Nummerierung ist der Witz:
Sie führt beide Probleme zugleich vor.

Part IV

Anhang

A Zahlen und Formeln

Verzögerungen im Überblick

Vorgang	Zeit	Verhältnis
Funktionsaufruf	1 ns	1
Hauptspeicherzugriff	65 ns	65
SSD-Zugriff	100 µs	10^5
Im selben Gestell	0,2 ms	$2 \cdot 10^5$
Im selben Rechenzentrum	0,5 ms	$5 \cdot 10^5$
Frankfurt–Amsterdam	10 ms	10^7
Frankfurt–New York	80 ms	$8 \cdot 10^7$
Frankfurt–Sydney	250 ms	$2{,}5 \cdot 10^8$
Geostationärer Satellit	600 ms	$6 \cdot 10^8$

Zwischen der ersten und der letzten Zeile liegen achteinhalb Größenordnungen. Wer eine Optimierung erwägt, sollte zuerst feststellen, in welcher Zeile er steht.

Kenngrößen und Formeln

Little	$L = \lambda W$
Wartezeit	$W = \dfrac{S}{1-\rho}$
Bandbreiten-Verzögerungs-Produkt	BDP = Rate × Umlaufzeit
TCP-Durchsatz	$\approx \dfrac{\text{MSS}}{\text{RTT}\sqrt{p}}$
Quorum	$R + W > N$
Konsens bei Absturz	$n \geq 2f + 1$
Konsens bei Lüge	$n \geq 3f + 1$
Optimale Knotenzahl	$n_{\text{opt}} = \sqrt{T_1(1-s)/c}$

Das Bandbreiten-Verzögerungs-Produkt sagt, wie viele Daten gleichzeitig unterwegs sein müssen, damit eine Leitung ausgelastet ist. Bei 1 Gbit/s und 80 ms sind das 10 MB. Wer ein Sendefenster von 64 KB benutzt, erreicht damit ein halbes Prozent der Leitungsgeschwindigkeit.

Rate	RTT	BDP	Fenster
100 Mbit/s	10 ms	125 KB	nötig
1 Gbit/s	80 ms	10 MB	**Skalierung nötig**
10 Gbit/s	80 ms	100 MB	

Verfügbarkeit

Neunen	Anteil	je Jahr	je Monat
99 %	zwei	3,65 Tage	7,2 h
99,9 %	drei	8,76 h	43 min
99,99 %	vier	52,6 min	4,3 min
99,999 %	fünf	5,26 min	26 s

In Reihe geschaltete Komponenten multiplizieren ihre Verfügbarkeiten. Fünf Dienste mit je vier Neunen ergeben zusammen 99,95 Prozent, also weniger als dreieinhalb Neunen. Jede zusätzliche Abhängigkeit senkt das Ergebnis.

B Landkarte des Bandes

Wo was steht

Kap.	Thema	Verweise
1	Schichten	Band 6 (Zwischenform)
2	Untere Schichten	Band 7 (Fehlererkennung)
3	Adressen, Wegewahl	Band 1 (kürzeste Wege)
4	Transport	Band 5 (Puffer)
5	Namen, Anwendungen	Band 8 (TLS)
6	Was es schwer macht	Band 5 (Teilausfall)
7	Zeit und Ordnung	**Band 7 (Lamport 1978)**
8	Kopien, Konsistenz	Band 10 (Transaktionen)
9	Konsens	Band 2 (Unmöglichkeit)
10	Byzantinische Fehler	Band 8 (Signaturen)
11	Daten verteilen	Band 1 (Streuung), Band 10
12	Nachrichten, Ströme	Band 5 (Dauerhaftigkeit)
13	Ausfälle, Betrieb	Band 5 (Zeitüberschreitungen)
14	Leistung	Band 4 (Amdahl)
15	Stand 2026	Band 10

Sechs Verbindungen zur Mathematikreihe

Wegewahl	Graphen, kürzeste Wege, Fixpunkte
Warteschlangen	Stochastische Prozesse, Poisson
Schwanzlatenz	Ordnungsstatistiken, Extremwerte
Vektoruhren	Halbordnungen, Verbände
Quoren	Kombinatorik der Schnittmengen
Konfliktfreie Datentypen	Halbverbände, monotone Abbildungen

Die letzte Zeile ist die elegante Verbindung des Bandes. Wenn die Zusammenführung zweier Kopien einem Verbandsverknüpfung entspricht, also verträglich, vertauschbar und selbstaufhebend ist, dann konvergieren alle Kopien zwangsläufig, ohne jede Absprache.

C Glossar

Begriffe

Ausfalldetektor	Verdachtsäußerung darüber, wer nicht mehr antwortet
BDP	Wie viele Daten gleichzeitig unterwegs sein müssen
CAP	Bei Netztrennung nur Konsistenz oder Verfügbarkeit
CRDT	Datentyp, dessen Kopien sich ohne Absprache vereinen
Geschieht vor	Die einzige gesicherte Ordnung im Netz
Idempotenz	Zweite Ausführung ändert nichts
Kapselung	Jede Schicht hängt einen Kopf an
Konsens	Alle einigen sich auf denselben Wert
Linearisierbarkeit	Alles verhält sich wie auf einem Rechner
Nebenläufig	Kein Ereignis geschah vor dem anderen
Netztrennung	Zwei Teile können nicht miteinander reden
Quorum	Mehrheit, die für alle entscheidet
Schwanzlatenz	Das obere Ende der Antwortzeitverteilung
Sanduhr	Viele unten, viele oben, IP in der Mitte
Teilausfall	Ein Teil läuft, ein Teil nicht
Vektoruhr	Zähler je Knoten; erkennt Nebenläufigkeit
Zurückweichen	Wachsende, zufällige Wartezeit vor der Wiederholung

D Zeittafel

Von 1964 bis 2026

1964	Baran, Davies: Paketvermittlung
1969	ARPANET, erste Nachricht `LO`
1973	Metcalfe: Ethernet
1974	**Cerf, Kahn: TCP**
1976	Metcalfe, Boggs: Ethernet-Aufsatz
1978	**Lamport: Zeit, Uhren, Ordnung**
1978	Zwei-Phasen-Commit
1981	IPv4
1982	**Lamport u. a.: Byzantinische Generäle**
1983	ARPANET stellt auf TCP/IP um
1984	**Saltzer, Reed, Clark: Ende-zu-Ende**
1984	Mockapetris: DNS
1985	**Fischer, Lynch, Paterson: FLP**
1986	Netzzusammenbruch bei Berkeley
1988	**Jacobson: Staukontrolle**
1989	Berners-Lee: Web
1990er	Die acht Trugschlüsse
1996	Chandra, Toueg: Ausfalldetektoren
1997	Karger u. a.: konsistentes Hashing
1998	**Lamport: Paxos**
1999	Castro, Liskov: PBFT

2000	**Brewer: CAP-Vermutung**
2002	Gilbert, Lynch beweisen CAP
2004	MapReduce
2006	Bigtable, Chubby
2007	**Dynamo**
2007	Chandra u. a.: Paxos Made Live
2008	Bitcoin
2011	Shapiro u. a.: CRDTs
2012	**Spanner: Uhren mit Fehlerschranke**
2012	Brewer: CAP zwölf Jahre später
2013	**Dean, Barroso: Die Schwanzlatenz**
2014	Ongaro, Ousterhout: Raft
2015	Kubernetes 1.0
2016	BBR-Staukontrolle
2018	TLS 1.3
2021	**QUIC wird genormt**
2024	Markierung statt Verlust verbreitet sich
2026	Rückkehr zum großen einzelnen Rechner

E Kommentierte Quellen

[1] Bailis, P., Ghodsi, A.: *Eventual Consistency Today: Limitations, Extensions, and Beyond*. ACM Queue 11(3), 2013.

Die klarste kurze Darstellung der schwächeren Konsistenzmodelle. Räumt mit dem Missverständnis auf, letztendliche Konsistenz sei einfach nur Konsistenz mit Verzögerung, und zeigt, welche Zwischenstufen es gibt.

[2] Birman, K.: *Guide to Reliable Distributed Systems*. Springer 2012.

Behandelt Gruppenkommunikation, Ausfalldetektoren und Zuverlässigkeit systematisch. Birman hat das Gebiet mit dem ISIS-System mitbegründet, und man merkt dem Buch die Betriebserfahrung an.

[3] Brewer, E.: *CAP Twelve Years Later: How the Rules Have Changed*. IEEE Computer 45(2), 2012.

Der Urheber der CAP-Vermutung korrigiert die verbreitete Vergröberung: Es geht nicht um zwei von drei, sondern um eine Entscheidung, die nur während einer Netztrennung ansteht. Lesenswert gerade wegen der Selbstkorrektur.

[4] Castro, M., Liskov, B.: *Practical Byzantine Fault Tolerance*. OSDI 1999.

Machte aus einem theoretischen Ergebnis von 1982 ein einsetzbares Verfahren. Der Aufwand von $\mathcal{O}(n^2)$ Nachrichten je Runde blieb, und genau daran hängt bis heute die Größenbeschränkung solcher Systeme.

[5] Cerf, V., Kahn, R.: *A Protocol for Packet Network Intercommunication*. IEEE Trans. Comm. 22(5), 1974.

Der Aufsatz, aus dem TCP/IP wurde. *Der entscheidende Gedanke ist nicht das Protokoll, sondern die Entscheidung, bestehende Netze unverändert zu lassen und sie über eine gemeinsame Schicht zu verbinden.*

[6] Chandra, T., Griesemer, R., Redstone, J.: *Paxos Made Live — An Engineering Perspective*. PODC 2007.

Ein ungewöhnlich ehrlicher Bericht darüber, was beim Bau eines Konsenssystems tatsächlich schwierig war: nicht der Algorithmus, sondern Plattenfehler, Zustandsverwaltung, Prüfbarkeit. Pflichtlektüre für jeden, der so etwas bauen will.

[7] Chandra, T., Toueg, S.: *Unreliable Failure Detectors for Reliable Distributed Systems*. Journal of the ACM 43(2), 1996.

Klärte, welche Zusatzannahme genau nötig ist, um FLP zu umgehen. Der schwächste Detektor, mit dem Konsens noch möglich ist, ist ein bemerkenswert präzises Ergebnis.

[8] Corbett, J., u. a.: *Spanner: Google's Globally-Distributed Database*. OSDI 2012.

Beschreibt eine Datenbank, die weltweit verteilt ist und trotzdem strenge Konsistenz zusagt. Der Kunstgriff ist TrueTime: Uhren mit einer bekannten Fehlerschranke, und man wartet diese Schranke einfach ab.

[9] Dean, J., Barroso, L.: *The Tail at Scale*. CACM 56(2), 2013.

Erklärt, warum in großen Systemen das obere Ende der Verteilung zählt und nicht der Mittelwert. Die beschriebenen Gegenmittel, vor allem die abgesicherten Anfragen, gehören zum Handwerkszeug. Acht Seiten, sehr lesenswert.

[10] DeCandia, G., u. a.: *Dynamo: Amazon's Highly Available Key-value Store*. SOSP 2007.

Die Arbeit, die Verfügbarkeit ausdrücklich über Konsistenz stellte und damit eine ganze Generation von Systemen prägte. Enthält konsistentes Hashing, Vektoruhren und Quoren an einem realen Beispiel.

[11] Fischer, M., Lynch, N., Paterson, M.: *Impossibility of Distributed Consensus with One Faulty Process*. Journal of the ACM 32(2), 1985.

Der wichtigste Unmöglichkeitssatz des Fachs. *In einem asynchronen System genügt ein einziger möglicher Ausfall, um Konsens unmöglich zu machen. Der Beweis ist kurz und lohnt die Mühe.*

[12] Gilbert, S., Lynch, N.: *Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services*. SIGACT News 33(2), 2002.

Machte aus Brewers Vermutung einen Satz. Die genaue Formulierung ist wichtiger als die verbreitete Kurzfassung: Es geht um Linearisierbarkeit und um das Verhalten während einer Trennung.

[13] Gray, J., Reuter, A.: *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann 1993.

Das Standardwerk zu Transaktionen, verteilten Festschreibungen und Wiederanlauf. Über tausend Seiten, gründlich und aus jahrzehntelanger Praxis geschrieben. Auch für Band 10 die maßgebliche Quelle.

[14] Jacobson, V.: *Congestion Avoidance and Control*. SIGCOMM 1988.

Die Antwort auf den Netzzusammenbruch von 1986. Führte langsamen Start, Stauvermeidung und die Neuberechnung der Zeitüberschreitung ein. Fast alles, was TCP heute tut, steht in diesen sechzehn Seiten.

[15] Karger, D., u. a.: *Consistent Hashing and Random Trees*. STOC 1997.

Löste ein Problem, das beim Hinzufügen und Entfernen von Knoten entsteht: Bei gewöhnlicher Streuung müssen fast alle Daten umziehen, hier nur ein Anteil von $1/n$. Grundlage jeder verteilten Datenhaltung.

[16] Kleppmann, M.: *Designing Data-Intensive Applications*. O'Reilly 2017.

Die beste Verbindung von Theorie und Praxis, die es zu diesem Gebiet gibt. *Behandelt Kopien, Aufteilung, Transaktionen, Konsens und Datenströme, jeweils mit den Originalarbeiten im Rücken und mit klarem Urteil. Wenn Sie ein Buch lesen, dann dieses.*

[17] Kurose, J., Ross, K.: *Computer Networking: A Top-Down Approach*. 8. Aufl., Pearson 2020.

Das verbreitetste Lehrbuch zu Netzen, und zu Recht. Beginnt bei den Anwendungen und arbeitet sich nach unten, was didaktisch klug ist. Viele durchgerechnete Beispiele.

[18] Lamport, L.: *Time, Clocks, and the Ordering of Events in a Distributed System*. CACM 21(7), 1978.

Der Gründungsaufsatz des verteilten Rechnens. *Die Beziehung „geschieht vor“ und die logischen Uhren stehen hier. Sieben Seiten, die man einmal im Original gelesen haben sollte. Der meistzitierte Aufsatz des Gebiets.*

[19] Lamport, L.: *The Part-Time Parliament*. ACM TOCS 16(2), 1998.

Paxos, eingekleidet in eine erfundene Geschichte über ein griechisches Parlament. Die Verpackung hat die Rezeption um Jahre verzögert; Lamport hat später eine nüchterne Kurzfassung nachgeliefert, die man zuerst lesen sollte.

[20] Lamport, L., Shostak, R., Pease, M.: *The Byzantine Generals Problem*. ACM TOPLAS 4(3), 1982.

Stellte die Frage, was gilt, wenn Beteiligte nicht nur ausfallen, sondern lügen dürfen. Das Ergebnis $n \geq 3f + 1$ ist scharf, und die untere Schranke ist ebenso lehrreich wie das Verfahren.

[21] Metcalfe, R., Boggs, D.: *Ethernet: Distributed Packet Switching for Local Computer Networks*. CACM 19(7), 1976.

Ein gemeinsames Kabel, auf das alle senden, mit zufälligem Zurückweichen bei Kollisionen. Der Entwurf galt als naiv und hat alle durchdachteren Konkurrenten überlebt.

[22] Mockapetris, P.: *Domain Names — Concepts and Facilities*. RFC 1034, 1987.

Der Entwurf des Namenssystems. Bemerkenswert ist die Delegation: Niemand muss alle Namen kennen, jeder kennt nur, wen er als Nächstes fragen muss. Eine der wenigen Stellen, an denen das Internet wirklich hierarchisch aufgebaut ist.

[23] Ongaro, D., Ousterhout, J.: *In Search of an Understandable Consensus Algorithm*. USENIX ATC 2014.

Raft, mit dem ausdrücklichen Ziel entworfen, verständlicher zu sein als Paxos. Der Aufsatz enthält sogar eine Untersuchung darüber, ob das gelungen ist. Der beste Einstieg in Konsensverfahren.

[24] Peterson, L., Davie, B.: *Computer Networks: A Systems Approach*. 6. Aufl., Morgan Kaufmann 2021.

Das Gegenstück zu Kurose und Ross, von unten nach oben aufgebaut und mit mehr Gewicht auf den Entwurfsgründen. Frei verfügbar. Wer beide liest, hat das Gebiet.

[25] Saltzer, J., Reed, D., Clark, D.: *End-To-End Arguments in System Design*. ACM TOCS 2(4), 1984.

Der Grundsatz, der entscheidet, welche Aufgabe in welche Schicht gehört. Elf Seiten, die weit über die Netztechnik hinaus wirken und in jedem Systementwurf anwendbar sind.

[26] Shapiro, M., u. a.: *Conflict-Free Replicated Data Types*. SSS 2011.

Datentypen, deren Kopien auseinanderlaufen dürfen und trotzdem garantiert wieder zusammenfinden. Die Bedingung ist algebraisch, nicht algorithmisch, und deshalb ist das Ergebnis so stark: Man braucht überhaupt keinen Konsens.

[27] Stevens, W. R.: *TCP/IP Illustrated, Volume 1*. 2. Aufl., Addison-Wesley 2011.

Beschreibt die Protokolle bis auf die Bitebene, mit echten Mitschnitten. Wer wissen will, was tatsächlich über die Leitung geht, findet es hier und sonst nirgends so gründlich.

[28] Tanenbaum, A., van Steen, M.: *Distributed Systems*. 4. Aufl., 2023.

Die systematische Übersicht über das Gebiet: Modelle, Uhren, Konsistenz, Fehlertoleranz, Sicherheit. Frei verfügbar. Meinungsfreudig geschrieben, wie man es von Tanenbaum kennt (Band 5).

Wenn Sie nur drei Bücher kaufen

[16]	**Kleppmann**	Zum Bauen. Verbindet Theorie und Praxis besser als alles andere
[17]	**Kurose/Ross**	Zum Lernen der Netzseite. Von oben nach unten, mit vielen Beispielen
[28]	**Tanenbaum/van Steen**	Zum Überblick über die verteilte Seite. Frei verfügbar

Wer danach noch etwas möchte: [18] Lamport und [25] Saltzer, Reed und Clark sind zusammen achtzehn Seiten und gehören zum Bildungskanon. [9] Dean und Barroso ändern die Art, wie man auf Messwerte sieht.